



Das ist ein Unity-Projektordner mit einem Locomotion und Interaction Parcour. Ich soll diesen Parcour mit einer Individuellen lösung für locomotion und interaction lösen. Was sind deine generellen Inputs zum parcour und strukturelle anmerkungen? Wenn du dich mit dem grundgerüst vertraut gemacht hast, können wir über meine folgeideen im detail reden.

Der Parcour ist als „Testbed“ gedacht: Du sollst deine eigene Locomotion (und in der neueren Variante auch Object-Interaction) einhängen und dann im Level möglichst schnell möglichst viele Coins einsammeln. In deinem Ordner heißt das Projekt „VR-locomotion-parkour-try2“. ^[1] ^[2]

Was der Parcour „prüft“

- Ob deine Locomotion **präzise** genug ist für enge Plattformen/Abstände (Fehler-Toleranz, Bremsen/Stoppen, Richtungswechsel).
- Ob deine Lösung robuste Komfort-Defaults hat (z.B. stabile Horizon/Rotation, sauberes Re-Centering, klare Feedbacks bei Locomotion-Locks).
- Ob Interaction „parcour-tauglich“ ist: schnelle Objektaufnahme/-ablage ohne Fummeln, klare Zustände (Hover/Select/Use) und kein Konflikt mit Locomotion.

Struktur für eigene Locomotion

Das Grundgerüst erwartet explizit, dass du deine Locomotion in `LocomotionTechnique.cs` implementierst. Ich würde das dort nur als „Adapter/Entry Point“ sehen und deine echte Logik in ein eigenes Modul auslagern (z.B. `ILocomotionMode` + State-Machine), damit du Varianten (Smooth, Teleport, Dash, Climb, „Parkour“-Moves) sauber vergleichen kannst. ^[2]

Praktisch bewährt:

- Klare Zustände: Idle, Aiming, Moving, Cooldown, Blocked (z.B. beim Greifen/Klettern).
- Harte Prioritäten: „Interaction hält Locomotion an“ (oder umgekehrt) statt Mischzustände.
- Messpunkte: Zeit, Coins, Fehltritte/Resets, Motion-Sickness-Mitigations an/aus (für spätere Evaluation).

Struktur für Interaction

Da die 2022er Version explizit „Object Interaction Task“ erwähnt, lohnt sich eine Trennung in „Greifen“ vs. „Benutzen“ (Grab/Use) und ein eigener Konflikt-Resolver zwischen Interaction und Locomotion. Als Strukturhinweis: Lege eine zentrale „Player Rig“-Schicht fest (Input → Intent → Locomotion/Interaction), damit du nie direkt aus einzelnen Hand-Scripts den Player-Root verschiebst. ^[2]

Konkrete Structural Checks, die ich in Projekten immer mache:

- Ein einziger „Owner“ für Player-Transform/CharacterController/Rigidbody (sonst Jitter/Desync).
- Interaction-Layer/Masken strikt definieren (was ist grabbable, was ist nur triggerbar, was ist environment).
- Ein einheitliches Feedback-System (Haptik/Sound/VFX) statt verteilt in jedem Interactable.

Technische Stolpersteine im Grundsetup

Das Projekt ist auf Unity 6000.0.23f1 LTS sowie Meta XR All-in-One SDK 69.0.1 und Oculus XR Plugin 4.3.0 ausgelegt—das ist gut, aber du solltest darauf achten, dass du nicht parallel „zweite“ XR-Stacks (z.B. doppelte Input/Locomotion-Komponenten) aktivierst. In den Hinweisen stehen außerdem typische Quest-Settings (Linear Color Space, OpenGL ES 3.0, IL2CPP, ARM64), die du bei Builds konsistent lassen solltest, damit Performance/Frametime nicht deine Locomotion verfälscht. ^[2]

Damit wir deine Folgeideen sauber planen

Damit ich wirklich strukturell zum Parcour selbst (Scene-Hierarchie, Interactables, Trigger-Logik, Coin-System) Feedback geben kann, fehlen mir in den Attachments noch die eigentlichen Assets/-Skripte und die Scene-Datei(en). In der nächsten Nachricht: Welche Headset-Zielplattform (Quest 2/3/Pro oder PCVR) und kannst du `LocomotionTechnique.cs` sowie die wichtigsten Interaction-Skripte (und die Hauptszene `.unity`) hier anhängen?



Der projektordner ist vollständig. nicht angezeigte dateien existieren noch nicht. ich arbeite mit der quest 3s. ich kann dir screenshots aus der scene schicken. es handelt dich um einen rundparcours, bei dem man münzen einsammeln und gelegentlich eine aufgabe lösen muss, bei dem man eine form korrekt in ein förmchen einsetzen muss.

Ich würde gerne als erstes ein menü interface hinzufügen, dass den wechsel zwischen verschiedenen locomotion und interaction-techniques ermöglicht. wie ist da die effizienteste herangehensweise?

Die effizienteste Herangehensweise ist: Du baust ein kleines „Technique Framework“ (Registry + Interfaces + Manager) und lässt das Menü nur noch eine Auswahl setzen, statt direkt Locomotion-/Interaction-Objekte in der UI zu verdrahten. Der Repo ist genau darauf ausgelegt, dass du Techniken austauschbar implementierst (z.B. Locomotion in `LocomotionTechnique.cs`).
^[10]

Zielbild (sauber & schnell)

- 1x `TechniqueManager` im `PlayerRig`, der Locomotion und Interaction als „Plugins“ lädt/aktiviert/deaktiviert.
- Jede Technique ist ein Prefab (oder Component), die ein gemeinsames Interface implementiert (z.B. `ILocomotionTechnique`, `IInteractionTechnique`).
- 1x `TechniqueRegistry` als `ScriptableObject`, das die verfügbaren Techniken (Name, Prefab, optional Beschreibung/Komfort-Flags) listet.

So bleibt dein Rundparcours unverändert, und du kannst später ohne Umbau neue Varianten hinzufügen.

Konkreter Aufbau (Unity)

1. Interfaces definieren

- `ILocomotionTechnique`: `Initialize(rig)`, `Enable()`, `Disable()`, optional `CanMove/IsBusy`.
- `IInteractionTechnique`: analog, plus optional `SetDominantHand`, `SetRayMode`, etc.

2. Technique als Prefab

- Jede Technique ist ein Prefab mit genau einer Root-Komponente (z.B. `DashLocomotionTechnique : MonoBehaviour, ILocomotionTechnique`).
- Der Manager instanziiert das Prefab als Child vom `PlayerRig` (damit Referenzen wie Head/Hands leicht zu finden sind), und toggelt über `Enable/Disable` statt ständig `Destroy/Instantiate` (besser für Iteration und Performance-Stabilität auf Quest).

3. Registry (ScriptableObject)

- Felder: `string displayName`, `GameObject prefab`, optional `bool comfortMode`, `Sprite icon`.
- Vorteil: Das Menü kann dynamisch aus der Registry gebaut werden (keine manuelle UI-Pflege).

4. Manager

- Hält `currentLocomotion`, `currentInteraction`.
- API: `SetLocomotion(id)` / `SetInteraction(id)`.
- Verantwortlich für Konflikte: Wenn Interaction „busy“ (Greifen/Einsetzen) → Locomotion blocken oder nur Snap-Turn erlauben (eine einzige Stelle, die Regeln kennt).

Menü-Implementierung (minimal)

- World-Space Canvas am linken Handgelenk oder als Start-Panel in der Szene.
- UI: 2 Dropdowns (Locomotion / Interaction) + „Apply“ + „Reset Run“.
- Menü spricht **nur** mit `TechniqueManager` (keine direkten Referenzen zu einzelnen Techniques).

Optional für den Parcours:

- Speichere Auswahl via `PlayerPrefs` (oder kleines Save-JSON), damit du beim Testen schneller iterierst.
- Zeige im Menü pro Technique 1–2 „Constraints“ an (z.B. „Benötigt freie Hände“, „Kein Teleport beim Puzzle“).

Typische Fallstricke (und wie du sie vermeidest)

- „Mehrere Scripts bewegen den Player“: Stelle sicher, dass nur die aktive Locomotion den `PlayerRoot` bewegt (Manager setzt die anderen hart auf `Disable()`).
- „Interaction kollidiert mit Locomotion“: Der Manager ist die einzige Instanz, die entscheiden darf, ob Bewegung gerade erlaubt ist (z.B. beim Formen-Puzzle).
- „UI Input in VR nervt“: Wenn du nicht schon ein Ray/Pointer-UI Setup hast, nimm für's Erste große Buttons + Wrist-Menu + Ray aus der Interaction-Technique (dann bleibt alles in einem System).

Welche Locomotion-Varianten willst du als erstes anbieten (z.B. Smooth, Teleport, Dash, Arm-Swing, Grapple), und soll Interaction immer „Grab + Ray UI“ können oder willst du UI getrennt lösen?



Das ist die aktuelle hierarchie, kannst du diese bitte einmal sauber formatiert aufschreiben und mich dann schritt für schritt durch den vorgeschlagenen änderungsprozess führen?

Hier ist die Hierarchie aus deinem Screenshot, und danach ein konkreter, effizienter Ablauf, um ein Menü zum Wechseln von Locomotion- und Interaction-Techniques einzubauen, ohne den Parcours umzubauen.^[19]

Aktuelle Hierarchie

Aktuelle Scene-Hierarchie (wie im Screenshot zu sehen):^[19]

```
VR-locomotion-parkour
├── Environment
│   ├── Directional Light
│   ├── terrain
│   ├── decorations
│   ├── FirstCoins
│   ├── SecondCoins
│   ├── FinalCoins
│   ├── StartBanner
│   ├── FirstBanner
│   ├── SecondBanner
│   ├── FinalBanner
│   ├── ObjectInteractionInitiator1
│   ├── ObjectInteractionInitiator2
│   └── ObjectInteractionInitiator3
├── ParkourControl
│   ├── Start2FirstRespawn
│   ├── First2SecondRespawn
│   ├── Second2FinalRespawn
│   ├── BackgroundMusic
│   └── EndSoundEffect
├── EventSystem
├── TaskUI
│   ├── TaskStart
│   └── Done
├── OVRCameraRigInteraction
│   └── OVRCameraRig
```

(Die beiden OVR-Rig-Objekte sind im Screenshot als eigener Block am Ende zu sehen, wobei OVRCameraRig unter OVRCameraRigInteraction hängt.)^[19]

Zielstruktur (kleinste Änderung)

Du lässt Environment, ParkourControl, TaskUI und die Coins/Respawns unangetastet und ergänzt nur eine „Player/Systems“-Schicht neben dem OVR-Rig. Der Kern ist ein TechniqueManager, der genau **eine** aktive Locomotion und **eine** aktive Interaction hält und alles andere deaktiviert.^[19]

Schritt-für-Schritt Umbau

1. Ordner & Namenskonvention

- Lege an: Assets/_Project/Scripts/Techniques, Assets/_Project/Prefabs/Techniques, Assets/_Project/UI, Assets/_Project/ScriptableObjects.
- Entscheide jetzt schon: „Technique = Prefab“ (empfohlen), damit du später Varianten ohne Scene-Änderungen hinzufügen kannst.

2. Interfaces anlegen

- Erstelle ILocomotionTechnique (z.B. Initialize(rigRefs), SetActive(bool), optional bool BlocksInteraction).
- Erstelle IInteractionTechnique (z.B. Initialize(rigRefs), SetActive(bool), optional bool BlocksLocomotion).
- Vorteil: Dein Menü spricht nie direkt mit konkreten Techniques, sondern nur über diese Schnittstellen.

3. TechniqueRegistry (ScriptableObject)

- Erstelle ein ScriptableObject TechniqueRegistry, das Listen hält: Locomotion[] und Interaction[] (DisplayName + Prefab + optional Beschreibung).
- Lege 1 Asset davon an: Assets/_Project/ScriptableObjects/TechniqueRegistry.asset.

4. TechniqueManager in die Scene

- Erstelle ein neues GameObject auf Root-Ebene: Systems (oder GameSystems) und darunter TechniqueManager. (Das ist bewusst getrennt von ParkourControl und TaskUI, damit Zuständigkeiten sauber bleiben.)^[19]
- TechniqueManager bekommt Referenzen auf dein Rig (Head/LeftHand/RightHand bzw. was OVRCameraRigInteraction/OVRCameraRig bereitstellt) und auf die TechniqueRegistry.^[19]
- Implementiere im Manager: SetLocomotion(index) und SetInteraction(index) → jeweils: vorherige Technique SetActive(false), neue instanzieren (oder aus Pool aktivieren), Initialize, dann SetActive(true).

5. Erste Techniques als „Dummies“

- Baue zuerst zwei Minimal-Prefabs:
- Locomotion_None (macht gar nichts, aber erfüllt das Interface).
- Interaction_None (macht gar nichts).
- Damit testest du das Umschalten im Menü, bevor du echte Locomotion/Grab-Logik einhängst.

Menü einbauen (Wrist-UI, schnell testbar)

1. World-Space Canvas

- Erstelle UI/TechniqueMenuCanvas (World Space) mit Panel, zwei Dropdowns/Buttons (Locomotion/Interaction), Apply-Button und Close-Button.

- Hänge das Canvas als Child an die linke Hand (oder an ein leeres `WristMenuAnchor` unter dem Rig), damit es immer erreichbar ist. ^[19]

2. MenuController

- `MenuController` bekommt Referenzen auf: `TechniqueManager` + `TechniqueRegistry`.
- Beim Start: UI aus Registry befüllen; bei Apply: `manager.SetLocomotion(...)` und `manager.SetInteraction(...)`.

3. Öffnen/Schließen

- Erstmal simpel: ein Button (z.B. Controller-Menüknopf) toggelt `canvas.SetActive(...)`.
- Wenn du beim Öffnen Locomotion pausieren willst: `TechniqueManager` setzt eine „UI-Override“-Flag, die Locomotion `SetActive(false)` triggert (oder Movement blockt), damit UI-Bedienung stabil ist.

Einbindung in deinen Parcours

Du musst an `ParkourControl/Coins/Respawns` nichts ändern; der Technique-Wechsel ist ein reines Player-Feature. Wenn du später feststellst, dass beim Formen-Puzzle die Locomotion stören darf (oder nicht), ist der richtige Ort dafür eine einzige Regel im `TechniqueManager` („wenn Interaction busy → block movement“), nicht im Puzzle selbst. ^[19]

Wenn du mir noch sagst, ob du für UI bereits einen Ray/Pointer im OVR-Setup aktiv hast (oder aktuell nur „TaskUI“ als Anzeige nutzt), kann ich dir die nächste Ausbaustufe (saubere VR-UI-Interaktion, ohne Gefummel) exakt passend zu deinem Rig vorschlagen. ^[19]



Das ist der aktuelle Assets folder. Bitte passe deine Vorgeschlagenen Änderungen dementsprechend an und gib Änderungen mit genauen Ortsangaben und Handlungsanweisungen an. (Bspws. "In Assets → Prefabs → Rechtsklick → erstelle XY")

Unten ist ein Vorgehen, das zu deiner bestehenden Struktur passt (du hast bereits `Assets/Prefabs`, `Assets/Scripts`, `Assets/Scenes`, `Assets/Resources` usw.). ^[29]

1) Ordner anlegen (ohne Umstrukturierung)

1. In **Assets** → **Scripts** → Rechtsklick → **Create** → **Folder** → `Techniques`.
2. In **Assets** → **Scripts** → **Techniques** → Rechtsklick → **Create** → **Folder** → `Interfaces`.
3. In **Assets** → **Scripts** → Rechtsklick → **Create** → **Folder** → `UI`.
4. In **Assets** → **Prefabs** → Rechtsklick → **Create** → **Folder** → `Techniques`. ^[29]
5. In **Assets** → **Prefabs** → Rechtsklick → **Create** → **Folder** → `UI`. ^[29]

6. Optional (wenn du das Registry-Asset per Name laden willst): In **Assets** → **Resources** → Rechtsklick → **Create** → **Folder** → Registries.^[29]

2) Scripts erstellen (exakte Pfade)

1. In **Assets** → **Scripts** → **Techniques** → **Interfaces** → Rechtsklick → **Create** → **C# Script** → `ILocomotionTechnique`.
2. In **Assets** → **Scripts** → **Techniques** → **Interfaces** → **Create** → **C# Script** → `IIInteractionTechnique`.
3. In **Assets** → **Scripts** → **Techniques** → **Create** → **C# Script** → `TechniqueManager`.
4. In **Assets** → **Scripts** → **Techniques** → **Create** → **C# Script** → `TechniqueRegistry` (das wird ein ScriptableObject, das die Prefabs listet).
5. In **Assets** → **Scripts** → **UI** → **Create** → **C# Script** → `TechniqueMenuController`.

Hinweis: Das Repo deutet an, dass Locomotion austauschbar gedacht ist („implement your locomotion technique in `LocomotionTechnique.cs`“), daher ist ein Manager/Registry-Ansatz für „Technique wechseln“ sehr passend.^[30]

3) Registry-Asset anlegen (damit UI automatisch befüllt)

1. Stelle sicher, dass dein `TechniqueRegistry`-Script ein `[CreateAssetMenu(...)]` hat (sonst erscheint es nicht im Create-Menü).
2. Dann: **Assets** → (**Resources** → **Registries**) → Rechtsklick → **Create** → **Technique Registry** → nenne es `TechniqueRegistry_Default`.^[29]
3. Im Inspector: Füge Einträge hinzu (DisplayName + Prefab) für Locomotion und Interaction (anfangs gern „None“-Techniken, siehe nächster Schritt).

4) Dummy-Techniques als Prefabs (damit Umschalten sofort testbar ist)

1. In der Hierarchy: Rechtsklick → **Create Empty** → nenne es `Locomotion_None`.
2. `Locomotion_None` → **Add Component** → dein Script `LocomotionNoneTechnique` (falls du es noch nicht hast: in **Assets** → **Scripts** → **Techniques** anlegen).
3. Ziehe `Locomotion_None` aus der Hierarchy in **Assets** → **Prefabs** → **Techniques** (Prefab erstellen) und lösche es danach wieder aus der Scene.^[29]
4. Wiederhole das gleiche für `Interaction_None` → Prefab nach **Assets** → **Prefabs** → **Techniques**.^[29]
5. Trage beide Prefabs im `TechniqueRegistry_Default` ein.

5) Menü-Prefab bauen und in die Scene hängen

1. In der Hierarchy: Rechtsklick → **UI** → **Canvas** → stelle **Render Mode = World Space**.
2. Canvas unter deine Rig-Hand hängen: Ziehe den Canvas als Child unter `OVRCameraRigInteraction` (oder unter ein leeres `WristMenuAnchor`, das du als Child dort anlegst).^[31]

3. Im Canvas: Rechtsklick → **UI** → **Panel**, dann Buttons/Dropdowns: „Locomotion Next“, „Interaction Next“, „Apply“, „Close“ (für VR ist Button-UI oft robuster als Dropdown).^[29]
4. Ziehe den Canvas aus der Hierarchy nach **Assets** → **Prefabs** → **UI** (Prefab erstellen).^[29]
5. Lege in der Scene ein Root-Objekt an: Hierarchy → Rechtsklick → **Create Empty** → Systems, darunter **TechniqueManager**.^[31]
6. **TechniqueManager-Inspector**: Referenzen setzen (dein Rig-Root **OVRCameraRigInteraction/OVRCameraRig** und **TechniqueRegistry_Default**).^[31]

6) Wiring (damit Buttons wirklich wechseln)

1. **TechniqueMenuController** auf das Canvas (oder ein Child **MenuRoot**) legen.
2. Im Inspector von **TechniqueMenuController**: Referenzen auf **TechniqueManager** und **TechniqueRegistry_Default** setzen.
3. Bei jedem Button: **OnClick()** → **TechniqueMenuController.NextLocomotion()**, **NextInteraction()**, **Apply()**, **ToggleMenu()**.

Wenn du willst, kann ich dir als nächsten Schritt eine **konkrete Minimal-API**

(Methodensignaturen) für **TechniqueManager** + **TechniqueMenuController** vorgeben, damit du ohne Design-Entscheidungen direkt loscodest.

Welche Bedienung willst du fürs Menü: Wrist-Menü per Button-Toggle (z.B. Menüknopf) oder ein Start-Menü am Spawn?



Wir müssen bei 3. nochmal kurz anhalten. Die Davor erstellten scripte sind nicht gefüllt. falls ich scripte füllen soll musst du mir das sagen. ansonsten fahre mit einer detaillierteren anweisung für 3. fort

Ja: Für Schritt 3 musst du **mindestens** das **TechniqueRegistry**-Script (und sinnvollerweise 1–2 kleine Datenklassen) füllen, sonst kannst du kein Registry-Asset anlegen und das Menü kann nicht automatisch befüllt werden.^[40]

Schritt 3 (detailliert): **TechniqueRegistry** sauber anlegen

3.1 Script füllen (genauer Ort + kompletter Inhalt)

1. Gehe zu **Assets** → **Scripts** → **Techniques**.
2. Öffne **TechniqueRegistry.cs** und ersetze den Inhalt durch Folgendes:

```
using System;
using System.Collections.Generic;
using UnityEngine;
```

```
[CreateAssetMenu(menuName = "VR Parkour/Technique Registry", fileName = "TechniqueRegistry",
public class TechniqueRegistry : ScriptableObject
{
    [Serializable]
    public class TechniqueEntry
    {
        public string id;           // z.B. "teleport", "dash", "grab_basic"
        public string displayName;   // Text im Menü
        public GameObject prefab;    // Prefab mit der jeweiligen Technique-Komponente
    }

    [Header("Locomotion")]
    public List<TechniqueEntry> locomotion = new();

    [Header("Interaction")]
    public List<TechniqueEntry> interaction = new();
}
```

Damit bekommst du ein ScriptableObject, das im Unity-Create-Menü erscheint (über `CreateAssetMenu`) und zwei Listen hat (Locomotion/Interaction).

3.2 Ordner für das Asset (wie in deinem Projekt)

Du hast bereits `Assets/Resources` im Projekt. ^[41]

Wenn du später das Registry-Asset bequem per `Resources.Load` finden willst, nutze diesen Pfad:

1. **Assets** → **Resources** → Rechtsklick → **Create** → **Folder** → `Registries`. ^[41]

(Alternativ könntest du das Registry-Asset auch außerhalb von `Resources` ablegen und es direkt im Inspector referenzieren; `Resources` ist nur „schnell & unkompliziert“ fürs Prototyping.)

3.3 Registry-Asset erstellen (genaue Klickfolge)

1. Gehe zu **Assets** → **Resources** → **Registries**.
2. Rechtsklick → **Create** → **VR Parkour** → **Technique Registry**.
3. Benenne das Asset um in: `TechniqueRegistry_Default`.

Wenn du den Menüpunkt nicht siehst: Warte, bis Unity kompiliert hat (keine roten Errors in der Console). Danach nochmal Rechtsklick → Create prüfen.

3.4 Asset befüllen (noch ohne echte Techniques)

Damit du Schritt 3 komplett abschließen kannst, brauchst du **noch keine** fertigen Locomotion-Algorithmen, aber du brauchst später Prefabs zum Eintragen.

Für den Moment kannst du:

- Entweder die Listen leer lassen (Menu kann dann „No techniques found“ anzeigen), oder
- Im nächsten Schritt (Schritt 4) „Dummy“-Prefabs anlegen und dann hier eintragen.

3.5 Warum das zu deinem Projekt passt

Das Repo ist darauf ausgelegt, dass Locomotion als austauschbare „Technique“ implementiert wird (es wird explizit erwähnt, dass du eine Locomotion-Technik in `LocomotionTechnique.cs` implementieren sollst). Ein `TechniqueRegistry` ist der sauberste Weg, daraus eine UI-umschaltbare Auswahl zu machen, ohne deine Scene-Hierarchie anfassen zu müssen. ^[40]

Wenn du willst, mache ich als nächsten Schritt **Schritt 4** genauso detailliert (Dummy-Technique-Scripts + Prefab-Erstellung + Eintragen in `TechniqueRegistry_Default`). Soll das Menü später automatisch beim Start die erste Technik aktivieren, oder erst nach „Apply“?

✱

Gerne

Als nächstes machen wir Schritt 4: Dummy-Techniques + Prefabs, sodass du sie im `TechniqueRegistry_Default` eintragen kannst und später wirklich umschalten kannst. (Das Projekt ist dafür ausgelegt, dass Locomotion als austauschbare „Technique“ implementiert wird.) ^[51]

4) Dummy-Techniques erstellen (mit genauen Orten)

4.1 Interfaces & Context füllen

Damit die Dummy-Techniques sauber kompilieren, müssen deine Interface-Scripte **gefüllt** werden.

1. **Assets** → **Scripts** → **Techniques** → **Interfaces** → Doppelklick `ILocomotionTechnique.cs` → Inhalt ersetzen durch:

```
public interface ILocomotionTechnique
{
    void Initialize(TechniqueContext context);
    void SetActive(bool active);
    bool IsActive { get; }
}
```

2. **Assets** → **Scripts** → **Techniques** → **Interfaces** → Doppelklick `IInteractionTechnique.cs` → Inhalt ersetzen durch:

```
public interface IInteractionTechnique
{
    void Initialize(TechniqueContext context);
    void SetActive(bool active);
    bool IsActive { get; }
}
```

3. **Assets** → **Scripts** → **Techniques** → Rechtsklick → **Create** → **C# Script** → `TechniqueContext` Öffnen und Inhalt einsetzen:

```

using UnityEngine;

[System.Serializable]
public class TechniqueContext
{
    public Transform rigRoot;
    public Transform head;
    public Transform leftHand;
    public Transform rightHand;
}

```

Warum: So können Locomotion/Interaction später auf dein Rig zugreifen, ohne dass jede Technique selbst in der Scene "sucht".

4.2 Ordner für Implementierungen anlegen

1. **Assets** → **Scripts** → **Techniques** → Rechtsklick → **Create** → **Folder** → Implementations.

4.3 Dummy-Locomotion Script anlegen

1. **Assets** → **Scripts** → **Techniques** → **Implementations** → Rechtsklick → **Create** → **C# Script** → LocomotionNoneTechnique

2. Öffnen → Inhalt einsetzen:

```

using UnityEngine;

public class LocomotionNoneTechnique : MonoBehaviour, ILocomotionTechnique
{
    public bool IsActive { get; private set; }
    private TechniqueContext _context;

    public void Initialize(TechniqueContext context)
    {
        _context = context;
    }

    public void SetActive(bool active)
    {
        IsActive = active;
        enabled = active;
        gameObject.SetActive(active);
    }
}

```

4.4 Dummy-Interaction Script anlegen

1. **Assets** → **Scripts** → **Techniques** → **Implementations** → Rechtsklick → **Create** → **C# Script** → InteractionNoneTechnique
2. Öffnen → Inhalt einsetzen:

```
using UnityEngine;

public class InteractionNoneTechnique : MonoBehaviour, IInteractionTechnique
{
    public bool IsActive { get; private set; }
    private TechniqueContext _context;

    public void Initialize(TechniqueContext context)
    {
        _context = context;
    }

    public void SetActive(bool active)
    {
        IsActive = active;
        enabled = active;
        gameObject.SetActive(active);
    }
}
```

Wichtig: Nach dem Einfügen kurz in die Console schauen, ob Errors da sind. Wenn Errors da sind, erst fixen, bevor du Prefabs erstellst.

4.5 Prefabs erstellen (exakte Klickfolge)

Du hast bereits Assets → Prefabs im Projekt. [\[52\]](#)

Locomotion_None Prefab

1. In der **Hierarchy**: Rechtsklick → **Create Empty** → nenne es Locomotion_None.
2. Locomotion_None auswählen → **Add Component** → LocomotionNoneTechnique.
3. Ziehe Locomotion_None aus der Hierarchy in **Assets** → **Prefabs** → **Techniques** (Prefab anlegen). [\[52\]](#)
4. Lösche Locomotion_None wieder aus der Hierarchy (damit du nicht aus Versehen eine Technique "hart" in der Scene hast).

Interaction_None Prefab

1. In der **Hierarchy**: Rechtsklick → **Create Empty** → nenne es Interaction_None.
2. **Add Component** → InteractionNoneTechnique.
3. Ziehe es nach **Assets** → **Prefabs** → **Techniques**. [\[52\]](#)
4. Lösche es wieder aus der Hierarchy.

4.6 Registry befüllen (jetzt wird Schritt 3 "lebendig")

1. Klick in **Project** auf: **Assets** → **Resources** → **Registries** → **TechniqueRegistry_Default** (sofern du es dort erstellt hast). ^[52]

2. Im Inspector:

Locomotion → Size = 1

- Element 0 → id: none
- Element 0 → displayName: None
- Element 0 → prefab: ziehe **Assets** → **Prefabs** → **Techniques** → **Locomotion_None** hinein. ^[52]

Interaction → Size = 1

- Element 0 → id: none
- Element 0 → displayName: None
- Element 0 → prefab: **Interaction_None** hineinziehen. ^[52]

Damit ist Schritt 4 fertig: Du hast jetzt "echte" auswählbare Einträge im Registry-Asset.

Nächster Schritt (5)

Als nächstes bauen wir den **TechniqueManager** so, dass er beim Start "None/None" lädt und später auf Button-Klick umschaltet (dann lohnt sich auch dein Menü).

Eine Rückfrage, bevor ich Schritt 5 exakt auf dein Setup zuschneide: Ist dein **PlayerRig** in der Scene genau **OVRCameraRigInteraction** → **OVRCameraRig**? ^[53]



Ich denke schon.

Was machen wir mit den anderen Scripten in **Assets** → **Scripts** ? Da gibt es noch **CoinRotator**, **DataRecorder**, **LocomotionTechnique**, **MyGrab**, **ParcourCounter** und **SelectionTaskMeasure** die vom Basisprojekt stammen. Soll ich die auch in unterordner packen? bzw sind **LocomotionTechnique** und **MyGrab** ja schon jeweils eine fertige Loc und Int technique

Du musst die bestehenden Scripts nicht zwingend umsortieren, aber es lohnt sich, sie in „Core/Gameplay“ vs. „Techniques“ zu trennen, damit dein neues Umschalt-System sauber bleibt. Dein Basisprojekt ist erkennbar darauf ausgelegt, dass **Locomotion** über **LocomotionTechnique.cs** implementiert/ausgetauscht wird, daher würde ich **LocomotionTechnique** und **MyGrab** als „Legacy“-Techniken in dein Registry-System integrieren statt sie wegzuwerfen. ^[62]

Was machen mit den alten Scripts?

- `CoinRotator`, `ParcourCounter`, `SelectionTaskMeasure`, `DataRecorder` sind typischerweise **Gameplay/Telemetry** (Coins drehen, Score/Timer, Task-Messung, Logging) und sollten unabhängig vom Technique-Switch bleiben.
- `LocomotionTechnique` und `MyGrab` sind sehr wahrscheinlich schon „fertige“ Basistechniken (eine Locomotion, eine Interaction), die du als erste Einträge in deiner `TechniqueRegistry_Default` verwenden kannst. ^[62]

Empfohlene Ordnerstruktur (angepasst)

Du hast aktuell `Assets/Scripts`, `Assets/Prefabs`, `Assets/Resources`, `Assets/Scenes` usw. ^[63]

Mach das so (minimal-invasiv, ohne Umbau anderer Ordner):

- In **Assets** → **Scripts** → erstelle Ordner `Core` (für `Coins/Score/Tasks/Recording`).
- In **Assets** → **Scripts** → **Core** → erstelle Ordner `Gameplay` und `Metrics` (optional).
- In **Assets** → **Scripts** → **Techniques** → erstelle Ordner `Legacy` (für `LocomotionTechnique`, `MyGrab` und ggf. alte Helfer-Scripts).
- In **Assets** → **Scripts** → **Techniques** bleiben deine neuen Sachen (`Interfaces`, `Implementations`, `TechniqueManager`, `TechniqueRegistry`).

Solltest du Dateien wirklich verschieben?

Ja, du kannst sie verschieben, **solange** du nur Dateien/Ordner im Project-Fenster verschiebst und keine Klassen/Dateien umbenennst (Unity kann Meta/References beim Verschieben aktualisieren).

Konkrete Handlungsanweisung:

1. **Assets** → **Scripts** → Rechtsklick → Create → Folder → `Core`.
2. Ziehe `CoinRotator.cs`, `ParcourCounter.cs`, `SelectionTaskMeasure.cs`, `DataRecorder.cs` per Drag&Drop nach **Assets** → **Scripts** → **Core**.
3. **Assets** → **Scripts** → **Techniques** → Rechtsklick → Create → Folder → `Legacy`.
4. Ziehe `LocomotionTechnique.cs` und `MyGrab.cs` nach **Assets** → **Scripts** → **Techniques** → **Legacy**.

Wenn nach dem Verschieben irgendwo „Missing Script“ in der Scene auftaucht: sofort Undo (Ctrl+Z) und sag mir, welches GameObject betroffen ist.

LocomotionTechnique & MyGrab als auswählbare Techniques

Da dein Umschalt-System auf Prefabs in der Registry basiert, machst du aus den vorhandenen Komponenten einfach Prefabs:

1. **Hierarchy** → Rechtsklick → Create Empty → nenne es `Locomotion_Legacy`.

2. Locomotion_Legacy auswählen → Add Component → LocomotionTechnique (dein bestehendes Script).
3. Ziehe Locomotion_Legacy nach **Assets** → **Prefabs** → **Techniques** (Prefab erstellen) und lösche das Scene-Objekt wieder.
4. Wiederhole das für Interaction_Legacy + Component MyGrab → Prefab nach **Assets** → **Prefabs** → **Techniques**.^[63]

Dann:

- Öffne TechniqueRegistry_Default und trage beide Prefabs als Einträge ein (z.B. id: legacy_loco, legacy_grab, displayName entsprechend).

Eine Rückfrage, damit wir nichts kaputt machen: Sind LocomotionTechnique und MyGrab aktuell schon irgendwo in der Scene an Objekten (z.B. unter OVRCameraRigInteraction/OVRCameraRig) oder existieren sie nur als Scripts im Ordner?^[64]



ja, locomotion technique ist auf OVRCameraRig parametrisiert

Wenn LocomotionTechnique aktuell direkt auf OVRCameraRig hängt, ist das völlig okay als Startpunkt—du musst nur verhindern, dass später **zwei** Locomotion-Systeme gleichzeitig aktiv sind, sobald dein TechniqueManager übernimmt.^[74]

Option A (empfohlen): „Legacy“ sauber in dein Technique-System migrieren

Ziel: LocomotionTechnique wird zu einem auswählbaren Prefab-Eintrag in TechniqueRegistry_Default, statt fest auf dem Rig zu kleben. (Das Projekt ist grundsätzlich darauf ausgelegt, dass du deine Locomotion in LocomotionTechnique.cs implementierst.)^[75]

1. In der **Hierarchy**: Klappe OVRCameraRigInteraction → OVRCameraRig auf.^[74]
2. Rechtsklick auf OVRCameraRig → **Create Empty** → nenne es TechniqueHosts.
3. Rechtsklick auf TechniqueHosts → **Create Empty** → nenne es LocomotionHost_Legacy.
4. Wähle **OVRCameraRig** (das Objekt, auf dem LocomotionTechnique aktuell sitzt).
5. Im Inspector bei der LocomotionTechnique-Komponente:
 - Rechtsklick auf den Komponententitel (oder „:“/Gear) → **Copy Component**.
6. Wähle LocomotionHost_Legacy → **Add Component** → LocomotionTechnique.
7. Auf der neu hinzugefügten LocomotionTechnique (auf LocomotionHost_Legacy) → Rechtsklick → **Paste Component Values**.
8. Jetzt erst: Entferne LocomotionTechnique vom OVRCameraRig (Component → :/Gear → **Remove Component**).
9. Teste Play Mode: Es sollte sich exakt wie vorher bewegen (nur dass die Komponente jetzt „entkoppelt“ unter TechniqueHosts hängt).

Danach kannst du daraus ein Prefab machen:

10. Ziehe LocomotionHost_Legacy aus der Hierarchy nach **Assets** → **Prefabs** → **Techniques** → nenne das Prefab z.B. Locomotion_Legacy.^[76]

11. (Optional) Lass das Scene-Objekt erstmal drin, bis dein TechniqueManager fertig ist.

Option B (quick & dirty): Legacy bleibt am Rig, Manager schaltet nur Enable/Disable

Wenn du sofort weiter willst, kannst du LocomotionTechnique erstmal auf OVRCameraRig lassen und später durch den Manager nur component.enabled toggeln (aber das ist langfristig unübersichtlicher, weil „Technique = Prefab“ dann nicht konsequent ist).^[74]

Was bedeutet das für MyGrab?

Gleicher Gedanke: Wenn MyGrab irgendwo am Rig/Hand-Objekt hängt, migrierst du es später analog in einen InteractionHost_Legacy unter TechniqueHosts, damit du Interaction ebenso per Registry umschalten kannst.^[74]

Nächster Schritt (damit du nichts doppelt aktiv hast)

Bevor wir TechniqueManager implementieren: Sag mir kurz, ob **nur** LocomotionTechnique aktuell auf OVRCameraRig hängt oder ob dort noch weitere movement-relevante Komponenten sitzen (CharacterController/Rigidbody/OVRPlayerController etc.).

**

Ich bin die schritte durchgegangen, aber die Bewegung funktioniert jetzt nichtmehr.

Das ist ein typischer Effekt davon, dass LocomotionTechnique sehr wahrscheinlich **den Transform seines eigenen GameObjects** bewegt oder Komponenten per GetComponent<...>() auf *demselben* Objekt erwartet – wenn du das Script von OVRCameraRig auf ein Child verschiebst, bewegt es dann nur dieses Child (oder findet benötigte Komponenten nicht mehr), und der Spieler steht still.^[86] ^[87]

Sofort-Fix (damit du wieder laufen kannst)

1. In der **Hierarchy**: OVRCameraRigInteraction → OVRCameraRig auswählen.^[87]
2. Prüfe im Inspector: Hat OVRCameraRig noch eine LocomotionTechnique-Komponente?
 - Wenn **nein**: OVRCameraRig → **Add Component** → LocomotionTechnique.
3. Wenn du noch LocomotionHost_Legacy hast: dort die LocomotionTechnique-Komponente vorerst **entfernen** (sonst riskierst du doppelte Logik später).
4. Play drücken und testen.

Damit sind wir wieder im „Baseline“-Zustand, den das Projekt ursprünglich vorsieht (Locomotion direkt in LocomotionTechnique.cs am Rig).^[86]

So integrieren wir Legacy trotzdem in dein Umschalt-System (ohne Umhängen)

Wir ändern den Plan minimal: **Legacy bleibt auf** `OVRCameraRig`, und der `TechniqueManager` schaltet sie nur **an/aus**.^[87]

Schritt A: Wrapper-Technique anlegen

1. **Assets** → **Scripts** → **Techniques** → **Implementations** → Rechtsklick → **Create** → **C# Script** → `LocomotionLegacyWrapper`.
2. Inhalt einsetzen:

```
using UnityEngine;

public class LocomotionLegacyWrapper : MonoBehaviour, ILocomotionTechnique
{
    [SerializeField] private MonoBehaviour locomotionTechnique; // hier LocomotionTechnique
    public bool IsActive { get; private set; }

    public void Initialize(TechniqueContext context) { }

    public void SetActive(bool active)
    {
        IsActive = active;
        if (locomotionTechnique != null) locomotionTechnique.enabled = active;
    }
}
```

Schritt B: Wrapper in der Scene setzen

1. In **Hierarchy**: `OVRCameraRigInteraction` → `OVRCameraRig` auswählen.^[87]
2. **Add Component** → `LocomotionLegacyWrapper`.
3. Im Inspector beim Wrapper-Feld `Locomotion Technique`: ziehe die **LocomotionTechnique-Komponente** vom selben Objekt in dieses Feld (Drag&Drop der Komponente).

Schritt C: Registry-Eintrag (ohne Prefab-Umzug)

Du kannst jetzt eine „Technique“ registrieren, die eigentlich nur toggelt:

- Entweder du machst dafür ein Prefab `Locomotion_LegacyWrapper` (ein leeres `GameObject` mit dem Wrapper drauf) und der Manager instanziiert es am Rig, **oder** (noch einfacher) du lässt den Manager die Wrapper-Komponente direkt auf dem Rig finden.

Für den Anfang empfehle ich: **kein Prefab**, sondern `TechniqueManager` hat ein Feld `LocomotionLegacyWrapper legacyLocomotion`; und toggelt dieses.

Was ich von dir brauche (damit ich den nächsten Schritt 100% passend schreibe)

Schick mir bitte einen Screenshot vom Inspector von `OVRCameraRig` mit aufgeklappten **Components**, insbesondere wo `LocomotionTechnique` sitzt und welche Felder dort belegt sind. Dann kann ich dir exakt sagen, ob der Wrapper reicht oder ob `LocomotionTechnique` intern wirklich `transform` bewegt und wir beim Manager eine „MoveRoot“-Referenz brauchen.



reicht das?

Ja, das reicht—und dein Setup sieht jetzt korrekt aus: `LocomotionTechnique` hängt wieder direkt auf `OVRCameraRig` und dein `Locomotion Legacy Wrapper` referenziert genau diese Komponente. Damit können wir Legacy-Locomotion später zuverlässig **an/aus schalten**, ohne sie umhängen zu müssen. ^[98]

Wichtig: Warum es vorher kaputt ging

In deinem Inspector sieht man, dass `LocomotionTechnique` u.a. `Hmd = CenterEyeAnchor` sowie Referenzen auf `ParcourCounter` und `SelectionTaskMeasure` erwartet. Solche Scripts funktionieren oft nur, wenn sie am „richtigen“ Rig-Root sitzen (oder wenn sie per `transform` den Root bewegen), daher war das Verschieben auf ein Child sehr wahrscheinlich der Auslöser. ^[98]

Nächster Schritt: TechniqueManager (minimal, nur toggeln)

Wir starten jetzt bewusst mit der simpelsten Manager-Version: Er toggelt die vorhandene Legacy-Locomotion (und später Legacy-Grab), ohne Instantiation/Prefabs. Das ist am robustesten, weil deine `LocomotionTechnique` bereits komplett verdrahtet ist. ^[98]

1) Script anlegen/füllen

1. Gehe zu **Assets** → **Scripts** → **Techniques** → `TechniqueManager.cs` öffnen. ^[98]
2. Inhalt ersetzen durch:

```
using UnityEngine;

public class TechniqueManager : MonoBehaviour
{
    [Header("Legacy references (existing components in scene)")]
    [SerializeField] private LocomotionLegacyWrapper legacyLocomotion;

    [Header("Defaults")]
    [SerializeField] private bool enableLegacyLocomotionOnStart = true;

    private void Start()
    {
        if (legacyLocomotion != null)
            legacyLocomotion.SetActive(enableLegacyLocomotionOnStart);
    }
}
```

```

public void SetLocomotionLegacy(bool active)
{
    if (legacyLocomotion == null) return;
    legacyLocomotion.SetActive(active);
}
}

```

2) Manager in die Scene setzen (exakt)

1. In der **Hierarchy**: Rechtsklick auf freien Bereich → **Create Empty** → nenne es Systems. ^[98]
2. Systems auswählen → **Add Component** → TechniqueManager.
3. Im Inspector vom TechniqueManager:
 - Feld Legacy Locomotion belegen: Ziehe die **Locomotion Legacy Wrapper**-Komponente vom OVRCameraRig in dieses Feld (Drag&Drop der Komponente). ^[98]
4. Enable Legacy Locomotion On Start = true lassen.

3) Test (schnell verifizieren)

1. Play Mode starten.
2. Es muss sich **genauso wie vorher** bewegen. ^[98]
3. Jetzt Test 2: Im Play Mode im Inspector bei Systems/TechniqueManager die Methode nicht direkt callbar, aber du kannst zum Testen den Haken bei LocomotionTechnique kurz aus/an machen oder den Wrapper über ein kleines Test-UI/Button (nächster Schritt) toggeln. ^[98]

Danach: Menü verbindet sich an den Manager

Als nächstes verdrahten wir dein Menü so, dass ein Button „Locomotion: Legacy ON/OFF“ den TechniqueManager.SetLocomotionLegacy(bool) aufruft (und später erweitern wir das auf mehrere Techniques + Registry). ^[98]

Eine kurze Rückfrage, damit ich das Menü sofort korrekt anschließe: Hast du das World-Space Canvas schon in der Scene, und wenn ja, hängt es an einer Hand oder liegt es im Raum?

✱

wie kann ich schritt 3 umsetzen? ich kann ja beim draggen nicht navigieren und ich muss ja den inspector switchen?

Du hast recht: „Komponente in ein Inspector-Feld ziehen“ ist in Unity manchmal fummelig, weil du beim Draggen nicht sauber navigieren kannst. In deinem Fall gibt es dafür ein paar sehr zuverlässige Wege (ohne Stress). ^[111]

Methode 1 (am einfachsten): Inspector sperren (Lock)

1. Öffne **zwei Inspector-Tabs**: Window → General → **Inspector** (zweimal).
2. In Inspector A wähle OVRCameraRig aus (damit du den **Locomotion Legacy Wrapper** siehst).
^[111]
3. Klicke oben rechts im Inspector A auf das **Schloss (Lock)**, sodass dieser Inspector „eingefroren“ bleibt.
4. Jetzt wähle in der Hierarchy Systems aus (Inspector B zeigt nun TechniqueManager).
5. Ziehe im Inspector A die Komponente **Locomotion Legacy Wrapper (Script)** (am besten am Komponententitel anfassen) auf das Feld Legacy Locomotion im Inspector B.

Methode 2: Objekt ziehen statt Komponente (geht oft schneller)

Wenn dein Feld im TechniqueManager den Typ LocomotionLegacyWrapper hat (so wie ich es vorgeschlagen habe), kannst du oft auch einfach das **GameObject** ziehen:

1. In der Hierarchy ziehe OVRCameraRig auf das Feld Legacy Locomotion im TechniqueManager-Inspector.
^[111]
2. Unity nimmt dann automatisch die passende Komponente (LocomotionLegacyWrapper) von diesem Objekt.

Methode 3: „Kleiner Kreis“ (Object Picker)

Das ist die zuverlässigste Methode, wenn Drag&Drop nervt:

1. Wähle Systems aus, damit du TechniqueManager im Inspector siehst.
2. Rechts neben dem Feld Legacy Locomotion ist ein **kleines Kreis-Symbol** (Object Picker).
3. Klick drauf → Suchfeld → tippe: OVRCameraRig oder LocomotionLegacyWrapper.
4. Wähle den passenden Eintrag aus der Liste.

Methode 4: Copy/Paste Component als Notlösung

Wenn du gar nicht draggen willst:

1. Wähle OVRCameraRig → Rechtsklick auf **Locomotion Legacy Wrapper (Script)** → **Copy Component**.
^[111]
2. Wähle Systems → (hier geht Paste nicht ins Feld), aber du kannst zumindest sicherstellen, dass du weißt, welche Komponente du brauchst.
Diese Methode ist eher „Fallback“, nicht die beste fürs Referenzfeld.

Wenn du magst, sag mir kurz welche Unity-Layout-Situation du hast (ein Inspector oder zwei, locked oder nicht). Dann sage ich dir exakt die schnellste Variante für dein Setup.

ok done, ich wusstre nicht, dass ovrcamerarig auch geht

wie gehts weiter?

Als nächstes bauen wir das **erste minimale Menü**, das die Legacy-Locomotion an/aus schaltet. Danach erweitern wir auf „mehrere Techniques“ über deine `TechniqueRegistry_Default`. (Das Projekt ist generell darauf ausgelegt, Techniken auszutauschen/zu testen.)^[124]

Schritt 5: Minimal-Menu (Toggle Legacy)

5.1 Canvas anlegen (World Space)

1. **Hierarchy** → Rechtsklick → **UI** → **Canvas**.
2. Canvas auswählen → Inspector: **Render Mode = World Space**.
3. (Optional, fürs Testen erstmal im Raum lassen) Setze Position z.B. vor den Startbereich.

5.2 Buttons anlegen

1. Unter Canvas: Rechtsklick → **UI** → **Button (TextMeshPro)** (oder normaler Button).
2. Benenne Button um in `Btn_LocoOn`. Text: „Locomotion ON“.
3. Dupliziere den Button (Ctrl+D) → `Btn_LocoOff`. Text: „Locomotion OFF“.

5.3 Menu-Controller Script füllen

1. **Assets** → **Scripts** → **UI** → Öffne `TechniqueMenuController.cs`.
2. Inhalt ersetzen durch:

```
using UnityEngine;

public class TechniqueMenuController : MonoBehaviour
{
    [SerializeField] private TechniqueManager manager;

    public void LocomotionOn()
    {
        if (manager != null) manager.SetLocomotionLegacy(true);
    }

    public void LocomotionOff()
    {
        if (manager != null) manager.SetLocomotionLegacy(false);
    }
}
```

3. In der Scene: Canvas auswählen → **Add Component** → `TechniqueMenuController`.

4. Im Inspector beim Feld Manager: Ziehe Systems (oder direkt die TechniqueManager-Komponente) hinein.

5.4 Button OnClick verdrahten

1. Btn_LocoOn auswählen → Inspector → Button-Komponente → **On Click()**.
2. Klicke + → ziehe den Canvas (mit TechniqueMenuController) in das Object-Feld.
3. Dropdown auswählen → TechniqueMenuController → LocomotionOn().
4. Wiederhole für Btn_LocoOff → TechniqueMenuController → LocomotionOff().

5.5 Test

- Play drücken → Klick „Locomotion OFF“ → Bewegung muss aus sein.
- Klick „Locomotion ON“ → Bewegung muss wieder gehen.

Schritt 6 (danach): Von Toggle zu Auswahlmenü

Wenn das Toggle läuft, machen wir:

- TechniqueRegistry_Default nutzen, um Buttons dynamisch zu füllen (Locomotion-Liste, Interaction-Liste).
- TechniqueManager erweitern: statt nur SetLocomotionLegacy(bool) bekommt er SetLocomotionById(string) und später auch Interaction.
- Legacy bleibt weiterhin „eine Option“ in der Liste (über Wrapper), neue Techniken kommen als Prefabs dazu.

Bevor wir Schritt 6 anfangen: Soll das Menü am Anfang **immer sichtbar** sein, oder als Wrist-Menu (an der linken Hand) nur per Button aufgehen?

✱✱

1. bei button gibt es kein textfeld, der heisst einfach button, 2. können wir das canvas ins sichtfeld togglen? aktuell ist das viel zu groß und total deplaziert. 3. können wir für das canvas einen ray interactor einführen, dass man nicht mit dem controller an das canvas ranmuss, sondern über entfernung togglen kann?

1. Dass du „kein Textfeld“ siehst, ist normal: Du hast vermutlich **UI** → **Button** (Legacy) erstellt, nicht **Button (TextMeshPro)**; der Text steckt dann als Child Text unter dem Button. 2) Ja, wir können das Canvas sauber **ins Sichtfeld togglen** und klein skalieren. 3) Ja, ein **Ray-Pointer** ist der richtige Weg (mit MetaXR/OVR ist das üblich), dann musst du nicht physisch ans Canvas ran. ^[137]

1) Button-Text ändern (Legacy-UI)

1. In der **Hierarchy**: Klappe deinen Button auf (kleiner Pfeil links).
2. Klicke das Child Text (oder Text (Legacy)), dann im Inspector bei der Text-Komponente den Text ändern (z.B. „Locomotion ON“).
3. Wenn du stattdessen TextMeshPro willst: Rechtsklick auf Canvas → **UI** → **Button - TextMeshPro** (falls verfügbar), dann ist der Text ein TMP-Element.

2) Canvas ins Sichtfeld togglen (und passend skalieren)

Wir machen ein „Head-locked“ Menü: beim Öffnen positioniert es sich 1–1.5 m vor der Kamera und schaut dich an.

2.1 Canvas richtig skalieren

1. Wähle das Canvas aus.
2. Inspector: Setze **Scale** z.B. auf 0.001, 0.001, 0.001 bis 0.01, 0.01, 0.01 (World-Space UI ist sonst riesig).
3. Im Canvas: Stelle in RectTransform z.B. **Width/Height** grob auf 600×400 und arbeite dann mit Scale.

2.2 Script: Menu im Sichtfeld platzieren + togglen

1. Öffne **Assets** → **Scripts** → **UI** → **TechniqueMenuController.cs**.
2. Ersetze den Inhalt durch:

```
using UnityEngine;

public class TechniqueMenuController : MonoBehaviour
{
    [Header("Refs")]
    [SerializeField] private TechniqueManager manager;
    [SerializeField] private Transform head;           // CenterEyeAnchor
    [SerializeField] private GameObject menuRoot;      // Panel oder Canvas-Child

    [Header("Placement")]
    [SerializeField] private float distance = 1.2f;
    [SerializeField] private float heightOffset = -0.15f; // leicht unter Blickmitte

    private void Awake()
    {
        if (menuRoot != null) menuRoot.SetActive(false);
    }

    public void ToggleMenu()
    {
        if (menuRoot == null || head == null) return;

        bool show = !menuRoot.activeSelf;
        menuRoot.SetActive(show);
    }
}
```



```

        if (show)
            PlaceInFrontOfHead();
    }

    private void PlaceInFrontOfHead()
    {
        Vector3 forward = Vector3.ProjectOnPlane(head.forward, Vector3.up).normalized;
        if (forward.sqrMagnitude < 0.001f) forward = head.forward;

        transform.position = head.position + forward * distance + Vector3.up * heightOffset;
        transform.rotation = Quaternion.LookRotation(forward, Vector3.up);
    }

    public void LocomotionOn() { if (manager != null) manager.SetLocomotionLegacy(true); }
    public void LocomotionOff() { if (manager != null) manager.SetLocomotionLegacy(false); }
}

```

2.3 In der Scene verdrahten (ohne Drag-Stress)

1. Wähle dein Canvas (oder ein leeres `MenuAnchor` GameObject, das das Canvas hält).
2. `TechniqueMenuController` hinzufügen (falls noch nicht).
3. Setze Referenzen:
 - **Manager:** dein `Systems/TechniqueManager`.
 - **Head:** ziehe `OVRCameraRig` → `TrackingSpace` → `CenterEyeAnchor` (oder wie es bei dir heißt; im `LocomotionTechnique` ist es als `CenterEyeAnchor` zu sehen).^[138]
 - **MenuRoot:** ziehe dein Panel (oder das Canvas selbst, wenn du lieber das ganze Canvas togglest).

2.4 Toggle per Taste (schnell)

1. In **Hierarchy**: wähle `OVRCameraRig` (oder ein passendes Input-Objekt).
2. Für den schnellen Test kannst du vorerst in `Update()` eine Tastaturtaste abfragen (im Editor), z.B. `M`—wenn du willst, gebe ich dir den 5-Zeilen-Code dafür.
3. Später mappen wir das auf Quest-Controller-Button (`OVRInput`).

3) Ray-Interactor für UI (ohne anfassen)

Du nutzt MetaXR/OVR, daher ist der typische Weg: **OVR Ray + UI Raycaster + OVR Input Module**.

3.1 Canvas vorbereiten

1. Canvas auswählen → **Add Component** → `OVRRaycaster` (falls verfügbar).
2. Canvas → stelle sicher: **Graphic Raycaster** ist vorhanden (bei Canvas standardmäßig).

3.2 EventSystem prüfen

Du hast in der Scene bereits ein EventSystem. ^[139]

1. Wähle EventSystem aus.
2. Stelle sicher, dass ein VR-taugliches Input-Module aktiv ist (bei OVR typischerweise `OVRInputModule`; bei neueren Setups kann es anders heißen).
Wenn du mir einen Screenshot vom EventSystem-Inspector schickst, sage ich dir exakt, welche Komponente du brauchst und was du deaktivieren solltest (damit es nicht doppelt feuert).

3.3 Ray am Controller

1. Unter `OVRCameraRig` suchst du den rechten Controller-Anker (z.B. `RightHandAnchor`).
2. Dort fügst du (je nach MetaXR-Version) entweder ein fertiges „Ray Interaction“-Prefab hinzu oder Komponenten wie `OVRRayInteractor/Pointer`.
Da ich deine installierten MetaXR-Komponenten nicht direkt sehen kann: Schick mir bitte einen Screenshot vom `OVRCameraRig`-Child `OVRInteractionComprehensive` (oder dessen Inspector), dann gebe ich dir die genaue Klickfolge für *dein* Paketstand. ^[138]

Eine kurze Rückfrage: Willst du das Menü als **Wrist-Menu** (an linker Hand „mitgeführt“) oder als **Head-locked** (erscheint vor dir im Blickfeld)?

✱

grafik.jpg

Dein Screenshot reicht: Du hast im EventSystem aktuell nur **Standalone Input Module** aktiv. Damit kannst du zwar mit Maus/Keyboard klicken, aber für VR-Ray-UI brauchst du ein VR-taugliches Input-Module (und du solltest nicht zwei Module parallel aktiv lassen, sonst gibt's Doppel-Events). ^[150]

1) Button-Text (warum kein Textfeld)

Bei deinem Button ist der Text als **Child-Objekt** drin (Legacy-UI), nicht als Feld am Button selbst.

- Hierarchy → klappe Button auf → wähle Text → ändere in der Text-Komponente den String.

2) Canvas ins Sichtfeld togglen (und richtig skalieren)

2.1 Canvas skalieren (Quickwerte)

- Canvas (World Space) auswählen → `RectTransform` Scale auf z.B. 0.002, 0.002, 0.002.
- Panel/Buttons im Canvas ruhig groß lassen (z.B. 400–800 px breit), die **Scale** macht es VR-tauglich.

2.2 Toggle-Placement (Head-locked)

Nutze mein `TechniqueMenuController` von vorher (mit `ToggleMenu()` + `PlaceInFrontOfHead()`), dann erscheint es sauber vor deinem Blick und ist nicht „deplatziert“.

Wichtig: head muss auf deinen `CenterEyeAnchor` zeigen (den hast du bereits als Referenz in `LocomotionTechnique` drin).^[151]

3) Ray-Interactor für UI (mit deinem EventSystem)

Du hast `Standalone Input Module`. Für VR-Ray gibt es zwei gangbare Wege:^[150]

Weg A (empfohlen, OVR/MetaXR): OVRInputModule + OVRRaycaster

1. **EventSystem** auswählen.^[150]
2. Im Inspector bei **Standalone Input Module**: Häkchen links deaktivieren (oder Component entfernen).^[150]
3. EventSystem → **Add Component** → suche nach `OVRIInputModule`.
 - Falls es nicht existiert: dann nutzt dein Projekt vermutlich nicht das OVR-UI-Modul (oder es ist in einem anderen Package), dann nimm Weg B.
4. Canvas auswählen → **Add Component** → suche nach `OVRRaycaster`.
5. Im Rig: am Controller (meist `RightHandAnchor`) einen Ray/Pointer aktivieren (je nach MetaXR-Version heißt das unterschiedlich).

Damit kannst du UI aus der Entfernung bedienen.

Weg B (Fallback, immer verfügbar): Physics Ray + Button-Clicks “per Script”

Wenn `OVRIInputModule/OVRRaycaster` bei dir nicht im Add-Component-Menü auftaucht:

1. Lass `Standalone Input Module` aktiv.^[150]
2. Du machst einen eigenen Ray (`LineRenderer` + `Physics.Raycast`) vom Controller und löst bei Hit auf UI-Elemente manuell `Button.onClick.Invoke()` aus.
Das ist weniger „Unity-UI-konform“, funktioniert aber immer.

Was ich als nächstes von dir brauche (damit ich dir exakt die Klickfolge für Weg A geben kann)

Bitte schick einen Screenshot von:

- `OVRCameraRig` aufgeklappt: insbesondere `TrackingSpace`, `LeftHandAnchor`, `RightHandAnchor` (oder wie sie heißen), und
- Inspector von `OVRIInteractionComprehensive` (der ist bei dir unter dem Rig vorhanden).^[151]

Dann kann ich dir sagen, ob du schon einen fertigen Ray-Interactor im MetaXR-Setup drin hast (oft ist er nur deaktiviert) und welche Komponenten du genau togglen musst.

den rest habe ich umgesetzt, allerdings noch keinen menu toggle implementiert

Passt – dein Rig hat `TrackingSpace` mit `CenterEyeAnchor` usw., damit können wir das Menü zuverlässig „vor die Kamera“ setzen. Für den Toggle machen wir jetzt den kleinsten, stabilen Schritt: **Menu per Controller-Button ein/ausblenden** (Quest), und beim Öffnen automatisch ins Sichtfeld positionieren. ^[164]

Schritt A: Menu-Root sauber definieren

1. In der **Hierarchy**: Wähle dein `Canvas`.
2. Erstelle darunter (falls nicht vorhanden): Rechtsklick → **UI** → **Panel** und nenne es `MenuRoot`.
3. Lege alle Buttons unter `MenuRoot` ab (damit du nur `MenuRoot` togglest, nicht das ganze `Canvas`).

Schritt B: TechniqueMenuController um Toggle erweitern

1. Öffne **Assets** → **Scripts** → **UI** → **TechniqueMenuController.cs**.
2. Ergänze/ersetze mit dieser Version (sie nutzt `OVRInput`, das zu deinem OVR-Setup passt): ^[164]

```
using UnityEngine;

public class TechniqueMenuController : MonoBehaviour
{
    [Header("Refs")]
    [SerializeField] private TechniqueManager manager;
    [SerializeField] private Transform head; // CenterEyeAnchor
    [SerializeField] private GameObject menuRoot; // Panel "MenuRoot"

    [Header("Placement")]
    [SerializeField] private float distance = 1.2f;
    [SerializeField] private float heightOffset = -0.15f;

    [Header("Input")]
    [SerializeField] private OVRInput.Button toggleButton = OVRInput.Button.Start; // rec

    private bool _lastTogglePressed;

    private void Start()
    {
        if (menuRoot != null) menuRoot.SetActive(false);
    }

    private void Update()
    {
        bool pressed = OVRInput.Get(toggleButton);
        if (pressed && !_lastTogglePressed) ToggleMenu();
        _lastTogglePressed = pressed;
    }
}
```

```

public void ToggleMenu()
{
    if (menuRoot == null || head == null) return;

    bool show = !menuRoot.activeSelf;
    menuRoot.SetActive(show);

    if (show) PlaceInFrontOfHead();
}

private void PlaceInFrontOfHead()
{
    Vector3 forward = Vector3.ProjectOnPlane(head.forward, Vector3.up).normalized;
    if (forward.sqrMagnitude < 0.001f) forward = head.forward;

    transform.position = head.position + forward * distance + Vector3.up * heightOffset;
    transform.rotation = Quaternion.LookRotation(forward, Vector3.up);
}

public void LocomotionOn() { if (manager != null) manager.SetLocomotionLegacy(true); }
public void LocomotionOff() { if (manager != null) manager.SetLocomotionLegacy(false); }
}

```

Schritt C: In der Scene verdrahten (konkret, ohne Drag-Probleme)

1. Wähle dein Canvas aus → **Add Component** → TechniqueMenuController (falls noch nicht drauf).
2. Im Inspector des TechniqueMenuController setze:
 - Manager: Ziehe Systems (oder die TechniqueManager-Komponente) rein.
 - Head: Ziehe OVRCameraRig → TrackingSpace → CenterEyeAnchor rein. ^[164]
 - Menu Root: Ziehe das Panel MenuRoot rein.
3. Stelle sicher, dass MenuRoot beim Start **aktiv** ist, ist egal – das Script deaktiviert es in Start().

Schritt D: Test auf Quest

1. Play/Build starten.
2. Drücke am rechten Controller **Menu/Start** (je nach Mapping) → Menü sollte vor dir erscheinen/verschwinden.

Wenn der Button nicht reagiert

Manche OVR-Setups unterscheiden Start und Back je nach Controller/Hand. Dann:

- Im Inspector toggleButton nacheinander testen: OVRInput.Button.Start, OVRInput.Button.Back, oder OVRInput.Button.One (A) als Debug.

Nächster Schritt: Ray-UI wirklich nutzen

Dein `LeftOVRHand` hat aktuell „Ray Helper = None“. Das heißt: ein fertiger Ray ist noch nicht gesetzt. Schick mir bitte noch einen Screenshot vom Inspector von `OVRInteractionComprehensive` (unten in deiner Hierarchy sichtbar), dann sage ich dir exakt, welche Ray/Pointer-Komponenten du aktivieren oder hinzufügen musst, damit du Buttons aus der Distanz anklicken kannst. ^[164]



habe ich schritt a so richtig verstanden?

Ja, genau so ist Schritt A gemeint: `Canvas` → `MenuRoot` → deine Buttons darunter. ^[179]

Zwei kleine Hinweise, damit es gleich sauber funktioniert: ^[179]

Was stimmt an deiner Struktur?

- `MenuRoot` ist ein gemeinsamer Parent für alle Menu-Elemente, den wir später per `menuRoot.SetActive(true/false)` togglen. ^[179]
- Die Text (TMP)-Children unter den Buttons sind korrekt (das ist der TMP-Button-Aufbau). ^[179]

Was du jetzt noch prüfen solltest

1. Stelle sicher, dass `TechniqueMenuController.menuRoot` wirklich auf **MenuRoot** zeigt (nicht auf `Canvas`). ^[179]
2. Das Script `TechniqueMenuController` sollte idealerweise am **Canvas** (oder einem leeren Parent direkt darüber) hängen, damit beim Positionieren/Rotieren das ganze Menü mitgeht. ^[179]

Wenn du willst, schick kurz einen Screenshot vom Inspector deines Canvas mit `TechniqueMenuController`, dann kann ich dir in 1–2 Sätzen sagen, ob die Referenzen korrekt gesetzt sind.



Sollte ich die buttons evtl noch auf überlappung testen?

Ja—kurz testen lohnt sich, weil in World-Space-UI Überlappung/„falsche“ Anklickbereiche häufig von `RectTransforms/Anchors` oder Layout-Komponenten kommen. (Deine Buttons sind aktuell einfach unter `MenuRoot` einsortiert, ohne dass man im Screenshot Position/Größe sieht.) ^[195]

Schneller Overlap-Check (Editor)

1. Wähle MenuRoot aus → Inspector → RectTransform: prüfe, ob es groß genug ist, um beide Buttons zu enthalten.
2. Wähle Btn_LocoOn und Btn_LocoOff jeweils aus → RectTransform:
 - Pos Y muss unterschiedlich sein (z.B. +60 und -60),
 - Width/Height sollten nicht identisch überlappen.
3. In der **Scene View**: aktiviere oben links **2D** (optional) und zoome auf das Canvas, dann siehst du die Rechtecke sauber.

Robusteste Lösung (empfohlen): Vertical Layout Group

Wenn du nicht manuell „pixel schubsen“ willst:

1. Wähle MenuRoot aus.
2. **Add Component** → Vertical Layout Group.
3. **Add Component** → Content Size Fitter.
4. Content Size Fitter:
 - Horizontal Fit = Preferred Size
 - Vertical Fit = Preferred Size
5. Bei jedem Button (Btn_LocoOn, Btn_LocoOff):
 - **Add Component** → Layout Element
 - Setze Preferred Height z.B. 80–120 (groß genug für VR).

Damit sind Overlaps praktisch ausgeschlossen und du kannst später mehr Buttons hinzufügen, ohne alles neu auszurichten.

Test in VR (wichtig)

Overlaps merkst du endgültig erst mit Ray/Pointer: Wenn du auf „ON“ zielst und „OFF“ triggert, dann überlappen die Raycast-Flächen. Dann ist Vertical Layout Group fast immer die schnellste Reparatur.

Wenn du möchtest: schick einen Screenshot vom RectTransform-Inspector von MenuRoot und einem Button, dann sage ich dir direkt, ob Overlap wahrscheinlich ist.

✱✱

Das menü ist aktuell sehr klein und sehr niedrig, kann ich einfach die entsprechenden parameter ändern?

Ja, genau—das ist sogar der richtige Weg: Größe/Position kommen bei dir im Wesentlichen aus **Canvas-Scale, RectTransform-Größen** und den zwei Script-Parametern `distance` und `heightOffset`. (Dein Menü hängt unter `Canvas` → `MenuRoot` → `Buttons`, also kannst du das sehr gezielt einstellen.) ^[211]

Größe größer machen

In dieser Reihenfolge vorgehen:

1. **Canvas auswählen** (World Space) → `RectTransform` → **Scale** erhöhen (z.B. von 0.002 auf 0.003–0.01).
 - Das ist der schnellste „alles wird größer“-Regler.
2. Wenn es dann unscharf/komisch wirkt: **Buttons/Panel größer machen statt Scale**
 - `MenuRoot` → `RectTransform` Width/Height erhöhen.
 - `Buttons` → `RectTransform` Height erhöhen (z.B. 80–120), Textgröße in `Text` (TMP) erhöhen.

Menü höher/zentraler setzen

Das kommt aus deinem `TechniqueMenuController`:

- `heightOffset`: weniger negativ machen oder positiv setzen (z.B. von -0.15 auf 0.0 oder +0.1).
- `distance`: etwas erhöhen, wenn es zu nah ist (z.B. 1.2 → 1.5), oder verringern, wenn es zu weit weg ist.

Wichtig: Wenn du `heightOffset` änderst, achte darauf, dass du es **nicht** so hoch setzt, dass es dir die Sicht auf den Parcours blockiert.

Praktischer VR-Default (Startwerte)

- `distance` = 1.2–1.6
- `heightOffset` = -0.05 bis +0.1
- Canvas Scale so, dass ein Button etwa „handgroß“ wirkt (meist irgendwo bei 0.003–0.01, je nach `RectTransform`-Größe).

Wenn du willst, sag mir kurz deine aktuellen Werte für `distance`, `heightOffset` und den Canvas-Scale (ein Screenshot vom Inspector reicht), dann gebe ich dir konkrete Zielwerte für Quest 3s.

So, jetzt funktioniert alles und ich habe die sachen für mich angenehm eingestellt. was steht als nächstes an? oder können wir jetzt zur implementierung anderer locomotion techniques übergehen?

Ja, jetzt ist ein guter Zeitpunkt, zur Implementierung weiterer Locomotion-Techniques überzugehen—das Projekt ist genau dafür gedacht (Locomotion-Technik implementieren und dann im Parcours testen).^[227]

Was als nächstes sinnvoll ist (Reihenfolge)

1. **Technique-Auswahl skalieren:** Statt nur „Legacy an/aus“ baust du im Menü eine echte Auswahl (mehrere Locomotions).
2. **Erste neue Locomotion implementieren** (ich empfehle als Einstieg „Dash“ oder „Teleport“, weil es klar abgrenzbar ist und die bestehende Legacy nicht stört).
3. **Messbarkeit sicherstellen:** Sobald du zwei Techniken hast, willst du sie mit denselben Metriken im Parcours vergleichen (Zeit, Coins, Task-Erfolg).^[227]

Minimal-Architektur für mehrere Locomotions (ohne dein Setup zu zerreißen)

Wir bleiben bei deinem stabilen Ansatz: Legacy bleibt am Rig, neue Techniken kommen als eigene Komponenten/Prefabs.

- `TechniqueManager` erweitert von `SetLocomotionLegacy(bool)` zu `SetLocomotionMode(enum/id)`.
- Er hält Referenzen auf mehrere Locomotion-Komponenten (z.B. `legacyWrapper`, `dashTechnique`, `teleportTechnique`) und aktiviert genau **eine**.

Das ist weniger „generisch“ als ein komplett dynamisches Prefab-System, aber für dein Uni-Projekt extrem effizient und robust.

Konkreter nächster Schritt (den wir jetzt als erstes umsetzen)

Sag mir bitte, welche 2. Locomotion du implementieren willst:

- **Dash** (kurzer Impuls nach vorne, oft komfortabler als smooth; gut für Parcours-Sprints)
- **Teleport** (sehr komfortabel, aber Parcours kann zu leicht werden)
- **Arm-Swing / Skiing** (cool, aber mehr Tuning)
- **Smooth locomotion + Snap turn** (klassisch, aber Sickness-Risiko)

Wenn du dich entscheidest, gebe ich dir direkt:

- genaue Ordner/Datei-Anlage in `Assets` → `Scripts` → `Techniques` → `Implementations`,
- die benötigten Inspector-Felder (Head, RigRoot, Gain, Cooldown, Collision-Check),
- und die Menü-Erweiterung (zusätzlicher Button „Dash“ + „Legacy“).



Ich würde gerne eine komplexere locomotion umsetzen, die sich assets bedient. Meine Idee wäre ein mit hand Prefab steuerbares Fahrzeug mit lenkrad und schubhebel, in das man, nachdem es bei auswahl der locomotion technique gespawnt wurde, einsteigen kann. kann man das in der bestehenden struktur umsetzen oder schlägst du eine abwandlung der idee vor, die besser in das bestehende system passt? Ansonsten könnte man anstelle von assets auch erstmal hand poses benutzen?

Das lässt sich in deiner bestehenden Struktur umsetzen, aber ich würde die Idee leicht abwandeln: nicht „komplettes Fahrzeug als neue Locomotion“, sondern „Vehicle-Rig als **Locomotion-Mode**, der den Player an ein gespawntes Vehicle bindet und dessen Controls (Lenkrad/Schubhebel) als Interaction nutzt“. Das passt sauber zu deinem Ziel „Locomotion + Interaction austauschbar“, ohne dass du Coins/Task/Respawn-Logik zerreißt. ^[243]

Warum die Abwandlung sinnvoll ist

Ein frei fahrbares Vehicle ist in einem Rundparcours mit engen Passagen schnell „Level-breaking“ (Kollisionen, Steigungen, Durchfahrbreiten, Coin-Trigger), und es erzeugt häufig mehr Motion Sickness als nötig. Ein „Vehicle-Mode mit Seat + Controls“ ist dagegen eine klare, gut bewertbare Technique: Einstieg/Ausstieg, Steuerung über Interactables, eigener Komfort-Fokus. ^[243]

Architekturvorschlag (passt zu deinem System)

- **LocomotionTechnique (VehicleMode)**: spawnt/despawnt das Vehicle-Prefab, verwaltet „entered/exited“, setzt den Player in den Sitz und bewegt das Vehicle (oder bewegt den RigRoot relativ dazu).
- **Interaction**: Lenkrad/Schubhebel sind Interactables, die Werte liefern (steer/throttle).
- **TechniqueManager**: beim Umschalten deaktiviert er Legacy-LoCo und aktiviert VehicleMode (und umgekehrt), so wie du es bereits fürs Menü vorbereitet hast.

Wichtig: VehicleMode sollte beim Aktivieren auch „alles andere“ blocken (z.B. Legacy-Locomotion aus), damit nie zwei Systeme gleichzeitig den Rig bewegen.

Umsetzung: Vehicle als Asset (Prefab) – ja

Das ist der richtige „asset-basierte“ Teil deiner Idee: ein Prefab mit

- Mesh/Collider/Rigidbody (oder kinematisch),
- Seat-Anchor (Transform),
- Wheel + Throttle-Handle (separate Childs),
- optional Audio/VFX.

Das Prefab kommt in **Assets** → **Prefabs** → **Techniques** → **Vehicle** (neuer Unterordner), und die Scripts in **Assets** → **Scripts** → **Techniques** → **Implementations**.

Hand Poses – als guter Zwischenschritt

Hand-Poses eignen sich super als erste Iteration, weil sie dir zwei Risiken abnehmen:

- du musst noch kein perfektes physisches Lenkrad/Lever-Grab lösen,
- du kannst trotzdem ein klares „Vehicle-Feeling“ erzeugen (Hände rasten in Pose ein, solange man „eingestiegen“ ist).

Du kannst das kombinieren: Vehicle spawnen + Seat + einfache Input-Steuerung über Controllersticks, aber beim Einsteigen werden Hand-Poses angezeigt/aktiviert (visuelles/UX-Plus). Das ist oft die beste MVP-Variante.

Nächster Schritt (Entscheidung, dann konkreter Bauplan)

Welche Variante willst du als erstes bauen?

A) **MVP VehicleMode**: Vehicle spawnen, einsteigen, fahren mit Thumbstick (kein echtes Lenkrad/Lever, nur Poses).

B) **Full Controls**: Lenkrad + Schubhebel als Interactables steuern steer/throttle.

Wenn du A oder B wählst, gebe ich dir als nächsten Schritt eine konkrete To-Do-Liste mit exakten Unity-Klickpfaden (Prefab anlegen, Seat Anchor, Scriptfelder, Menu-Button „Vehicle“), plus eine minimale State-Machine (Idle → Spawned → Seated → Driving → Exiting).



ich würde am liebsten beides als individuelle systeme integrieren. wir starten mit a und machen nach erfolgreicher integration mit b als dritter locomotion weiter.

Perfekt—dann machen wir **A (MVP VehicleMode)** als zweite Locomotion (neben Legacy) und bauen es so, dass du später **B (Full Controls)** als dritte Locomotion hinzufügen kannst, ohne das System neu zu verdrahten. ^[259]

A: VehicleMode (MVP) – Setup

1) Ordner anlegen

1. **Assets** → **Prefabs** → **Techniques** → Rechtsklick → Create → Folder → Vehicle. [\[260\]](#)
2. **Assets** → **Scripts** → **Techniques** → **Implementations** → Rechtsklick → Create → Folder → Vehicle. [\[260\]](#)

2) Vehicle-Prefab erstellen (sehr simpel)

1. **Hierarchy** → Rechtsklick → 3D Object → **Cube** (nur fürs MVP).
2. Benenne es Vehicle_MVP.
3. **Add Component** → Rigidbody (erstmal Is Kinematic = true, wir bewegen es per Script).
4. **Add Component** → Box Collider (bleibt).
5. Unter Vehicle_MVP: Rechtsklick → Create Empty → SeatAnchor (Position: da, wo der Spieler „sitzen“ soll).
6. Ziehe Vehicle_MVP in **Assets** → **Prefabs** → **Techniques** → **Vehicle** (Prefab anlegen). [\[260\]](#)
7. Lösche Vehicle_MVP wieder aus der Scene.

A: Scripts (konkret)

Wir bauen **eine** Technique-Komponente, die:

- Vehicle spawnt/despawnt,
- Player-Rig auf Seat setzt,
- mit Thumbstick das Vehicle bewegt,
- Exit ermöglicht.

3) Script anlegen: VehicleLocomotionMVP

1. **Assets** → **Scripts** → **Techniques** → **Implementations** → **Vehicle** → Rechtsklick → Create → C# Script → VehicleLocomotionMVP.
2. Inhalt einsetzen:

```
using UnityEngine;

public class VehicleLocomotionMVP : MonoBehaviour, ILocomotionTechnique
{
    [Header("Vehicle")]
    [SerializeField] private GameObject vehiclePrefab;
    [SerializeField] private float moveSpeed = 3.0f;
    [SerializeField] private float turnSpeed = 60f;

    [Header("Rig refs")]
    [SerializeField] private Transform rigRoot; // meist OVRCameraRig (Root den du be
    [SerializeField] private Transform head; // CenterEyeAnchor
```

```

[Header("Seat")]
[SerializeField] private Vector3 spawnOffset = new Vector3(0, 0, 1.5f);
[SerializeField] private bool autoEnterOnEnable = true;

public bool IsActive { get; private set; }

private GameObject _vehicleInstance;
private Transform _seatAnchor;
private bool _isSeated;
private Vector3 _rigPosBeforeSeat;
private Quaternion _rigRotBeforeSeat;

public void Initialize(TechniqueContext context) { }

public void SetActive(bool active)
{
    IsActive = active;
    enabled = active;

    if (active)
    {
        SpawnVehicle();
        if (autoEnterOnEnable) EnterVehicle();
    }
    else
    {
        ExitVehicle();
        DespawnVehicle();
    }
}

private void SpawnVehicle()
{
    if (_vehicleInstance != null || vehiclePrefab == null || head == null) return;

    Vector3 forward = Vector3.ProjectOnPlane(head.forward, Vector3.up).normalized;
    if (forward.sqrMagnitude < 0.001f) forward = head.forward;

    Vector3 pos = head.position + forward * spawnOffset.z + Vector3.up * spawnOffset.z;
    _vehicleInstance = Instantiate(vehiclePrefab, pos, Quaternion.LookRotation(forward));

    _seatAnchor = _vehicleInstance.transform.Find("SeatAnchor");
    if (_seatAnchor == null) _seatAnchor = _vehicleInstance.transform;
}

private void DespawnVehicle()
{
    if (_vehicleInstance == null) return;
    Destroy(_vehicleInstance);
    _vehicleInstance = null;
    _seatAnchor = null;
    _isSeated = false;
}

public void EnterVehicle()

```

```

    {
        if (_vehicleInstance == null || rigRoot == null || _seatAnchor == null) return;
        if (!_isSeated) return;

        _rigPosBeforeSeat = rigRoot.position;
        _rigRotBeforeSeat = rigRoot.rotation;

        rigRoot.position = _seatAnchor.position;
        rigRoot.rotation = _seatAnchor.rotation;

        _isSeated = true;
    }

    public void ExitVehicle()
    {
        if (!_isSeated || rigRoot == null) return;

        rigRoot.position = _rigPosBeforeSeat;
        rigRoot.rotation = _rigRotBeforeSeat;

        _isSeated = false;
    }

    private void Update()
    {
        if (!IsActive || _vehicleInstance == null || !_isSeated) return;

        // Thumbstick input (OVRInput): Left stick als Movement
        Vector2 stick = OVRInput.Get(OVRInput.Axis2D.PrimaryThumbstick);

        float move = stick.y * moveSpeed * Time.deltaTime;
        float turn = stick.x * turnSpeed * Time.deltaTime;

        _vehicleInstance.transform.Rotate(0f, turn, 0f, Space.World);
        _vehicleInstance.transform.position += _vehicleInstance.transform.forward * move;

        // Exit auf B (kannst du später ins Menü packen)
        if (OVRInput.GetDown(OVRInput.Button.Two))
        {
            ExitVehicle();
        }
    }
}

```

A: In die Scene einhängen (wie bei Legacy)

Jetzt fügen wir es als zweite Technique hinzu, ohne Registry-Overkill:

4) VehicleMode-Komponente auf Rig setzen

1. **Hierarchy** → wähle `OVRCameraRig` (das Objekt, auf dem auch `LocomotionTechnique` sitzt). ^[261]
2. **Add Component** → `VehicleLocomotionMVP`.
3. Im Inspector belegen:

- Vehicle Prefab: **Assets** → **Prefabs** → **Techniques** → **Vehicle** → **Vehicle_MVP** reinziehen. [\[260\]](#)
 - Rig Root: ziehe **OVRCameraRig** rein. [\[261\]](#)
 - Head: ziehe **OVRCameraRig** → **TrackingSpace** → **CenterEyeAnchor** rein. [\[261\]](#)
4. Wichtig: Lass **enabled** erstmal **aus**, wir schalten es über den Manager.

5) TechniqueManager erweitern (2 Modi)

1. Öffne **Assets** → **Scripts** → **Techniques** → **TechniqueManager.cs**.
2. Ergänze zwei Felder + Methoden:

```
[SerializeField] private LocomotionLegacyWrapper legacyLocomotion;
[SerializeField] private VehicleLocomotionMVP vehicleLocomotion;

public void SetLocomotionLegacyMode()
{
    if (vehicleLocomotion != null) vehicleLocomotion.SetActive(false);
    if (legacyLocomotion != null) legacyLocomotion.SetActive(true);
}

public void SetLocomotionVehicleMode()
{
    if (legacyLocomotion != null) legacyLocomotion.SetActive(false);
    if (vehicleLocomotion != null) vehicleLocomotion.SetActive(true);
}
```

3. In der Scene: Systems auswählen → **TechniqueManager** → ziehe **OVRCameraRig** ins Feld **Vehicle Locomotion** (oder direkt die Komponente). [\[261\]](#)

6) Menü erweitern (2 Buttons)

1. Unter MenuRoot: Button duplizieren → **Btn_VehicleMode** Text: „Vehicle“. [\[262\]](#)
2. **OnClick** → **TechniqueManager.SetLocomotionVehicleMode()**
3. Dein vorhandener Legacy-Button: **OnClick** → **TechniqueManager.SetLocomotionLegacyMode()**

Testplan (damit du sofort siehst, ob's "passt")

1. Start: Legacy bewegen.
2. Menü → „Vehicle“: Vehicle spawnt vor dir, du wirst gesetzt, linkes Thumbstick bewegt/lenkt.
3. B (Button Two) → Exit (du stehst wieder am alten Ort).
4. Menü → „Legacy“: wieder normale Bewegung.

Danach: B als dritte Technique vorbereiten

Wenn A stabil ist, bauen wir B als **eigenes Prefab** (Vehicle + Lenkrad + Schubhebel), aber die Schnittstelle bleibt gleich (`VehicleLocomotionFullControls` : `ILocomotionTechnique`), sodass der Manager nur eine weitere Methode/Option bekommt.

Eine Rückfrage, bevor du A implementierst: Soll das Vehicle beim Exit **stehen bleiben** (und weiter existieren), oder soll es beim Exit direkt despawnen?

**

Ich habe schonmal ein Sci-Fi Car Prefab aus dem internet heruntergeladen. können wir das auch benutzen?

Ja, das können wir benutzen—dein Projekt ist genau darauf ausgelegt, dass du alternative Locomotion-Techniken einhängst und im Parcours testest. ^[275]

Sci-Fi Car Prefab integrieren (MVP VehicleMode)

1) Prefab sauber ablegen

1. **Assets** → **Prefabs** → **Techniques** → **Vehicle** → ziehe dein Sci-Fi-Car Prefab (oder den kompletten Prefab-Ordner) da hinein.
2. Falls dein Download als `.unitypackage` kam: **Assets** → **Import Package** → **Custom Package...** → importieren, danach das Prefab in den oben genannten Ordner verschieben.

2) „Vehicle-ready“ machen (SeatAnchor + Collider)

1. Doppelklick auf das Car-Prefab (Prefab Mode öffnen).
2. Root-Objekt auswählen (das oberste GameObject des Autos).
3. Rechtsklick auf Root → **Create Empty** → nenne es `SeatAnchor`.
4. Setze `SeatAnchor` an die gewünschte Sitzposition (meist leicht über dem Sitzpolster; Y nicht zu tief).
5. Prüfe Collider:
 - Wenn **kein** Collider vorhanden: Root auswählen → **Add Component** → **Box Collider** (für MVP reicht ein grober Collider; MeshCollider ist oft teuer/unzuverlässig).
 - Wenn das Modell sehr groß/klein wirkt: Root → Transform Scale prüfen (VR ist empfindlich auf falsche Maße).

3) Rigidbody (für MVP: kinematisch)

1. Im Prefab Mode: Root auswählen → **Add Component** → **Rigidbody** (falls nicht vorhanden).
2. Setze `Is Kinematic = true` (wir bewegen es erstmal per Script, nicht per Physik).

4) In dein Script eintragen

1. Wähle in der Scene `OVRCameraRig` → Komponente `VehicleLocomotionMVP`.
2. Ziehe dein Sci-Fi-Car Prefab in das Feld `vehiclePrefab`.
3. Test: Menü → `VehicleMode`, Spawn sollte vor dir passieren, `SeatAnchor` muss gefunden werden (Name exakt `SeatAnchor`).

Häufige Stolpersteine (und schnelle Fixes)

- Auto spawnnt „seitlich/verkehrt“: Im Prefab Mode prüfen, ob das Modell tatsächlich in **+Z Richtung** „vorne“ hat; sonst Root-Rotation korrigieren oder im Script beim Spawn eine zusätzliche Rotation addieren.
- Du sitzt „falsch“: `SeatAnchor` verschieben/rotieren (das ist genau dafür da).
- Performance auf Quest 3s: Wenn das Car high-poly ist, reduziere Materialien/Shader (kein Transparent/Realtime Reflections), deaktiviere unnötige Partikel/Animatoren, optional LODGroup.

Für später (B) vorbereiten

Wenn du heute schon sauber vorbauen willst: lege im Car-Prefab zusätzlich leere Anchors an:

- `SteeringWheelAnchor`
 - `ThrottleAnchor`
- Dann können wir B später als „Full Controls“-Variante bauen, ohne das Prefab erneut anfassen zu müssen.

Wenn du mir sagst, wie dein Car-Prefab heißt und ob es bereits Collider/Rigidbody mitbringt, kann ich dir die wahrscheinlichsten Einstellungen (kinematic vs. physikbasiert, Collider-Art, SpawnOffset) direkt passend vorschlagen.



Das ist der Aufbau, jedes Objekt (ausser SeatAnchor) hat einen eigenen Mesh collider

Danke, das ist gut strukturiert (Root `ScifiCar_Prefab` + Submeshes + `SeatAnchor`). Dass jedes Teil einen eigenen **MeshCollider** hat, würde ich für Quest 3s im MVP aber **vereinfachen**, sonst riskierst du unnötige Physik-/Raycast-Kosten und seltsame Kollisionen. ^[291]

Empfehlung für MVP (A): Collider vereinfachen

Ziel: Du bewegst das Vehicle erstmal kinematisch per Script (wie geplant), dafür brauchst du keine vielen MeshCollider. [\[292\]](#)

Variante 1 (empfohlen): Ein grober Collider am Root

1. Öffne das Prefab im Prefab Mode (Doppelklick ScifiCar_Prefab). [\[291\]](#)
2. Root ScifiCar_Prefab auswählen → **Add Component** → **Box Collider**.
3. Box Collider so skalieren, dass er grob die Karosserie abdeckt (Räder können "ignoriert" werden).
4. Bei Body, FrontLeftWheel, FrontRightWheel, RearLeftWheel, RearRightWheel, Parts:
 - Wähle jeweils das Objekt → im Inspector den **Mesh Collider** deaktivieren (Häkchen links) oder entfernen.

Variante 2: MeshCollider nur für Body

Wenn du wirklich genauer kollidieren willst:

- Lass **nur** Body MeshCollider aktiv, deaktiviere die Wheel/Parts MeshCollider. [\[291\]](#)

Wichtig: Rigidbody-Setup

1. Root ScifiCar_Prefab auswählen.
2. Wenn noch kein Rigidbody dran ist: **Add Component** → **Rigidbody**.
3. Für MVP: Is Kinematic = true lassen (wir bewegen Transform, nicht Physik).

SeatAnchor passt schon – eine Kleinigkeit

- SeatAnchor ist genau richtig platziert/benannt, weil unser Script es per `.Find("SeatAnchor")` findet. [\[291\]](#)
- Achte nur darauf, dass SeatAnchor leicht über dem Sitz ist (sonst „clipping“ in den Boden/Seat).

Danach: VehicleLocomotionMVP anschließen

1. In der Scene OVRCameraRig auswählen.
2. In VehicleLocomotionMVP:
 - Vehicle Prefab = ScifiCar_Prefab
 - Rig Root = OVRCameraRig
 - Head = CenterEyeAnchor
3. Im Manager/Menu die Umschalt-Buttons wie geplant: Legacy ↔ Vehicle.

Mini-Upgrade (optional, aber empfehlenswert)

Wenn du später Coins/Triggers sauber treffen willst: Gib dem Auto am Root ein Tag/Layer (z.B. `Vehicle`) und stelle Coin-Trigger so ein, dass sie `Vehicle` korrekt erkennen (falls aktuell nur `Player/Rig` getaggt ist).

Wenn du mir sagst: Willst du beim Fahren Kollisionen mit der Umgebung **wirklich** verhindern (Auto stoppt an Wänden), oder reicht fürs MVP „durchfahren“/no-clip? Davon hängt ab, ob wir kinematische Bewegung + simple Collision-Checks (Raycasts) einbauen.



Ich glaube ich habe bei der integration der neuen buttons etwas falsch gemacht, kannst du die schritte bitte nochmal wiederholen?

Klar—hier sind die Schritte für die **neuen Menü-Buttons** (Legacy ↔ Vehicle) nochmal sauber und in der richtigen Reihenfolge, inkl. typischer Fehlerquellen. ^[308]

1) TechniqueManager vorbereiten

1. In **Hierarchy**: wähle `Systems` aus (da hängt `TechniqueManager`).
2. Im Inspector muss `TechniqueManager` diese Referenzen haben:
 - `Legacy Locomotion` → dein `LocomotionLegacyWrapper` (sitzt bei dir am `OVRCameraRig`).
 - `Vehicle Locomotion` → deine `VehicleLocomotionMVP`-Komponente (auch am `OVRCameraRig`, disabled).Wenn eins fehlt: ziehe **`OVRCameraRig`** in das Feld; Unity nimmt dann automatisch die passende Komponente.
3. Öffne **Assets** → **Scripts** → **Techniques** → **`TechniqueManager.cs`** und stelle sicher, dass du diese zwei Methoden hast (Namen exakt so oder entsprechend deiner UI-Auswahl):

```
public void SetLocomotionLegacyMode()
{
    if (vehicleLocomotion != null) vehicleLocomotion.SetActive(false);
    if (legacyLocomotion != null) legacyLocomotion.SetActive(true);
}

public void SetLocomotionVehicleMode()
{
    if (legacyLocomotion != null) legacyLocomotion.SetActive(false);
    if (vehicleLocomotion != null) vehicleLocomotion.SetActive(true);
}
```

Wichtig: `SetActive(true/false)` muss bei beiden wirklich aktiv/deaktivieren (bei Legacy macht das dein Wrapper). ^[309]

2) Buttons im Canvas anlegen

1. In **Hierarchy**: Canvas → MenuRoot. (So wie du es schon hattest.)^[310]
2. Rechtsklick auf Btn_LocoOn oder Btn_LocoOff → **Duplicate**.
3. Benenne den duplizierten Button um:
 - z.B. Btn_ModeLegacy (Text: „Legacy“)
 - und einen zweiten Btn_ModeVehicle (Text: „Vehicle“)

Text ändern:

- Button aufklappen → Text (TMP) auswählen → im TMP-Textfeld „Legacy“/„Vehicle“ eintragen.

Optional gegen Overlap:

- MenuRoot → Vertical Layout Group (wenn du das schon drin hast: super).

3) OnClick korrekt verdrahten (das ist meist der Fehler)

Du machst das für **jeden** Button so:

Legacy Button

1. Wähle Btn_ModeLegacy.
2. Inspector → Button (Script) → Bereich **On Click()**.
3. Wenn dort schon alte Einträge drin sind: lösche sie erst (Minus -).
4. Drücke +.
5. Ziehe **Systems** (GameObject mit TechniqueManager) in das leere Object-Feld.
6. Dropdown rechts: wähle TechniqueManager → SetLocomotionLegacyMode().

Vehicle Button

1. Wähle Btn_ModeVehicle.
2. On Click() → + → ziehe **Systems** rein.
3. Dropdown: TechniqueManager → SetLocomotionVehicleMode().

Typische Fehler

- Du ziehst den Canvas rein statt Systems → dann findest du die Methoden nicht / falsches Script.
- Du hast mehrere TechniqueManager in der Scene → dann wird der falsche aufgerufen.
- Du hast die Methode im Code umbenannt → Button zeigt "Missing" oder ruft nichts auf.

4) Schneller Test

1. Play Mode.
2. Menü öffnen.
3. Drücke „Vehicle“:
 - Legacy-Bewegung muss aus sein,
 - Vehicle spawn und du sitzt drin.
4. Drücke „Legacy“:
 - Vehicle muss deaktiviert/despawnt sein,
 - normale Bewegung wieder aktiv.

Wenn es trotzdem nicht geht (Debug in 30 Sekunden)

Sag mir kurz **was genau passiert**, wenn du auf den neuen Button klickst:

- Passiert gar nichts?
- Vehicle spawn, aber du kannst nicht fahren?
- Beide Locomotions laufen gleichzeitig?

Und schick einen Screenshot vom Btn_ModeVehicle-Inspector (OnClick-Bereich aufgeklappt) + vom Systems/TechniqueManager-Inspector.

✱✱

using UnityEngine;

```
public class TechniqueManager : MonoBehaviour
{
    [Header("Legacy references (existing components in scene)")]
    [SerializeField] private LocomotionLegacyWrapper legacyLocomotion;
    [SerializeField] private VehicleLocomotionMVP vehicleLocomotion;

    [Header("Defaults")]
    [SerializeField] private bool enableLegacyLocomotionOnStart = true;

    private void Start()
    {
        if (legacyLocomotion != null)
            legacyLocomotion.SetActive(enableLegacyLocomotionOnStart);
    }

    public void SetLocomotionLegacy(bool active)
    {
        if (legacyLocomotion == null) return;
        legacyLocomotion.SetActive(active);
    }
}
```

```

public void SetLocomotionLegacyMode()
{
    if (vehicleLocomotion != null) vehicleLocomotion.SetActive(false);
    if (legacyLocomotion != null) legacyLocomotion.SetActive(true);
}

public void SetLocomotionVehicleMode()
{
    if (legacyLocomotion != null) legacyLocomotion.SetActive(false);
    if (vehicleLocomotion != null) vehicleLocomotion.SetActive(true);
}
}

```

Das ist die aktuelle Technique Manager Methode. kannst du die korrigieren?

Ja—die Klasse funktioniert grundsätzlich, aber sie hat zwei typische Probleme:

- `Start()` aktiviert nur Legacy, schaltet Vehicle aber nicht sicher aus (wenn Vehicle irgendwo noch aktiv/instanziiert ist, kann es trotzdem laufen).
- Du hast doppelte API (`SetLocomotionLegacy(bool)` *und* `SetLocomotionLegacyMode()`), das führt oft dazu, dass Buttons auf die „falsche“ Methode zeigen.

Hier ist eine **bereinigte** Version, die beim Start immer einen definierten Zustand setzt und nur noch eine saubere Umschalt-API anbietet. (Das ist wichtig, weil das Projekt auf vergleichbare Locomotion-Tests ausgelegt ist.) [\[325\]](#)

TechniqueManager.cs (ersetzen)

Ersetze den Inhalt deiner Datei durch:

```

using UnityEngine;

public class TechniqueManager : MonoBehaviour
{
    public enum LocomotionMode
    {
        Legacy,
        Vehicle
    }

    [Header("References (existing components in scene)")]
    [SerializeField] private LocomotionLegacyWrapper legacyLocomotion;
    [SerializeField] private VehicleLocomotionMVP vehicleLocomotion;

    [Header("Defaults")]
    [SerializeField] private LocomotionMode startMode = LocomotionMode.Legacy;

    private void Start()
    {
        ApplyLocomotionMode(startMode);
    }
}

```

```

public void ApplyLocomotionMode(LocomotionMode mode)
{
    // Always force a clean state: disable both first
    if (legacyLocomotion != null) legacyLocomotion.SetActive(false);
    if (vehicleLocomotion != null) vehicleLocomotion.SetActive(false);

    // Then enable exactly one
    switch (mode)
    {
        case LocomotionMode.Legacy:
            if (legacyLocomotion != null) legacyLocomotion.SetActive(true);
            break;

        case LocomotionMode.Vehicle:
            if (vehicleLocomotion != null) vehicleLocomotion.SetActive(true);
            break;
    }
}

// UI-friendly wrappers (Buttons can call these)
public void SetLocomotionLegacyMode() => ApplyLocomotionMode(LocomotionMode.Legacy);
public void SetLocomotionVehicleMode() => ApplyLocomotionMode(LocomotionMode.Vehicle);
}

```

Was du danach im UI ändern musst

Wenn du vorher Buttons auf `SetLocomotionLegacy(true/false)` gelegt hast: bitte die `OnClick`-Events auf die neuen Methoden umstellen:

- `TechniqueManager` → `SetLocomotionLegacyMode()`
- `TechniqueManager` → `SetLocomotionVehicleMode()`

Mini-Checkliste (damit's sofort klappt)

- In `Systems/TechniqueManager` sind beide Felder gesetzt (`legacyLocomotion` und `vehicleLocomotion`).
- `VehicleLocomotionMVP` ist nicht manuell aktiv in der Scene (der Manager übernimmt).
- `StartMode` = `Legacy` (oder `Vehicle`, wie du willst).

Wenn du mir sagst, was beim Klick auf „Vehicle“ aktuell passiert (nichts / spawnt aber keine Steuerung / sofort wieder weg), kann ich dir als nächsten Schritt gezielt die Fehlerstelle eingrenzen.

✱

und wie genau soll ich jetzt die einzelnen buttons einstellen? Ich habe jetzt 4

Du bist korrekt verdrahtet: `Systems/TechniqueManager` hat beide Referenzen (Legacy Wrapper + Vehicle MVP). Jetzt geht's nur noch darum, **welcher Button welche Methode** aufruft (und doppelte `OnClick`-Einträge zu vermeiden).^[342]

Welche 4 Buttons du sinnvoll belegst

Du hast typischerweise:

1. „Legacy“ (Locomotion-Modus 1)
2. „Vehicle“ (Locomotion-Modus 2)
3. „Locomotion ON“
4. „Locomotion OFF“

Sobald du **Modus-Buttons** hast, brauchst du „ON/OFF“ eigentlich nicht mehr. Daher:

- Empfehlung: Behalte **2 Buttons**: `Legacy` und `Vehicle`.
- Optional: Behalte zusätzlich **Menu Close** (oder Toggle per Controller-Button ist schon genug).

Wenn du die 4 Buttons behalten willst, dann so:

Button-Zuordnung (konkret)

Button 1: Legacy

- `OnClick` → **Systems** (`TechniqueManager`) → `SetLocomotionLegacyMode()`.

Button 2: Vehicle

- `OnClick` → **Systems** (`TechniqueManager`) → `SetLocomotionVehicleMode()`.

Button 3: Locomotion ON

- Wenn du ON/OFF behalten willst, mach ON = „Legacy an“ (weil „an“ sonst unklar ist):
- `OnClick` → **Systems** → `SetLocomotionLegacyMode()`.

Button 4: Locomotion OFF

- „OFF“ sollte beide deaktivieren. Dafür brauchst du eine Methode:
1. Öffne `TechniqueManager.cs` und füge hinzu:

```
public void DisableAllLocomotion()
{
    if (legacyLocomotion != null) legacyLocomotion.SetActive(false);
```



```
    if (vehicleLocomotion != null) vehicleLocomotion.SetActive(false);  
}
```

2. Dann im Button „OFF“:

- **OnClick** → **Systems** → `DisableAllLocomotion()`.

So stellst du jeden Button richtig ein (Schritt für Schritt)

Für **jeden** Button einzeln:

1. Button in der Hierarchy auswählen (z.B. `Btn_ModeVehicle`).^[342]
2. Inspector → `Button (Script)` → Bereich **On Click()**.
3. Lösche alle alten Einträge mit `-`, bis die Liste leer ist (wichtig, sonst feuern mehrere Methoden).
4. `+` drücken.
5. Ziehe **Systems** (GameObject) in das Object-Feld.^[342]
6. Dropdown auswählen → `TechniqueManager` → gewünschte Methode (siehe Zuordnung oben).

Mini-Check danach

- Wenn du „Vehicle“ klickst und trotzdem Legacy noch läuft, dann hast du sehr wahrscheinlich noch einen zweiten OnClick-Eintrag oder dein altes `SetLocomotionLegacy(true)` hängt noch irgendwo.^[342]

Wenn du mir die Namen deiner 4 Buttons kurz schreibst (so wie sie in der Hierarchy heißen), sage ich dir 1:1, welche Methode du pro Button nehmen sollst.

✱✱

Ok ich bin auf zwei buttons heruntergegangen, kann ich jetzt testen oder fehlt noch etwas?

Ja, du kannst jetzt testen—wenn diese kurze Checkliste passt, fehlt nichts Wesentliches mehr. (Das Projekt ist genau dafür gedacht: Technik einhängen und im Parcours ausprobieren.)^[360]

Pre-Flight Check (30 Sekunden)

1. Buttons

- Button „Legacy“ → OnClick ruft `Systems/TechniqueManager.SetLocomotionLegacyMode()` auf.
- Button „Vehicle“ → OnClick ruft `Systems/TechniqueManager.SetLocomotionVehicleMode()` auf.

2. TechniqueManager

- In `Systems` sind `Legacy Locomotion` und `Vehicle Locomotion` belegt (bei dir war das schon korrekt).^[361]

3. VehicleLocomotionMVP Felder

- `vehiclePrefab` ist dein Car-Prefab.
- `rigRoot` ist `OVRCameraRig`.
- `head` ist `TrackingSpace/CenterEyeAnchor`. [\[362\]](#)

4. Startzustand

- `startMode` (oder bei deiner alten Version „Enable Legacy...“) ist so gesetzt, dass du beim Start laufen kannst (meist Legacy). [\[361\]](#)

Testablauf (so erkennst du sofort Fehler)

1. Start → du kannst mit Legacy laufen.
2. Menü öffnen → „Vehicle“ drücken → Auto spawnt vor dir, du wirst in den Sitz gesetzt, linkes Thumbstick bewegt/lenkt.
3. Menü öffnen → „Legacy“ drücken → Auto/VehicleMode wird deaktiviert, normale Bewegung wieder da.

Wenn beim Test etwas schiefgeht

Sag mir dann genau **welcher Schritt** aus dem Testablauf fehlschlägt (z.B. „Vehicle klick: nichts passiert“ oder „Auto spawnt, aber keine Bewegung“), dann kann ich dir die passende Debug-Stelle nennen (meist fehlende Prefab-Referenz oder falscher `rigRoot`).

**

Wir haben vergessen, den techniqu menu controller anzupassen...

```
using UnityEngine;
```

```
public class TechniqueMenuController : MonoBehaviour
{
```

```
    [Header("Refs")]
```

```
    [SerializeField] private TechniqueManager manager;
```

```
    [SerializeField] private Transform head;    // CenterEyeAnchor
```

```
    [SerializeField] private GameObject menuRoot; // Panel "MenuRoot"
```

```
    [Header("Placement")]
```

```
    [SerializeField] private float distance = 1.2f;
```

```
    [SerializeField] private float heightOffset = -0.15f;
```

```
    [Header("Input")]
```

```
    [SerializeField] private OVRInput.Button toggleButton = OVRInput.Button.Start; // rechter
Controller Menü
```

```
    private bool _lastTogglePressed;
```

```

private void Start()
{
    if (menuRoot != null) menuRoot.SetActive(false);
}

private void Update()
{
    bool pressed = OVRInput.Get(toggleButton);
    if (pressed && !_lastTogglePressed) ToggleMenu();
    _lastTogglePressed = pressed;
}

public void ToggleMenu()
{
    if (menuRoot == null || head == null) return;

    bool show = !menuRoot.activeSelf;
    menuRoot.SetActive(show);

    if (show) PlaceInFrontOfHead();
}

private void PlaceInFrontOfHead()
{
    Vector3 forward = Vector3.ProjectOnPlane(head.forward, Vector3.up).normalized;
    if (forward.sqrMagnitude < 0.001f) forward = head.forward;

    transform.position = head.position + forward * distance + Vector3.up * heightOffset;
    transform.rotation = Quaternion.LookRotation(forward, Vector3.up);
}

public void LocomotionOn() { if (manager != null) manager.SetLocomotionLegacy(true); }
public void LocomotionOff() { if (manager != null) manager.SetLocomotionLegacy(false); }
}

```

Das ist der alte code und die fehlermeldungen

Genau: die Fehlermeldung kommt daher, dass `TechniqueManager.SetLocomotionLegacy(bool)` nicht mehr existiert (oder du ihn ersetzt hast), dein `TechniqueMenuController` aber noch `LocomotionOn/Off()` damit aufruft. [\[378\]](#)

Du hast jetzt **zwei Buttons** (Legacy/Vehicle), also passen wir den MenuController so an, dass er diese beiden Modi aufruft (und optional das Menü nach Klick schließt).

Fix: TechniqueMenuController.cs ersetzen

Öffne **Assets** → **Scripts** → **UI** → **TechniqueMenuController.cs** und ersetze den Inhalt komplett durch:

```
using UnityEngine;

public class TechniqueMenuController : MonoBehaviour
{
    [Header("Refs")]
    [SerializeField] private TechniqueManager manager;
    [SerializeField] private Transform head;          // CenterEyeAnchor
    [SerializeField] private GameObject menuRoot;    // Panel "MenuRoot"

    [Header("Placement")]
    [SerializeField] private float distance = 1.2f;
    [SerializeField] private float heightOffset = -0.15f;

    [Header("Input")]
    [SerializeField] private OVRInput.Button toggleButton = OVRInput.Button.Start;

    private bool _lastTogglePressed;

    private void Start()
    {
        if (menuRoot != null) menuRoot.SetActive(false);
    }

    private void Update()
    {
        bool pressed = OVRInput.Get(toggleButton);
        if (pressed && !_lastTogglePressed) ToggleMenu();
        _lastTogglePressed = pressed;
    }

    public void ToggleMenu()
    {
        if (menuRoot == null || head == null) return;

        bool show = !menuRoot.activeSelf;
        menuRoot.SetActive(show);

        if (show) PlaceInFrontOfHead();
    }

    private void PlaceInFrontOfHead()
    {
        Vector3 forward = Vector3.ProjectOnPlane(head.forward, Vector3.up).normalized;
        if (forward.sqrMagnitude < 0.001f) forward = head.forward;

        transform.position = head.position + forward * distance + Vector3.up * heightOffset;
        transform.rotation = Quaternion.LookRotation(forward, Vector3.up);
    }

    // --- Button callbacks (2 buttons) ---
}
```

```

public void SelectLegacy()
{
    if (manager != null) manager.SetLocomotionLegacyMode();
    if (menuRoot != null) menuRoot.SetActive(false);
}

public void SelectVehicle()
{
    if (manager != null) manager.SetLocomotionVehicleMode();
    if (menuRoot != null) menuRoot.SetActive(false);
}
}

```

Buttons neu verdrahten (wichtig)

Für **jeden** deiner zwei Buttons:

1. Button auswählen → Inspector → Button (Script) → **On Click()**.
2. Alle alten Einträge entfernen (-), bis leer.
3. + → ziehe das GameObject mit `TechniqueMenuController` (dein Canvas oder MenuAnchor) ins Feld.
4. Wähle:
 - Legacy Button → `TechniqueMenuController.SelectLegacy()`
 - Vehicle Button → `TechniqueMenuController.SelectVehicle()`

Damit kompiliert es wieder, und die Buttons schalten sauber die Modi um. [\[378\]](#)

Wenn du mir kurz sagst, ob dein `TechniqueManager` die neue Enum-Version (`ApplyLocomotionMode`) oder noch die ältere Version mit `EnableLegacyLocomotionOnStart` ist, kann ich dir auch sagen, ob du im Inspector nach dem Update noch Felder neu setzen musst.

**

Ok, es funktioniert alles soweit, allerdings bewegt sich die kamera nicht mit dem fahrzeug, sondern bleibt stationär.

Das passiert, weil dein MVP-Script den `RigRoot` nur **einmal** beim „Einsteigen“ auf den `SeatAnchor` setzt, aber danach bewegst du nur das Fahrzeug – der `RigRoot` wird nicht mitgeführt. Das ist genau der Punkt, an dem „Vehicle als Locomotion-Technique“ komplexer wird als normales Laufen. [\[397\]](#)

Fix (empfohlen): Rig an SeatAnchor parenten

Beim Einsteigen setzt du den OVRCameraRig als Child vom SeatAnchor, dann folgt die Kamera automatisch allen Bewegungen/Rotationen des Fahrzeugs.

1) VehicleLocomotionMVP anpassen

Öffne VehicleLocomotionMVP.cs und ergänze diese Felder oben in der Klasse:

```
private Transform _rigParentBeforeSeat;  
private Vector3 _rigLocalPosBeforeSeat;  
private Quaternion _rigLocalRotBeforeSeat;
```

Ersetze dann deine EnterVehicle() und ExitVehicle() durch:

```
public void EnterVehicle()  
{  
    if (_vehicleInstance == null || rigRoot == null || _seatAnchor == null) return;  
    if (_isSeated) return;  
  
    // Backup  
    _rigParentBeforeSeat = rigRoot.parent;  
    _rigPosBeforeSeat = rigRoot.position;  
    _rigRotBeforeSeat = rigRoot.rotation;  
    _rigLocalPosBeforeSeat = rigRoot.localPosition;  
    _rigLocalRotBeforeSeat = rigRoot.localRotation;  
  
    // Parent to seat so we follow vehicle motion  
    rigRoot.SetParent(_seatAnchor, worldPositionStays: true);  
  
    // Snap into seat (local = 0)  
    rigRoot.localPosition = Vector3.zero;  
    rigRoot.localRotation = Quaternion.identity;  
  
    _isSeated = true;  
}  
  
public void ExitVehicle()  
{  
    if (!_isSeated || rigRoot == null) return;  
  
    // Restore parent  
    rigRoot.SetParent(_rigParentBeforeSeat, worldPositionStays: true);  
  
    // Restore world pose (sauberer als local restore)  
    rigRoot.position = _rigPosBeforeSeat;  
    rigRoot.rotation = _rigRotBeforeSeat;  
  
    _isSeated = false;  
}
```

2) Prüfen, ob du den richtigen RigRoot referenzierst

In deinem `VehicleLocomotionMVP` Inspector muss `rigRoot` wirklich auf **OVRCameraRig** zeigen (nicht auf `TrackingSpace` oder `CenterEyeAnchor`), sonst bewegt sich der HMD nicht mit. ^[398]

Wenn Parenting Probleme macht (Fallback)

Falls du beim Parenten "komische Offsets" bekommst, sag Bescheid – dann machen wir stattdessen ein „Follow“-Update (pro Frame `rigRoot.position = seatAnchor.position`), aber Parenting ist normalerweise die sauberere Lösung fürs MVP.

Wenn du kurz sagst, ob dein `OVRCameraRig` selbst noch irgendwo parented ist (z.B. unter `OVRCameraRigInteraction`), kann ich dir sagen, ob `worldPositionStays` besser auf `true` oder `false` stehen sollte.



Klappt super jetzt. Können wir die Buttons jetzt ummappen? Die Controller als Wheel (links höher → rechts lenken und rechts höher → links lenken) fahren und bremsen auf left und right trigger und upshift und downshift auf rightselect und leftselect?

Ja—das kannst du direkt im `VehicleLocomotionMVP` ummappen: Steering aus dem Höhenunterschied der beiden Controller, Gas/Bremse über **Left/Right Trigger** (Axis1D), und Schalten über Buttons. ^[416] ^[417]

1) VehicleLocomotionMVP: neue Felder

Öffne `VehicleLocomotionMVP.cs` und ergänze (zusätzlich zu `rigRoot/head`) diese `SerializeFields`:

```
[Header("Hands for wheel steering")]
[SerializeField] private Transform leftController;    // z.B. TrackingSpace/LeftController
[SerializeField] private Transform rightController;  // z.B. TrackingSpace/RightController
[SerializeField] private float steerGain = 8f;
[SerializeField] private bool invertSteer = true;

[Header("Pedals (triggers)")]
[SerializeField] private float accel = 6f;
[SerializeField] private float brake = 10f;

[Header("Gears")]
[SerializeField] private int minGear = 1;
[SerializeField] private int maxGear = 5;
[SerializeField] private float[] gearMaxSpeed = { 0f, 3f, 5f, 7f, 9f, 11f }; // Index = gear
[SerializeField] private OVRInput.Button upShiftButton = OVRInput.Button.One; // meist 1
[SerializeField] private OVRInput.Button downShiftButton = OVRInput.Button.Three;
```

```
private int _gear;
private float _speed;
private float _neutralHeightDelta;
```

Hinweis: `OVRInput.Axis1D.PrimaryIndexTrigger` / `SecondaryIndexTrigger` sind die typischen Achsen für Left/Right Index Trigger (0..1).^[416]

2) Neutral-„Lenkrad“ kalibrieren beim Einsteigen

In `EnterVehicle()` (nachdem `_isSeated = true;`) ergänzen:

```
_gear = Mathf.Clamp(_gear == 0 ? 1 : _gear, minGear, maxGear);

if (leftController != null && rightController != null)
    _neutralHeightDelta = leftController.position.y - rightController.position.y;
else
    _neutralHeightDelta = 0f;
```

Damit ist „beide Hände auf gleicher Höhe“ dein Null-Lenkpunkt.

3) Update(): Steering + Trigger-Pedals + Schalten

Ersetze den Input-Teil in `Update()` (dein bisheriges Thumbstick-Zeug) durch:

```
if (!IsActive || _vehicleInstance == null || !_isSeated) return;

// --- Steering: Controller als Lenkrad (Höhe links vs rechts) ---
float heightDelta = 0f;
if (leftController != null && rightController != null)
    heightDelta = (leftController.position.y - rightController.position.y) - _neutralHeightDelta;

float steer = Mathf.Clamp(heightDelta * steerGain, -1f, 1f);
if (invertSteer) steer = -steer; // links höher -> rechts lenken

// --- Pedals: Trigger ---
float gas = OVRInput.Get(OVRInput.Axis1D.PrimaryIndexTrigger); // Left Trigger [web:2]
float bremse = OVRInput.Get(OVRInput.Axis1D.SecondaryIndexTrigger); // Right Trigger [web:2]

// --- Gears: Buttons ---
if (OVRInput.GetDown(upShiftButton)) _gear = Mathf.Min(_gear + 1, maxGear);
if (OVRInput.GetDown(downShiftButton)) _gear = Mathf.Max(_gear - 1, minGear);

float maxSpeed = (_gear < gearMaxSpeed.Length) ? gearMaxSpeed[_gear] : gearMaxSpeed[gearMaxSpeed.Length - 1];

// Geschwindigkeit integrieren
_speed += gas * accel * Time.deltaTime;
_speed -= bremse * brake * Time.deltaTime;
_speed = Mathf.Clamp(_speed, 0f, maxSpeed);

// Bewegung anwenden
```



```
_vehicleInstance.transform.Rotate(0f, steer * turnSpeed * Time.deltaTime, 0f, Space.World);  
_vehicleInstance.transform.position += _vehicleInstance.transform.forward * (_speed * Time.deltaTime);
```

Wenn die Lenkrichtung falsch herum ist, togglest du einfach `invertSteer` im Inspector.

4) Inspector-Verdrahtung (wichtig)

In der Scene bei `OVRCameraRig` → `VehicleLocomotionMVP`:

- `Left Controller` = `OVRCameraRig/TrackingSpace/LeftControllerAnchor`
- `Right Controller` = `OVRCameraRig/TrackingSpace/RightControllerAnchor`

Das passt zu deiner Rig-Hierarchie mit `TrackingSpace` und den Controller-Anchors. [\[418\]](#)

Kurze Rückfrage zu „RightSelect/LeftSelect“

Meinst du mit „Select“ die **A/X Buttons** oder das **Thumbstick-Click**? (In `OVRInput` wäre das typischerweise `Button.One/Three` für A/X, oder `Button.PrimaryThumbstick/SecondaryThumbstick` für Stick-Click.) [\[419\]](#)

✱

ok. als nächstes hätte ich gerne eine deadzone (ca 5cm) in denen der höhenunterscheid nicht zu einer lenkbewegung führt. zusätzlich hätte ich gerne eine ganganzeige und einen rückwärtsgang. desweiteren soll sich das auto nicht im stand drehen können, sondern nur in vorwärts oder rückwärtsbewegung

Ja—das sind alles saubere Erweiterungen für dein `VehicleLocomotionMVP`: (1) Deadzone fürs „Hände als Lenkrad“, (2) Gangsystem inkl. Rückwärtsgang + Anzeige, (3) Lenken nur bei Bewegung (vorwärts oder rückwärts). Für Gas/Bremse bleiben die Index-Trigger als Achsen (0..1) über `OVRInput.Axis1D.PrimaryIndexTrigger` und `OVRInput.Axis1D.SecondaryIndexTrigger` wie bisher. [\[450\]](#) [\[451\]](#)

1) Deadzone (5 cm) fürs Lenken

Füge im Script Felder hinzu:

```
[SerializeField] private float steerDeadzoneMeters = 0.05f; // 5cm  
[SerializeField] private float steerGain = 8f;  
[SerializeField] private bool invertSteer = true;
```

Ersetze die Steering-Berechnung durch Deadzone + Remap:

```

float heightDelta = 0f;
if (leftController != null && rightController != null)
    heightDelta = (leftController.position.y - rightController.position.y) - _neutralHeight;

// Deadzone (5cm)
float abs = Mathf.Abs(heightDelta);
float steer = 0f;

if (abs > steerDeadzoneMeters)
{
    float sign = Mathf.Sign(heightDelta);
    float effective = abs - steerDeadzoneMeters;    // erst ab hier lenken
    steer = sign * Mathf.Clamp(effective * steerGain, 0f, 1f);
    if (invertSteer) steer = -steer;
}

```

2) Rückwärtsgang + Ganglogik

Ich empfehle: gear = 0 = Reverse, gear = 1..maxGear = Vorwärts.

Felder:

```

[SerializeField] private int maxGear = 5;
[SerializeField] private float maxReverseSpeed = 3f;
[SerializeField] private float[] gearMaxSpeed = { 0f, 3f, 5f, 7f, 9f, 11f }; // Index 1..maxGear
private int _gear = 1;    // Start: 1. Gang
private float _speed;     // kann negativ sein (Reverse)

```

Shift-Logik:

```

// Up/downshift
if (OVRInput.GetDown(upShiftButton))
{
    if (_gear == 0) _gear = 1;           // von R nach 1
    else _gear = Mathf.Min(_gear + 1, maxGear);
}

if (OVRInput.GetDown(downShiftButton))
{
    if (_gear == 1) _gear = 0;           // von 1 nach R
    else if (_gear > 1) _gear -= 1;
}

```

Speed-Update mit Reverse:

```

float gas = OVRInput.Get(OVRInput.Axis1D.PrimaryIndexTrigger);    // 0..1 [web:20][web:21]
float brake = OVRInput.Get(OVRInput.Axis1D.SecondaryIndexTrigger); // 0..1 [web:20][web:21]

float maxFwd = (_gear >= 1 && _gear < gearMaxSpeed.Length) ? gearMaxSpeed[_gear] : gearMaxSpeed[maxGear];
float maxSpd = (_gear == 0) ? maxReverseSpeed : maxFwd;

```

```
// Beschleunigungsrichtung hängt vom Gang ab
float dir = (_gear == 0) ? -1f : 1f; // Reverse = negativ
_speed += (gas * accel * Time.deltaTime) * dir;

// Bremse zieht Speed Richtung 0 (egal ob vorwärts/rückwärts)
_speed = Mathf.MoveTowards(_speed, 0f, brake * brakeStrength * Time.deltaTime);

// Clamp: Reverse [-maxReverseSpeed..0], Forward [0..maxFwd]
if (_gear == 0) _speed = Mathf.Clamp(_speed, -maxSpd, 0f);
else _speed = Mathf.Clamp(_speed, 0f, maxSpd);
```

3) Nicht im Stand drehen

Füge ein Feld hinzu:

```
[SerializeField] private float minSpeedToTurn = 0.05f; // ca. 5 cm/s
```

Und setze Turn nur, wenn Bewegung vorhanden:

```
float turnInput = steer;
if (Mathf.Abs(_speed) < minSpeedToTurn)
    turnInput = 0f;

_vehicleInstance.transform.Rotate(0f, turnInput * turnSpeed * Time.deltaTime, 0f, Space.V
_vehicleInstance.transform.position += _vehicleInstance.transform.forward * (_speed * Tin
```

4) Ganganzeige (UI)

Am einfachsten: ein TMP-Text im gleichen Canvas.

Unity-Schritte:

1. **Hierarchy** → **Canvas** → **MenuRoot** (oder ein extra Panel) → Rechtsklick → **UI** → **Text - TextMeshPro**.
2. Benenne es Txt_Gear.
3. Im VehicleLocomotionMVP:

```
using TMPro;
[SerializeField] private TMP_Text gearText;
```

4. Update-Methode:

```
private void UpdateGearUI()
{
    if (gearText == null) return;
    gearText.text = (_gear == 0) ? "Gear: R" : $"Gear: {_gear}";
}
```

5. Rufe UpdateGearUI() auf in EnterVehicle() und immer wenn du schaltest.

Wenn du „RightSelect/LeftSelect“ als konkrete Buttons sagst (A/B oder Stick-Click?), gebe ich dir die exakte `OVRInput.Button`-Zuordnung, damit Upshift/Downshift wirklich auf deinen gewünschten Tasten liegt.

✱✱

using UnityEngine;

```
public class VehicleLocomotionMVP : MonoBehaviour, ILocomotionTechnique
{
    [Header("Vehicle")]
    [SerializeField] private GameObject vehiclePrefab;
    [SerializeField] private float moveSpeed = 3.0f;
    [SerializeField] private float turnSpeed = 60f;

    [Header("Rig refs")]
    [SerializeField] private Transform rigRoot;    // meist OVRCameraRig (Root den du bewegen
willst)
    [SerializeField] private Transform head;      // CenterEyeAnchor

    [Header("Seat")]
    [SerializeField] private Vector3 spawnOffset = new Vector3(0, 0, 1.5f);
    [SerializeField] private bool autoEnterOnEnable = true;

    [Header("Hands for wheel steering")]
    [SerializeField] private Transform leftController; // z.B. TrackingSpace/LeftControllerAnchor
    [SerializeField] private Transform rightController; // z.B. TrackingSpace/RightControllerAnchor
    [SerializeField] private float steerGain = 8f;
    [SerializeField] private bool invertSteer = true;

    [Header("Pedals (triggers)")]
    [SerializeField] private float accel = 6f;
    [SerializeField] private float brake = 10f;

    [Header("Gears")]
    [SerializeField] private int minGear = 1;
    [SerializeField] private int maxGear = 5;
    [SerializeField] private float[] gearMaxSpeed = { 0f, 3f, 5f, 7f, 9f, 11f }; // Index = gear
    [SerializeField] private OVRInput.Button upShiftButton = OVRInput.Button.One; // meist A
(rechts) / X (links) je nach Controller [web:25]
    [SerializeField] private OVRInput.Button downShiftButton = OVRInput.Button.Three;

    [SerializeField] private float steerDeadzoneMeters = 0.05f; // 5cm
    [SerializeField] private float steerGain = 8f;
    [SerializeField] private bool invertSteer = true;

    private int _gear;
    private float _speed;
```

```

    private float _neutralHeightDelta;

    public bool IsActive { get; private set; }

    private GameObject _vehicleInstance;
    private Transform _seatAnchor;
    private bool _isSeated;
    private Vector3 _rigPosBeforeSeat;
    private Quaternion _rigRotBeforeSeat;
    private Transform _rigParentBeforeSeat;
    private Vector3 _rigLocalPosBeforeSeat;
    private Quaternion _rigLocalRotBeforeSeat;

    public void Initialize(TechniqueContext context) { }

    public void SetActive(bool active)
    {
        IsActive = active;
        enabled = active;

        if (active)
        {
            SpawnVehicle();
            if (autoEnterOnEnable) EnterVehicle();
        }
        else
        {
            ExitVehicle();
            DespawnVehicle();
        }
    }

    private void SpawnVehicle()
    {
        if (_vehicleInstance != null || vehiclePrefab == null || head == null) return;

        Vector3 forward = Vector3.ProjectOnPlane(head.forward, Vector3.up).normalized;
        if (forward.sqrMagnitude < 0.001f) forward = head.forward;

        Vector3 pos = head.position + forward * spawnOffset.z + Vector3.up * spawnOffset.y +
            head.right * spawnOffset.x;
        _vehicleInstance = Instantiate(vehiclePrefab, pos, Quaternion.LookRotation(forward,
            Vector3.up));

        _seatAnchor = _vehicleInstance.transform.Find("SeatAnchor");
        if (_seatAnchor == null) _seatAnchor = _vehicleInstance.transform;
    }

```

```

private void DespawnVehicle()
{
    if (_vehicleInstance == null) return;
    Destroy(_vehicleInstance);
    _vehicleInstance = null;
    _seatAnchor = null;
    _isSeated = false;
}

public void EnterVehicle()
{
    if (_vehicleInstance == null || rigRoot == null || _seatAnchor == null) return;
    if (_isSeated) return;

    // Backup
    _rigParentBeforeSeat = rigRoot.parent;
    _rigPosBeforeSeat = rigRoot.position;
    _rigRotBeforeSeat = rigRoot.rotation;
    _rigLocalPosBeforeSeat = rigRoot.localPosition;
    _rigLocalRotBeforeSeat = rigRoot.localRotation;

    // Parent to seat so we follow vehicle motion
    rigRoot.SetParent(_seatAnchor, worldPositionStays: true);

    // Snap into seat (local = 0)
    rigRoot.localPosition = Vector3.zero;
    rigRoot.localRotation = Quaternion.identity;

    _isSeated = true;
    _gear = Mathf.Clamp(_gear == 0 ? 1 : _gear, minGear, maxGear);

    if (leftController != null && rightController != null)
        _neutralHeightDelta = leftController.position.y - rightController.position.y;
    else
        _neutralHeightDelta = 0f;
}

public void ExitVehicle()
{
    if (!_isSeated || rigRoot == null) return;

    // Restore parent
    rigRoot.SetParent(_rigParentBeforeSeat, worldPositionStays: true);

    // Restore world pose (sauberer als local restore)
    rigRoot.position = _rigPosBeforeSeat;
    rigRoot.rotation = _rigRotBeforeSeat;
}

```

```

_isSeated = false;
}

private void Update()
{
    if (!IsActive || _vehicleInstance == null || !_isSeated) return;

    // --- Steering: Controller als Lenkrad (Höhe links vs rechts) ---
    float heightDelta = 0f;
    if (leftController != null && rightController != null)
        heightDelta = (leftController.position.y - rightController.position.y) - _neutralHeightDelta;

    float steer = Mathf.Clamp(heightDelta * steerGain, -1f, 1f);
    if (invertSteer) steer = -steer; // links höher → rechts lenken

    // --- Pedals: Trigger ---
    float gas = OVRInput.Get(OVRInput.Axis1D.PrimaryIndexTrigger); // Left Trigger [web:23]
    float bremse = OVRInput.Get(OVRInput.Axis1D.SecondaryIndexTrigger); // Right Trigger [web:23]

    // --- Gears: Buttons ---
    if (OVRInput.GetDown(upShiftButton)) _gear = Mathf.Min(_gear + 1, maxGear);
    if (OVRInput.GetDown(downShiftButton)) _gear = Mathf.Max(_gear - 1, minGear);

    float maxSpeed = (_gear < gearMaxSpeed.Length) ? gearMaxSpeed[_gear] :
        gearMaxSpeed[gearMaxSpeed.Length - 1];

    // Geschwindigkeit integrieren
    _speed += gas * accel * Time.deltaTime;
    _speed -= bremse * brake * Time.deltaTime;
    _speed = Mathf.Clamp(_speed, 0f, maxSpeed);

    // Bewegung anwenden
    _vehicleInstance.transform.Rotate(0f, steer * turnSpeed * Time.deltaTime, 0f, Space.World);
    _vehicleInstance.transform.position += _vehicleInstance.transform.forward * (_speed *
        Time.deltaTime);
}
}

```

Passe das skript entsprechend an bitte

Ja—dein Script ist aktuell inkonsistent (du hast `steerGain/invertSteer` doppelt drin) und es fehlen Deadzone-Logik, Rückwärtsgang, Ganganzeige sowie „kein Drehen im Stand“. Unten ist eine **bereinigte, komplette** Version von `VehicleLocomotionMVP.cs`, die genau das umsetzt (Gas/Bremse weiter über Index-Trigger-Achsen).^[476]

VehicleLocomotionMVP.cs (komplett ersetzen)

Datei: Assets/Scripts/Techniques/Implementations/Vehicle/VehicleLocomotionMVP.cs

```
using UnityEngine;
using TMPPro;

public class VehicleLocomotionMVP : MonoBehaviour, ILocomotionTechnique
{
    [Header("Vehicle")]
    [SerializeField] private GameObject vehiclePrefab;
    [SerializeField] private float turnSpeed = 60f;

    [Header("Rig refs")]
    [SerializeField] private Transform rigRoot; // OVRCameraRig
    [SerializeField] private Transform head;    // CenterEyeAnchor

    [Header("Seat")]
    [SerializeField] private Vector3 spawnOffset = new Vector3(0, 0, 1.5f);
    [SerializeField] private bool autoEnterOnEnable = true;

    [Header("Hands for wheel steering")]
    [SerializeField] private Transform leftController; // TrackingSpace/LeftController/
    [SerializeField] private Transform rightController; // TrackingSpace/RightController/
    [SerializeField] private float steerDeadzoneMeters = 0.05f; // 5 cm
    [SerializeField] private float steerGain = 8f;
    [SerializeField] private bool invertSteer = true;

    [Header("Pedals (Index triggers)")]
    [SerializeField] private float accelStrength = 6f;
    [SerializeField] private float brakeStrength = 10f;

    [Header("Gears")]
    [SerializeField] private int maxForwardGear = 5; // 1..maxForwardGear, Reverse = 0
    [SerializeField] private float maxReverseSpeed = 3f;
    [SerializeField] private float[] forwardGearMaxSpeed = { 0f, 3f, 5f, 7f, 9f, 11f }; //
    [SerializeField] private OVRInput.Button upShiftButton = OVRInput.Button.One;
    [SerializeField] private OVRInput.Button downShiftButton = OVRInput.Button.Three;

    [Header("Steering rule")]
    [SerializeField] private float minSpeedToTurn = 0.05f; // kein Drehen im Stand (m/s)

    [Header("UI (optional)")]
    [SerializeField] private TMP_Text gearText;

    public bool IsActive { get; private set; }

    private GameObject _vehicleInstance;
    private Transform _seatAnchor;

    private bool _isSeated;
    private float _neutralHeightDelta;

    private int _gear = 1; // 0 = R, 1.. = vorwärts
    private float _speed = 0f; // kann negativ sein (R)
```



```

private Vector3 _rigPosBeforeSeat;
private Quaternion _rigRotBeforeSeat;
private Transform _rigParentBeforeSeat;

public void Initialize(TechniqueContext context) { }

public void SetActive(bool active)
{
    IsActive = active;
    enabled = active;

    if (active)
    {
        SpawnVehicle();
        if (autoEnterOnEnable) EnterVehicle();
    }
    else
    {
        ExitVehicle();
        DespawnVehicle();
    }
}

private void SpawnVehicle()
{
    if (_vehicleInstance != null || vehiclePrefab == null || head == null) return;

    Vector3 forward = Vector3.ProjectOnPlane(head.forward, Vector3.up).normalized;
    if (forward.sqrMagnitude < 0.001f) forward = head.forward;

    Vector3 pos = head.position + forward * spawnOffset.z + Vector3.up * spawnOffset.y;
    _vehicleInstance = Instantiate(vehiclePrefab, pos, Quaternion.LookRotation(forward));

    _seatAnchor = _vehicleInstance.transform.Find("SeatAnchor");
    if (_seatAnchor == null) _seatAnchor = _vehicleInstance.transform;
}

private void DespawnVehicle()
{
    if (_vehicleInstance == null) return;
    Destroy(_vehicleInstance);
    _vehicleInstance = null;
    _seatAnchor = null;
    _isSeated = false;
}

public void EnterVehicle()
{
    if (_vehicleInstance == null || rigRoot == null || _seatAnchor == null) return;
    if (_isSeated) return;

    _rigParentBeforeSeat = rigRoot.parent;
    _rigPosBeforeSeat = rigRoot.position;
    _rigRotBeforeSeat = rigRoot.rotation;

    rigRoot.SetParent(_seatAnchor, worldPositionStays: true);
}

```

```

    rigRoot.localPosition = Vector3.zero;
    rigRoot.localRotation = Quaternion.identity;

    _isSeated = true;

    // Null-Lenkrad: aktuelle Höhendifferenz merken
    if (leftController != null && rightController != null)
        _neutralHeightDelta = leftController.position.y - rightController.position.y;
    else
        _neutralHeightDelta = 0f;

    UpdateGearUI();
}

public void ExitVehicle()
{
    if (!_isSeated || rigRoot == null) return;

    rigRoot.SetParent(_rigParentBeforeSeat, worldPositionStays: true);
    rigRoot.position = _rigPosBeforeSeat;
    rigRoot.rotation = _rigRotBeforeSeat;

    _isSeated = false;
}

private void Update()
{
    if (!IsActive || _vehicleInstance == null || !_isSeated) return;

    // --- Inputs ---
    float gas = OVRInput.Get(OVRInput.Axis1D.PrimaryIndexTrigger); // 0..1 [web:
    float brake = OVRInput.Get(OVRInput.Axis1D.SecondaryIndexTrigger); // 0..1 [web:

    // Shift: 0=R, 1..maxForwardGear
    if (OVRInput.GetDown(upShiftButton))
    {
        if (_gear == 0) _gear = 1;
        else _gear = Mathf.Min(_gear + 1, maxForwardGear);
        UpdateGearUI();
    }

    if (OVRInput.GetDown(downShiftButton))
    {
        if (_gear == 1) _gear = 0;
        else if (_gear > 1) _gear -= 1;
        UpdateGearUI();
    }

    // --- Steering (height delta with deadzone) ---
    float heightDelta = 0f;
    if (leftController != null && rightController != null)
        heightDelta = (leftController.position.y - rightController.position.y) - _neu

    float steer = 0f;
    float abs = Mathf.Abs(heightDelta);
    if (abs > steerDeadzoneMeters)

```

```

    {
        float sign = Mathf.Sign(heightDelta);
        float effective = abs - steerDeadzoneMeters;
        steer = sign * Mathf.Clamp(effective * steerGain, 0f, 1f);
        if (invertSteer) steer = -steer; // links höher -> rechts lenken
    }

    // --- Speed integration (with reverse) ---
    float dir = (_gear == 0) ? -1f : 1f;

    float maxFwd = GetForwardMaxSpeed(_gear);
    float maxSpd = (_gear == 0) ? maxReverseSpeed : maxFwd;

    _speed += (gas * accelStrength * Time.deltaTime) * dir;
    _speed = Mathf.MoveTowards(_speed, 0f, brake * brakeStrength * Time.deltaTime);

    if (_gear == 0) _speed = Mathf.Clamp(_speed, -maxSpd, 0f);
    else _speed = Mathf.Clamp(_speed, 0f, maxSpd);

    // --- No turning while standing still ---
    float turnInput = (Mathf.Abs(_speed) < minSpeedToTurn) ? 0f : steer;

    // Apply movement
    _vehicleInstance.transform.Rotate(0f, turnInput * turnSpeed * Time.deltaTime, 0f,
    _vehicleInstance.transform.position += _vehicleInstance.transform.forward * (_spe
}

private float GetForwardMaxSpeed(int gear)
{
    if (gear < 1) return 0f;

    if (forwardGearMaxSpeed != null && gear < forwardGearMaxSpeed.Length)
        return forwardGearMaxSpeed[gear];

    // Fallback: letzter Eintrag
    if (forwardGearMaxSpeed != null && forwardGearMaxSpeed.Length > 0)
        return forwardGearMaxSpeed[forwardGearMaxSpeed.Length - 1];

    return 0f;
}

private void UpdateGearUI()
{
    if (gearText == null) return;
    gearText.text = (_gear == 0) ? "Gear: R" : $"Gear: {_gear}";
}
}

```

Ganganzeige einrichten (1 Minute)

1. Canvas → MenuRoot → Rechtsklick → **UI** → **Text** - **TextMeshPro** (nenne es z.B. Txt_Gear).
2. Ziehe Txt_Gear (die TMP_Text-Komponente) ins Feld gearText vom VehicleLocomotionMVP.

Wenn du magst, erweitern wir als nächstes die Anzeige um Speed (z.B. „R 2.1 m/s“), und ich passe Upshift/Downshift exakt auf deine „RightSelect/LeftSelect“-Definition an (A/X vs. Stick-

Click).



Die Ganganzeige soll bitte in der vehicle locomotion immer im fov sichtbar sein

Ja—mach die Ganganzeige als **eigenes Head-locked HUD**, getrennt vom Menü. Dann ist sie während Vehicle-Locomotion immer im Sichtfeld, unabhängig davon ob MenuRoot zu ist. ^[503]

HUD in die Scene bauen

1. **Hierarchy** → Rechtsklick → **UI** → **Canvas**.
2. Canvas auswählen → Inspector:
 - **Render Mode = World Space**
 - Scale klein setzen (z.B. 0.002, 0.002, 0.002).
3. Unter diesem Canvas: Rechtsklick → **UI** → **Text - TextMeshPro** → nenne es GearHUDText.
4. Setze GearHUDText im Canvas oben/zentral (z.B. Y = +200), Schrift groß genug.

HUD an den Kopf "kleben"

1. Ziehe das HUD-Canvas als Child unter
OVRCameraRig → TrackingSpace → CenterEyeAnchor (dein Head).
2. Setze die Local-Transform vom HUD-Canvas:
 - Local Position z.B. (0, -0.15, 1.2) (leicht unter Blickmitte, 1.2m vor dir)
 - Local Rotation (0, 0, 0)

Damit bleibt es immer im FOV und folgt Kopfbewegungen.

VehicleLocomotionMVP anpassen (HUD nur im Vehicle-Mode sichtbar)

Im Script nutzt du schon TMP_Text gearText. Du musst nur:

1. gearText auf GearHUDText referenzieren.
2. Den HUD bei Aktivierung/Deaktivierung togglen:

Füge in SetActive(bool active) direkt nach enabled = active; ein:

```
if (gearText != null)
    gearText.gameObject.SetActive(active);
```

Und in EnterVehicle() einmal UpdateGearUI(); (hast du schon).

Wichtig: Menü nicht wieder "mit togglen"

Die HUD sollte **nicht** unter `MenuRoot` hängen, sonst verschwindet sie wenn du das Menü schließt (genau das willst du vermeiden).

Wenn du willst, kann ich dir als nächstes noch eine kleine Speed-Anzeige ergänzen (z.B. „R | 2.3 m/s“), die ebenfalls im HUD landet.

✱

Ich glaube, ich habe beim HUD einen Fehler gemacht. Kannst du die stzruktur überprüfen und mir ggfs vorschläge amchen, warum ich ihn nicht sehe?

Die Hierarchie sieht grundsätzlich **plausibel** aus (HUD als Child vom `Rig`, plus `GearHUDText` darunter). Dass du ihn nicht siehst, liegt in VR in 90% der Fälle an

Transform/Scale/RenderMode oder daran, dass `GearHUDText` gar keinen sichtbaren Text rendert (z.B. leer, Alpha 0, zu klein). [\[522\]](#)

Struktur-Check (was ich sehe)

- Dein HUD hängt aktuell direkt unter `OVRCameraRig` (nicht unter `CenterEyeAnchor`). [\[522\]](#)
Das kann funktionieren, aber dann musst du die HUD-Position/Rotation so setzen, dass sie zur Head-Orientierung passt – sonst ist es leicht „hinter dir“ oder weit weg.

Fix 1 (empfohlen): HUD an CenterEyeAnchor hängen

1. Ziehe in der Hierarchy HUD per Drag&Drop unter `OVRCameraRig` → `TrackingSpace` → `CenterEyeAnchor`. [\[522\]](#)
2. Wähle HUD aus → im Inspector bei Transform:
 - **Local Position:** (0, -0.15, 1.2)
 - **Local Rotation:** (0, 0, 0)
 - **Local Scale:** erstmal (1, 1, 1) (Scale machen wir am Canvas/RectTransform, nicht hier)

Damit ist es garantiert im FOV und rotiert mit dem Kopf.

Fix 2: Canvas Settings prüfen (wenn HUD ein Canvas ist)

In deinem Screenshot sehe ich nur HUD und `GearHUDText`, aber nicht, ob HUD wirklich ein **Canvas** ist. [\[522\]](#)

Wähle HUD aus und prüfe im Inspector:

- Hat HUD eine **Canvas**-Komponente?
 - Wenn nein: Dann ist `GearHUDText` vermutlich nur ein `GameObject` ohne UI-Renderer → erstelle ein Canvas und lege den Text darunter.

- Wenn ja:
 - **Render Mode = World Space**
 - RectTransform Scale: z.B. 0.002, 0.002, 0.002 (oder größer, falls zu klein)
 - Canvas → Sorting Order testweise hoch setzen (z.B. 50), damit er nicht „hinter“ anderem UI verschwindet.

Fix 3: TextMeshPro sichtbar machen

Wähle GearHUDText aus und prüfe:

- Text ist nicht leer (z.B. „Gear: 1“).
- Farbe/Alpha ist nicht transparent (Alpha = 1).
- Font Size deutlich erhöhen (z.B. 60–120, abhängig von Scale).
- Z-Position: Wenn GearHUDText local z = 0 ist, ist das okay; wichtig ist, dass **HUD** selbst vor der Kamera liegt.

Fix 4: Wird der Text evtl. vom Script deaktiviert?

Wenn du in VehicleLocomotionMVP diese Zeile drin hast:

```
gearText.gameObject.SetActive(active);
```

dann ist der Text nur sichtbar, wenn VehicleMode aktiv ist. Das ist gewollt, kann aber beim Testen verwirren. Stelle sicher, dass du im Vehicle-Mode bist, und dass im VehicleLocomotionMVP-Inspector das Feld gearText wirklich auf GearHUDText zeigt.

Schnellster Debug:

1. HUD auswählen → setze **Local Position** mal auf (0, 0, 0.5) und **Scale** (Canvas RectTransform) deutlich größer.
2. GearHUDText auf „TEST“ setzen.
Wenn du's dann siehst, war's nur Placement/Scale.

Wenn du mir einen Screenshot vom **Inspector von HUD** (Canvas/RectTransform sichtbar) schickst, kann ich dir 1:1 sagen, welcher konkrete Parameter falsch ist.



Ok, ger:1 wird jetzt angezeigt, allerdings nicht in bzw dekrementiert und der rückwärtsgang fehlt noch

Dass „Gear: 1“ angezeigt wird, aber sich nie ändert, hat fast immer eine von zwei Ursachen: (a) deine Shift-Buttons feuern nicht (falsches OVRInput.Button gemappt), oder (b) deine aktuelle Ganglogik erlaubt kein 0 (Reverse), weil du mit `minGear = 1 clampst`.^[542]

1) Warum Reverse fehlt (bei deinem geposteten Script)

In deinem Script steht `minGear = 1` und du machst beim Downshift `Mathf.Max(_gear - 1, minGear)` – dadurch kommst du **nie** auf 0, also gibt es keinen Rückwärtsgang.

Fix (minimal): Gang 0 erlauben

- Entferne `minGear` komplett (oder setze ihn auf 0).
- Ersetze deine Shift-Logik in `Update()` durch:

```
// --- Gears: 0 = R, 1..maxGear = vorwärts ---
if (OVRInput.GetDown(upShiftButton))
{
    if (_gear == 0) _gear = 1;                // R -> 1
    else _gear = Mathf.Min(_gear + 1, maxGear);
    UpdateGearUI();
}

if (OVRInput.GetDown(downShiftButton))
{
    if (_gear == 1) _gear = 0;                // 1 -> R
    else if (_gear > 1) _gear -= 1;
    UpdateGearUI();
}
```

Und passe die `MaxSpeed`-Auswahl so an:

```
[SerializeField] private float maxReverseSpeed = 3f;

// ...
float maxSpeed = (_gear == 0)
    ? maxReverseSpeed
    : ((_gear < gearMaxSpeed.Length) ? gearMaxSpeed[_gear] : gearMaxSpeed[gearMaxSpeed.Length - 1]);
```

Dann musst du noch `_speed` für Reverse negativ werden lassen (sonst fährt er nie rückwärts):

```
float dir = (_gear == 0) ? -1f : 1f;

_speed += gas * accel * Time.deltaTime * dir;
// Bremse zieht Richtung 0 (egal ob vorwärts oder rückwärts)
_speed = Mathf.MoveTowards(_speed, 0f, bremsen * brake * Time.deltaTime);

// Clamp
if (_gear == 0) _speed = Mathf.Clamp(_speed, -maxSpeed, 0f);
else _speed = Mathf.Clamp(_speed, 0f, maxSpeed);
```

2) Warum der Gang nicht hoch/runter zählt

Deine `OVRInput.Button.One / Three` sind evtl. nicht die Tasten, die du drückst (oder du meinst mit „Select“ etwas anderes wie Stick-Click). `Button.One` entspricht in vielen Quest/OVR-Mappings dem **A-Button**.^[542]

Fix: Shift-Buttons im Inspector richtig setzen

In `VehicleLocomotionMVP` hast du `upShiftButton` und `downShiftButton` als `[SerializeField]` – stell die im Inspector testweise so ein:

- Upshift: `One` (A)
- Downshift: `Three` (X)

Wenn du eigentlich „RightSelect/LeftSelect“ = **Thumbstick-Click** meinst, setze stattdessen:

- Upshift: `SecondaryThumbstick` (rechter Stick Click)
- Downshift: `PrimaryThumbstick` (linker Stick Click)

Debug (10 Sekunden)

Füge temporär direkt in die Shift-Abfragen ein:

```
if (OVRInput.GetDown(upShiftButton)) Debug.Log("Upshift pressed");  
if (OVRInput.GetDown(downShiftButton)) Debug.Log("Downshift pressed");
```

Wenn du beim Drücken nichts in der Console siehst, ist das Mapping noch falsch.

3) „Kein Drehen im Stand“ (falls du’s noch nicht drin hast)

Vor dem Rotate:

```
float minSpeedToTurn = 0.05f;  
float turnInput = (Mathf.Abs(_speed) < minSpeedToTurn) ? 0f : steer;  
_vehicleInstance.transform.Rotate(0f, turnInput * turnSpeed * Time.deltaTime, 0f, Space.V
```

Wenn du mir kurz sagst, was bei dir genau „RightSelect/LeftSelect“ bedeutet (A/X? Grip? Stick-Click?), gebe ich dir die **exakte** `OVRInput.Button`-Belegung für Upshift/Downshift.

✱✱

vergiss es, ich hatte andere buttons erwartet, aber das können wir später remappen.

Jetzt wollen wir bitte drive physics implementieren. Vorschläge?

Für „Drive Physics“ hast du in Unity zwei sinnvolle Wege—und für Quest würde ich mit **WheelCollider** anfangen, weil du sehr schnell zu fahrbarem Ergebnis kommst (Unity beschreibt WheelColliders als raycast-basiert und dafür gedacht). Wenn du maximale Kontrolle/Arcade-Feel willst, ist danach ein eigener **Raycast-Suspension Controller** der nächste Schritt. [\[571\]](#) [\[572\]](#) [\[573\]](#)

Option 1 (empfohlen als nächster Schritt): WheelCollider-Vehicle

Vorteile

- Schnell integrierbar, viele Parameter sind „Standard“ (MotorTorque, BrakeTorque, SteerAngle). [\[573\]](#)
- Gute Basis, um später Lenkrad/Schubhebel (B) anzuschließen.

Setup (konkret)

1. Öffne dein Car-Prefab im Prefab Mode.
2. Root: Rigidbody hinzufügen (nicht kinematisch), Mass z.B. 1200–2000, Interpolate = Interpolate.
3. Erstelle 4 Empty GameObjects unter dem Root: WC_FL, WC_FR, WC_RL, WC_RR (Position jeweils im Zentrum des Rads).
4. Auf jedes WC_... → **Add Component** → **WheelCollider**.
5. Erstelle zusätzlich 4 Transforms für die visuellen Räder (oder nutze deine vorhandenen Wheel-Meshes) und synchronisiere sie per Script mit `WheelCollider.GetWorldPose(...)`.

Script-Änderung

- In `VehicleLocomotionMVP` ersetzt du das direkte `transform.position += ...` durch:
 - `WheelCollider.steerAngle` aus deiner Steering-Logik,
 - `WheelCollider.motorTorque` aus Gas,
 - `WheelCollider.brakeTorque` aus Bremse.

Wenn du willst, gebe ich dir als nächsten Schritt ein fertiges `WheelColliderVehicleController.cs`, das du nur mit 4 WheelColliders und 4 WheelMeshes verdrahtest.

Option 2 (später / „Arcade & kontrolliert“): Raycast Suspension (Custom)

Das ist der Ansatz „4 Rays nach unten + Feder/Dämpfer + ForceAtPosition“, wie er in vielen Arcade-Car-Tutorials erklärt wird. Er ist super, wenn WheelCollider dir zu „floaty“ ist oder du sehr enge Parcours-Kontrolle brauchst, aber dauert deutlich länger zu tunen. [\[572\]](#)

Empfehlung für deinen Parcours

1. Jetzt: **WheelCollider** → schnell fahrbar + Kollisionen „echt“. [\[573\]](#)
2. Wenn nötig: danach Raycast-Suspension (Option 2) als „VehiclePhysicsV2“.

Nächste Frage (entscheidet den Implementationsplan)

Willst du, dass sich das Auto physikalisch „real“ verhält (rutscht/kippt/rollt), oder eher „Arcade“ (stabil, kaum kippen, gut kontrollierbar) für den Parcours?

✱

wir fahre mit option 1 fort, ich habe an die 4 räder wheel collider angefügt, visual transform ist erstmal nicht nötig, da die räder aus der fahrerperspektive unsichtbar sind

Super – dann ist der nächste Schritt: **Rigidbody + WheelCollider** wirklich nutzen (kein kinematisches Transform-Moving mehr), und dein VehicleLocomotionMVP wird zum reinen **Input/Mode-Controller**. WheelColliders werden in Unity über `motorTorque`, `brakeTorque` und `steerAngle` gesteuert. [\[606\]](#)

1) Prefab-Setup (einmalig)

1. Öffne dein Auto-Prefab im **Prefab Mode**.
2. **Root (ScifiCar_Prefab)**: Rigidbody
 - Is Kinematic **aus** (false).
 - Interpolate = Interpolate (optional).
3. WheelCollider-GameObjects müssen **Childs des Roots** sein und ihre Transforms bleiben fix relativ zum Auto (WheelCollider-Objekte werden nicht „gedreht/gerollt“, nur die Werte werden gesetzt). [\[607\]](#)
4. Optional (stabiler gegen Umkippen): Rigidbody → Center Of Mass tiefer setzen (z.B. y -0.3 bis -0.8, je nach Größe), das macht Fahrzeuge oft deutlich stabiler. [\[608\]](#)

2) Neues Script auf dem Fahrzeug: WheelColliderDrive

Wir legen ein Script an, das **auf dem Vehicle-Prefab** sitzt und alle 4 WheelCollider-Referenzen hält (damit du sie im Prefab verdrahten kannst).

Script anlegen

1. **Assets** → **Scripts** → **Techniques** → **Implementations** → **Vehicle** → Rechtsklick → Create → C# Script → WheelColliderDrive.

Inhalt (komplett)

```
using UnityEngine;

public class WheelColliderDrive : MonoBehaviour
{
    [Header("Refs")]
    [SerializeField] private Rigidbody rb;
    [SerializeField] private WheelCollider frontLeft;
    [SerializeField] private WheelCollider frontRight;
    [SerializeField] private WheelCollider rearLeft;
    [SerializeField] private WheelCollider rearRight;

    [Header("Tuning")]
    [SerializeField] private float maxSteerAngle = 32f;
    [SerializeField] private float maxMotorTorque = 1800f;
    [SerializeField] private float maxBrakeTorque = 3500f;
    [SerializeField] private float minSpeedToSteer = 0.2f; // m/s, kein Lenken im Stand

    private float _steer;    // -1..1
    private float _throttle; // 0..1
    private float _brake;    // 0..1
    private int _gear;        // 0=R, 1.. = vorwärts

    public float SpeedMS => rb != null ? rb.velocity.magnitude : 0f;

    public void SetInputs(float steer, float throttle, float brake, int gear)
    {
        _steer = Mathf.Clamp(steer, -1f, 1f);
        _throttle = Mathf.Clamp01(throttle);
        _brake = Mathf.Clamp01(brake);
        _gear = gear;
    }

    private void Reset()
    {
        rb = GetComponent<Rigidbody>();
    }

    private void FixedUpdate()
    {
        if (rb == null) return;

        float speed = rb.velocity.magnitude;

        // Steering: nur wenn bewegt
        float steerAngle = (speed < minSpeedToSteer) ? 0f : _steer * maxSteerAngle;
        if (frontLeft != null) frontLeft.steerAngle = steerAngle;
        if (frontRight != null) frontRight.steerAngle = steerAngle;

        // Motor direction: Reverse -> negatives Torque
        float dir = (_gear == 0) ? -1f : 1f;
        float motor = _throttle * maxMotorTorque * dir;

        // Brake (und beim Bremsen Motor wegnehmen)
        float brakeTorque = _brake * maxBrakeTorque;
```

```

        if (brakeTorque > 0.01f) motor = 0f;

        // Rear-wheel drive (einfach); wenn du AWD willst: auch vorne motorTorque setzen
        if (rearLeft != null) rearLeft.motorTorque = motor;
        if (rearRight != null) rearRight.motorTorque = motor;

        if (rearLeft != null) rearLeft.brakeTorque = brakeTorque;
        if (rearRight != null) rearRight.brakeTorque = brakeTorque;
        if (frontLeft != null) frontLeft.brakeTorque = brakeTorque;
        if (frontRight != null) frontRight.brakeTorque = brakeTorque;
    }
}

```

Im Prefab verdrahten

1. Prefab Root auswählen (ScifiCar_Prefab).
2. **Add Component** → WheelColliderDrive.
3. Felder zuweisen:
 - Rb = Rigidbody am Root
 - FrontLeft/FrontRight/RearLeft/RearRight = deine 4 WheelCollider-Komponenten

3) VehicleLocomotionMVP: auf Physik umstellen

Dein VehicleLocomotionMVP soll jetzt **nicht mehr** das Fahrzeug per Transform bewegen, sondern nur Inputs berechnen und an WheelColliderDrive schicken. WheelColliders werden über motorTorque/brakeTorque/steerAngle gefahren.^[606]

Änderungen

1. In VehicleLocomotionMVP füge ein Feld hinzu:

```
private WheelColliderDrive _drive;
```

2. In SpawnVehicle() nach dem Instantiieren:

```
_drive = _vehicleInstance.GetComponent<WheelColliderDrive>();
```

3. In Update() entferne diese beiden Zeilen komplett:

```
_vehicleInstance.transform.Rotate(...);
_vehicleInstance.transform.position += ...;
```

4. Stattdessen am Ende von Update():

```
if (_drive != null)
    _drive.SetInputs(steer, gas, bremse, _gear);
```

Damit bewegt sich die Kamera weiterhin mit, weil du den `RigRoot` an `SeatAnchor` parentest, aber die Bewegung kommt jetzt sauber aus der Physik. [\[606\]](#)

4) Typische Stolpersteine (wenn "es bewegt sich nicht")

- `Rigidbody` ist noch `Is Kinematic = true` → dann passiert physikalisch nichts.
- `WheelCollider`-Referenzen im `WheelColliderDrive` sind nicht gesetzt.
- `Collider/Auto` "steht in der Luft": `WheelCollider Radius/Suspension Distance` sind unpassend; dann hat er keinen Bodenkontakt.

Wenn du willst: Schick mir einen Screenshot von einem `WheelCollider-Inspector` (Radius, Suspension Distance, Center), dann kann ich dir Startwerte für dein Car empfehlen.

✱

Es scheint alles zu funktionieren, allerdings springt das kfz aktuell nur wild in der gegend umher und kommt nicht zur ruhe

Das „wild umherspringen“ kommt bei `WheelColliders` fast immer von (a) **zusätzlichen Collidern** an Rädern/Car-Teilen, die mit dem `WheelCollider` „gegen“ den Boden arbeiten, oder (b) zu aggressiver **Suspension/Friction** (Spring/Damper/Stiffness) bzw. unruhiger Track-Collision-Geo. [\[636\]](#) [\[637\]](#)

1) Collider-Setup bereinigen (wichtigster Fix)

- Entferne/deaktiviere alle **MeshCollider** an den Wheel-Meshes und sonstigen Deko-Teilen (FrontLeftWheel/Rear.../Parts), und lass nur **einen** simplen Collider für die Karosse (BoxCollider oder wenige primitive Collider) am Root/Body.
- `WheelCollider` basiert auf Raycasts und erwartet, dass `WheelCollider`-Objekte relativ zum Auto fix bleiben; zusätzliche physische Wheel-Mesh-Collider erzeugen oft genau dieses "Bouncen/Explodieren". [\[638\]](#) [\[636\]](#)

2) Suspension beruhigen (Spring/Damper)

In jedem `WheelCollider` → Suspension Spring:

- Erhöhe den **Damper** deutlich (zu niedrig = bouncy), und senke ggf. den **Spring** etwas. [\[637\]](#)
- Startwerte (nur als grobe Hausnummer für ein ~1500kg Auto): Spring 20000–40000, Damper 3000–8000, Target Position ~0.5.

Wenn es immer noch hüpft: Damper weiter hoch, Spring etwas runter (kritische Dämpfung anpeilen). [\[637\]](#)

3) Friction/Stiffness zähmen

WheelCollider-Friction läuft über eine eigene Slip/Friction-Logik; "Stiffness" ist ein starker Multiplikator. [\[636\]](#)

- Setze `ForwardFriction.stiffness` und `SidewaysFriction.stiffness` testweise auf 0.6–1.0 (statt 2+), bis es stabil ist. [\[636\]](#)

4) Track/Umgebungs-Collider glätten

Wenn dein Boden/Track sehr detailreiche MeshCollider hat (kleine Kanten/Bumps), kann das WheelCollider-Raycasts "hacken" und das Fahrzeug springen lassen. [\[636\]](#)

- Nutze für den Parcours am besten einen vereinfachten, glatten Collision-Mesh (separat vom sichtbaren Mesh). [\[636\]](#)

5) Quick-Debug in 2 Minuten

1. Setze im `WheelColliderDrive` temporär `maxMotorTorque = 0` und `maxBrakeTorque = 0`.
2. Wenn er *immer noch* springt → Problem ist Collider/Suspension/Track (nicht Input/Torque).
3. Wenn er dann ruhig ist → Torque zu hoch oder du brauchst Drag/AngularDrag am Rigidbody (z.B. Drag 0.1, Angular Drag 0.5–2.0).

Wenn du mir einen Screenshot von **einem WheelCollider-Inspector** (Radius, Suspension Distance, Suspension Spring + Forward/Sideways Friction) und vom **Rigidbody** (Mass/Drag/AngularDrag/IsKinematic) schickst, kann ich dir konkrete Startwerte geben, die zu deinem Car-Scale passen.



springt auch mit torque auf 0 noch

Wenn es auch mit `motorTorque = 0` springt, ist das praktisch sicher ein **Collision/Suspension-Setup** (WheelCollider + Colliders + Boden), nicht dein Input. [\[669\]](#) [\[670\]](#)

1) Self-collision ausschließen

1. Deaktiviere/entferne **alle MeshCollider an den Rädern** (Wheel-Meshes), nutze stattdessen nur WheelColliders für "Rad ↔ Boden". [\[669\]](#)
2. Für die Karosserie verwende **1–3 primitive Collider** (Box/Capsule), keine vielen MeshCollider auf Sub-Teilen (die erzeugen oft Penetrations-/Solver-Stress). [\[669\]](#)
3. Stelle sicher, dass an den WheelCollider-Objekten **kein Rigidbody** hängt (nur am Root des Autos). [\[671\]](#)

2) WheelCollider-Position/Radius prüfen (häufigster Auslöser)

- In der Scene View: WheelCollider auswählen und die Gizmos checken; der Kreis muss **zentriert im Rad** sitzen, und der Boden sollte im Stand "knapp" unterhalb des Radius liegen (nicht dass die WheelCollider-Spur schon im Boden steckt). [\[669\]](#)
- Wenn dein Chassis-Collider den Boden berührt (z.B. Unterboden zu tief), "prallt" das Auto auch ohne Torque ständig hoch; erhöhe den Body-Collider oder setze das Auto minimal höher. [\[672\]](#)

3) Suspension beruhigen (Spring/Damper)

WheelCollider nutzt `suspensionSpring` (Spring/Damper); zu wenig Dämpfung macht genau dieses Hüpfen. [\[670\]](#)

- Start-Fix: **Damper hoch, Spring runter**, danach fein tunen. [\[673\]](#) [\[670\]](#)

4) Friction/Stiffness runter

Zu hohe `Forward/SidewaysFriction.stiffness` kann Jitter/Hop verstärken; setz testweise beide auf ~0.6–1.0 und taste dich hoch. [\[673\]](#) [\[669\]](#)

5) Physics-Substeps erhöhen (oft der "letzte" Stabilitätshebel)

Unity erlaubt es, für Fahrzeuge zusätzliche Substeps zu konfigurieren (reduziert WheelCollider-Jitter in manchen Setups). [\[674\]](#)

Füge z.B. im `WheelColliderDrive` einmalig in `Start()` hinzu:

```
private void Start()
{
    if (frontLeft != null) frontLeft.ConfigureVehicleSubsteps(5f, 12, 15);
    if (frontRight != null) frontRight.ConfigureVehicleSubsteps(5f, 12, 15);
    if (rearLeft != null) rearLeft.ConfigureVehicleSubsteps(5f, 12, 15);
    if (rearRight != null) rearRight.ConfigureVehicleSubsteps(5f, 12, 15);
}
```

Schnellster Debug-Plan (damit wir die Ursache eindeutig finden)

1. Karosserie: **nur 1 BoxCollider** am Root, sonst keine Collider am Auto außer WheelColliders. [\[669\]](#)
2. Auto 0.5 m über den Boden setzen, Play drücken: fällt es ruhig auf die Räder oder explodiert es weiter?
3. Wenn es weiter explodiert: Screenshot von **einem WheelCollider Inspector** (Radius, Suspension Distance, Spring/Damper, Forward/Sideways friction) + Rigidbody (Mass/Drag/AngularDrag/Collision Detection).

Sag mir noch: Hast du aktuell am Auto (außer WheelColliders) noch MeshCollider aktiv, und berührt der Unterboden im Stand den Boden?

nach einführen von 5 gibt es weniger, aber dafür aggressivere bounces

Das Verhalten „weniger, aber aggressivere Bounces“ nach Schritt 5 passt dazu, dass `ConfigureVehicleSubsteps` die Wheel-Simulation in mehr Substeps aufteilt und die dabei berechneten Kräfte/Torque pro Substep aufsummiert—bei einem instabilen Setup werden die einzelnen Impulse zwar seltener, können aber stärker wirken. ^[702]

Sofortmaßnahmen (in Reihenfolge)

1. **Suspension weicher + stärker gedämpft:** Senke `Spring` deutlich und erhöhe `Damper`, weil eine harte Feder Energie sehr schnell zurückgibt und zu Hopsern führt, während ein stärkerer Damper das Ausschwingen reduziert. ^[703] ^[704]
2. **Spring/Damper passend zur Masse skalieren:** Unitys Default (Spring 35000, Damper 4500) ist auf ca. 1500 kg Fahrzeugmasse ausgelegt; wenn dein RigidBody viel leichter/schwerer ist oder dein Model skaliert wurde, musst du die Werte proportional anpassen. ^[703]
3. **Target Position prüfen:** Lass `suspensionSpring.targetPosition` erstmal bei ~0.5 (typisch 0.3–0.7), weil extreme Werte die Feder ständig in Richtung Anschlag treiben können. ^[704]

Häufigste “Explodierer”-Ursache: Interpenetration

Wenn der Body/irgendein Collider im Stand auch nur minimal den Boden berührt (oder der WheelCollider-Radius/Center so ist, dass das Rad „im Boden startet“), versucht der Solver das zu lösen und es gibt Sprünge. ^[705]

- Test: Stelle das Auto im Play Mode deutlich höher (z.B. 0.5–1 m über Ground) und beobachte, ob es beim Aufsetzen ruhig einfedert oder sofort “kickt” (dann sind Radius/Center/SuspensionDistance oder Body-Collider-Höhe falsch). ^[705]

Physik-Stabilität erhöhen (wenn Setup ok ist)

Erhöhe in **Project Settings** → **Physics** die Solver Iterations (z.B. Default 6 → 12 oder 20), weil mehr Iterationen Kontaktauflösung stabilisieren können (kostet Performance). ^[706]

Wenn du Substeps nutzt, reduziere außerdem die Substep-Zahlen testweise wieder (z.B. `speedThreshold=5`, `below=5`, `above=8`), bis du ein ruhiges Grundverhalten hast. ^[702]

Was ich von dir brauche, um es präzise zu fixen

Schick bitte einen Screenshot von **einem** WheelCollider-Inspector (Radius, Suspension Distance, Spring/Damper/Target Position, Forward/Sideways friction) und vom **RigidBody** (Mass, Drag, Angular Drag, Collision Detection). Dann kann ich dir konkrete Startwerte geben, die zu deinem Maßstab/Masse passen.

Ok, ich fürchte, dass da aktuell eine kleine Wand den fortschritt hindert. können wir die physics kurz zurückdrehen (abschalten um basisfunktionalität zu erhalten) und woanders weiterarbeiten?

Ja—das ist absolut sinnvoll: wir drehen die Physik zurück auf den **kinematischen** MVP-Drive, damit du wieder stabil weiterarbeiten kannst (HUD, Controls, Enter/Exit, später Full Controls), und nehmen die WheelCollider-Physics später nochmal strukturiert in Angriff. `Rigidbody.isKinematic = true` sorgt dafür, dass Kräfte/Kollisionen/Joints das Rigidbody nicht mehr beeinflussen und du die Bewegung wieder per Script/Transform steuern kannst. ^[736]

Schritt 1: Wheel-Physics "aus"

Am Vehicle-Prefab (Root des Autos):

1. **WheelColliderDrive deaktivieren** (Häkchen links an der Komponente aus) oder Komponente entfernen.
2. Alle **WheelCollider**-Komponenten deaktivieren (Häkchen aus) – so kann nichts mehr „federn“. ^[737]
3. Rigidbody am Root:
 - `Is Kinematic = true` (damit Physik nicht mehr eingreift). ^[736]
 - Optional: `Use Gravity = false`, wenn du komplett „schwebend“ fahren willst (für Parcours oft okay).

Schritt 2: Kinematic-Bewegung wieder aktivieren

In `VehicleLocomotionMVP`:

- Stelle sicher, dass du wieder die *alte* Bewegung nutzt (dein Transform-Move/Rotate bzw. `_speed Integration + transform.position += forward * ...`).
- Wenn du zuletzt auf `_drive.SetInputs(...)` umgestellt hast: diese Zeile entfernen und wieder die Transform-Bewegung einbauen.

Wenn du Kollisionen trotzdem möchtest (ohne WheelCollider), ist ein guter Zwischenweg:

Kinematic Rigidbody + MovePosition/MoveRotation (in `FixedUpdate`) statt `transform.position`, weil das robuster mit Collidern zusammenspielt. ^[738]

Schritt 3: Minimaler Stabilitäts-Check

- Auto spawnen → EnterVehicle → fahren/lenken → ExitVehicle → alles ruhig?
Wenn ja, sind wir wieder auf „Baseline“.

Wo wir dann weiterarbeiten können (ohne Physik-Blocker)

- Lenkrad-Deadzone/Steering-UX weiter tunen.
- HUD (Gear + Speed) polish, ggf. Komfortfeatures (Vignette, Snap-turn sperren im Vehicle-Mode).
- Vorbereitung für Locomotion B: Interactable Lenkrad + Schubhebel (ohne Drive-Physics erstmal nur als Input-Quelle).

Eine kurze Rückfrage: willst du beim kinematischen Drive **mit Kollisionen** (also nicht durch Wände) fahren, oder reicht erstmal „no-clip“ für Prototyping?

✱✱

using UnityEngine;

```
using TMPro;
```

```
public class VehicleLocomotionMVP : MonoBehaviour, ILocomotionTechnique
{
    [Header("Vehicle")]
    [SerializeField] private GameObject vehiclePrefab;
    [SerializeField] private float turnSpeed = 60f;

    [Header("Rig refs")]
    [SerializeField] private Transform rigRoot; // OVRCameraRig
    [SerializeField] private Transform head; // CenterEyeAnchor

    [Header("Seat")]
    [SerializeField] private Vector3 spawnOffset = new Vector3(0, 0, 1.5f);
    [SerializeField] private bool autoEnterOnEnable = true;

    [Header("Hands for wheel steering")]
    [SerializeField] private Transform leftController; // TrackingSpace/LeftControllerAnchor
    [SerializeField] private Transform rightController; // TrackingSpace/RightControllerAnchor
    [SerializeField] private float steerDeadzoneMeters = 0.05f; // 5 cm
    [SerializeField] private float steerGain = 8f;
    [SerializeField] private bool invertSteer = true;

    [Header("Pedals (Index triggers)")]
    [SerializeField] private float accelStrength = 6f;
    [SerializeField] private float brakeStrength = 10f;

    [Header("Gears")]
    [SerializeField] private int maxForwardGear = 5; // 1..maxForwardGear, Reverse = 0
    [SerializeField] private float maxReverseSpeed = 3f;
    [SerializeField] private float[] forwardGearMaxSpeed = { 0f, 3f, 5f, 7f, 9f, 11f }; // Index
    1..maxForwardGear
```

```

[SerializeField] private OVRInput.Button upShiftButton = OVRInput.Button.One;
[SerializeField] private OVRInput.Button downShiftButton = OVRInput.Button.Three;

[Header("Steering rule")]
[SerializeField] private float minSpeedToTurn = 0.05f; // kein Drehen im Stand (m/s)

[Header("UI (optional)")]
[SerializeField] private TMP_Text gearText;

private WheelColliderDrive _drive;

public bool IsActive { get; private set; }

private GameObject _vehicleInstance;
private Transform _seatAnchor;

private bool _isSeated;
private float _neutralHeightDelta;

private int _gear = 1; // 0 = R, 1.. = vorwärts
private float _speed = 0f; // kann negativ sein (R)

private Vector3 _rigPosBeforeSeat;
private Quaternion _rigRotBeforeSeat;
private Transform _rigParentBeforeSeat;

public void Initialize(TechniqueContext context) { }

public void SetActive(bool active)
{
    IsActive = active;
    enabled = active;
    if (gearText != null)
        gearText.gameObject.SetActive(active);

    if (active)
    {
        SpawnVehicle();
        if (autoEnterOnEnable) EnterVehicle();
    }
    else
    {
        ExitVehicle();
        DespawnVehicle();
    }
}

private void SpawnVehicle()
{
    if (_vehicleInstance != null || vehiclePrefab == null || head == null) return;

```

```

Vector3 forward = Vector3.ProjectOnPlane(head.forward, Vector3.up).normalized;
    if (forward.sqrMagnitude < 0.001f) forward = head.forward;

Vector3 pos = head.position + forward * spawnOffset.z + Vector3.up * spawnOffset.y +
head.right * spawnOffset.x;
    _vehicleInstance = Instantiate(vehiclePrefab, pos, Quaternion.LookRotation(forward,
Vector3.up));

_seaAnchor = _vehicleInstance.transform.Find("SeatAnchor");
    if (_seaAnchor == null) _seaAnchor = _vehicleInstance.transform;

_drive = _vehicleInstance.GetComponent<WheelColliderDrive>();

}

private void DespawnVehicle()
{
    if (_vehicleInstance == null) return;
    Destroy(_vehicleInstance);
    _vehicleInstance = null;
    _seaAnchor = null;
    _isSeated = false;
}

public void EnterVehicle()
{
    if (_vehicleInstance == null || rigRoot == null || _seaAnchor == null) return;
    if (_isSeated) return;

    _rigParentBeforeSeat = rigRoot.parent;
    _rigPosBeforeSeat = rigRoot.position;
    _rigRotBeforeSeat = rigRoot.rotation;

    rigRoot.SetParent(_seaAnchor, worldPositionStays: true);
    rigRoot.localPosition = Vector3.zero;
    rigRoot.localRotation = Quaternion.identity;

    _isSeated = true;

    // Null-Lenkrad: aktuelle Höhendifferenz merken
    if (leftController != null && rightController != null)
        _neutralHeightDelta = leftController.position.y - rightController.position.y;
    else
        _neutralHeightDelta = 0f;

    UpdateGearUI();
}

public void ExitVehicle()
{

```

```

        if (!_isSeated || rigRoot == null) return;

    rigRoot.SetParent(_rigParentBeforeSeat, worldPositionStays: true);
    rigRoot.position = _rigPosBeforeSeat;
    rigRoot.rotation = _rigRotBeforeSeat;

    _isSeated = false;
}

private void Update()
{
    if (!IsActive || _vehicleInstance == null || !_isSeated) return;

    // --- Inputs ---
    float gas = OVRInput.Get(OVRInput.Axis1D.PrimaryIndexTrigger);    // 0..1 [web:25]
    float brake = OVRInput.Get(OVRInput.Axis1D.SecondaryIndexTrigger); // 0..1 [web:25]

    // Shift: 0=R, 1..maxForwardGear
    if (OVRInput.GetDown(upShiftButton))
    {
        if (_gear == 0) _gear = 1;
        else _gear = Mathf.Min(_gear + 1, maxForwardGear);
        UpdateGearUI();
    }

    if (OVRInput.GetDown(downShiftButton))
    {
        if (_gear == 1) _gear = 0;
        else if (_gear > 1) _gear -= 1;
        UpdateGearUI();
    }

    // --- Steering (height delta with deadzone) ---
    float heightDelta = 0f;
    if (leftController != null && rightController != null)
        heightDelta = (leftController.position.y - rightController.position.y) - _neutralHeightDelta;

    float steer = 0f;
    float abs = Mathf.Abs(heightDelta);
    if (abs > steerDeadzoneMeters)
    {
        float sign = Mathf.Sign(heightDelta);
        float effective = abs - steerDeadzoneMeters;
        steer = sign * Mathf.Clamp(effective * steerGain, 0f, 1f);
        if (invertSteer) steer = -steer; // links höher → rechts lenken
    }

    // --- Speed integration (with reverse) ---
    float dir = (_gear == 0) ? -1f : 1f;

```

```

float maxFwd = GetForwardMaxSpeed(_gear);
float maxSpd = (_gear == 0) ? maxReverseSpeed : maxFwd;

_speed += (gas * accelStrength * Time.deltaTime) * dir;
_speed = Mathf.MoveTowards(_speed, 0f, brake * brakeStrength * Time.deltaTime);

if (_gear == 0) _speed = Mathf.Clamp(_speed, -maxSpd, 0f);
else _speed = Mathf.Clamp(_speed, 0f, maxSpd);

// --- No turning while standing still ---
float turnInput = (Mathf.Abs(_speed) < minSpeedToTurn) ? 0f : -steer;

if (_drive != null)
    _drive.SetInputs(steer, gas, brake, _gear);
}

private float GetForwardMaxSpeed(int gear)
{
    if (gear < 1) return 0f;

    if (forwardGearMaxSpeed != null && gear < forwardGearMaxSpeed.Length)
        return forwardGearMaxSpeed[gear];

    // Fallback: letzter Eintrag
    if (forwardGearMaxSpeed != null && forwardGearMaxSpeed.Length > 0)
        return forwardGearMaxSpeed[forwardGearMaxSpeed.Length - 1];

    return 0f;
}

private void UpdateGearUI()
{
    if (gearText == null) return;
    gearText.text = (_gear == 0) ? "Gear: R" : $"Gear: {_gear}";
}
}

```

Kommentiere die physics aus und reintegriere das alte setup bitte

Ja—wir schalten WheelCollider-Physics jetzt komplett aus und gehen zurück auf **kinematisches** Fahren (Script bewegt das Fahrzeug), damit du wieder eine stabile Basis hast. Für kinematische Rigidbodies ist `Rigidbody.MovePosition/MoveRotation` in `FixedUpdate()` der sauberste Weg (statt `transform.position`), weil Unity das für kinematische Bewegung so vorsieht und Interpolation korrekt greifen kann.^[768]

1) Physik am Prefab deaktivieren

Am Fahrzeug-Prefab (Root):

- WheelColliderDrive **disable** (Häkchen aus) oder entfernen.
- Alle WheelCollider **disable** (Häkchen aus).
- Rigidbody: isKinematic = true; optional Interpolate = Interpolate für smooth. ^[768]

2) VehicleLocomotionMVP: Wheel-Physics auskommentieren + kinematic movement rein

Ersetze deine Klasse durch diese Version (wichtig: WheelColliderDrive _drive ist raus, und wir bewegen das Auto wieder selbst):

```
using UnityEngine;
using TMPPro;

public class VehicleLocomotionMVP : MonoBehaviour, ILocomotionTechnique
{
    [Header("Vehicle")]
    [SerializeField] private GameObject vehiclePrefab;
    [SerializeField] private float turnSpeed = 60f;

    [Header("Kinematic Drive")]
    [SerializeField] private float minSpeedToTurn = 0.05f; // kein Drehen im Stand (m/s)

    [Header("Rig refs")]
    [SerializeField] private Transform rigRoot; // OVRCameraRig
    [SerializeField] private Transform head; // CenterEyeAnchor

    [Header("Seat")]
    [SerializeField] private Vector3 spawnOffset = new Vector3(0, 0, 1.5f);
    [SerializeField] private bool autoEnterOnEnable = true;

    [Header("Hands for wheel steering")]
    [SerializeField] private Transform leftController;
    [SerializeField] private Transform rightController;
    [SerializeField] private float steerDeadzoneMeters = 0.05f;
    [SerializeField] private float steerGain = 8f;
    [SerializeField] private bool invertSteer = true;

    [Header("Pedals (Index triggers)")]
    [SerializeField] private float accelStrength = 6f;
    [SerializeField] private float brakeStrength = 10f;

    [Header("Gears")]
    [SerializeField] private int maxForwardGear = 5; // 1..maxForwardGear, Reverse = 0
    [SerializeField] private float maxReverseSpeed = 3f;
    [SerializeField] private float[] forwardGearMaxSpeed = { 0f, 3f, 5f, 7f, 9f, 11f };
    [SerializeField] private OVRInput.Button upShiftButton = OVRInput.Button.One;
    [SerializeField] private OVRInput.Button downShiftButton = OVRInput.Button.Three;

    [Header("UI")]
```

```

[SerializeField] private TMP_Text gearText;

public bool IsActive { get; private set; }

private GameObject _vehicleInstance;
private Transform _seatAnchor;
private Rigidbody _vehicleRb;

private bool _isSeated;
private float _neutralHeightDelta;

private int _gear = 1;    // 0 = R, 1.. = vorwärts
private float _speed = 0f; // kann negativ sein (R)

private Vector3 _rigPosBeforeSeat;
private Quaternion _rigRotBeforeSeat;
private Transform _rigParentBeforeSeat;

// Für FixedUpdate Bewegung
private float _steerCached;

public void Initialize(TechniqueContext context) { }

public void SetActive(bool active)
{
    IsActive = active;
    enabled = active;

    if (gearText != null)
        gearText.gameObject.SetActive(active);

    if (active)
    {
        SpawnVehicle();
        if (autoEnterOnEnable) EnterVehicle();
    }
    else
    {
        ExitVehicle();
        DespawnVehicle();
    }
}

private void SpawnVehicle()
{
    if (_vehicleInstance != null || vehiclePrefab == null || head == null) return;

    Vector3 forward = Vector3.ProjectOnPlane(head.forward, Vector3.up).normalized;
    if (forward.sqrMagnitude < 0.001f) forward = head.forward;

    Vector3 pos = head.position + forward * spawnOffset.z + Vector3.up * spawnOffset.y;
    _vehicleInstance = Instantiate(vehiclePrefab, pos, Quaternion.LookRotation(forward));

    _seatAnchor = _vehicleInstance.transform.Find("SeatAnchor");
    if (_seatAnchor == null) _seatAnchor = _vehicleInstance.transform;
}

```



```

        _vehicleRb = _vehicleInstance.GetComponent<Rigidbody>(); // optional
        if (_vehicleRb != null) _vehicleRb.isKinematic = true;
    }

    private void DespawnVehicle()
    {
        if (_vehicleInstance == null) return;
        Destroy(_vehicleInstance);
        _vehicleInstance = null;
        _seatAnchor = null;
        _vehicleRb = null;
        _isSeated = false;
    }

    public void EnterVehicle()
    {
        if (_vehicleInstance == null || rigRoot == null || _seatAnchor == null) return;
        if (_isSeated) return;

        _rigParentBeforeSeat = rigRoot.parent;
        _rigPosBeforeSeat = rigRoot.position;
        _rigRotBeforeSeat = rigRoot.rotation;

        rigRoot.SetParent(_seatAnchor, worldPositionStays: true);
        rigRoot.localPosition = Vector3.zero;
        rigRoot.localRotation = Quaternion.identity;

        _isSeated = true;

        if (leftController != null && rightController != null)
            _neutralHeightDelta = leftController.position.y - rightController.position.y;
        else
            _neutralHeightDelta = 0f;

        UpdateGearUI();
    }

    public void ExitVehicle()
    {
        if (!_isSeated || rigRoot == null) return;

        rigRoot.SetParent(_rigParentBeforeSeat, worldPositionStays: true);
        rigRoot.position = _rigPosBeforeSeat;
        rigRoot.rotation = _rigRotBeforeSeat;

        _isSeated = false;
    }

    private void Update()
    {
        if (!IsActive || _vehicleInstance == null || !_isSeated) return;

        float gas = OVRInput.Get(OVRInput.Axis1D.PrimaryIndexTrigger);
        float brake = OVRInput.Get(OVRInput.Axis1D.SecondaryIndexTrigger);

        // Shifting: 0=R, 1..maxForwardGear

```

```

if (OVRInput.GetDown(upShiftButton))
{
    if (_gear == 0) _gear = 1;
    else _gear = Mathf.Min(_gear + 1, maxForwardGear);
    UpdateGearUI();
}

if (OVRInput.GetDown(downShiftButton))
{
    if (_gear == 1) _gear = 0;
    else if (_gear > 1) _gear -= 1;
    UpdateGearUI();
}

// Steering with deadzone
float heightDelta = 0f;
if (leftController != null && rightController != null)
    heightDelta = (leftController.position.y - rightController.position.y) - _neu

float steer = 0f;
float abs = Mathf.Abs(heightDelta);
if (abs > steerDeadzoneMeters)
{
    float sign = Mathf.Sign(heightDelta);
    float effective = abs - steerDeadzoneMeters;
    steer = sign * Mathf.Clamp(effective * steerGain, 0f, 1f);
    if (invertSteer) steer = -steer;
}

// Speed integration (reverse)
float dir = (_gear == 0) ? -1f : 1f;
float maxFwd = GetForwardMaxSpeed(_gear);
float maxSpd = (_gear == 0) ? maxReverseSpeed : maxFwd;

_speed += (gas * accelStrength * Time.deltaTime) * dir;
_speed = Mathf.MoveTowards(_speed, 0f, brake * brakeStrength * Time.deltaTime);

if (_gear == 0) _speed = Mathf.Clamp(_speed, -maxSpd, 0f);
else _speed = Mathf.Clamp(_speed, 0f, maxSpd);

_steerCached = steer;
}

private void FixedUpdate()
{
    if (!IsActive || _vehicleInstance == null || !_isSeated) return;

    // Kein Drehen im Stand
    float steer = (Mathf.Abs(_speed) < minSpeedToTurn) ? 0f : _steerCached;

    Quaternion deltaRot = Quaternion.Euler(0f, steer * turnSpeed * Time.fixedDeltaTime, 0f);
    Vector3 deltaPos = _vehicleInstance.transform.forward * (_speed * Time.fixedDeltaTime);

    if (_vehicleRb != null)
    {
        // Kinematic RB bewegen -> in FixedUpdate [web:118]
    }
}

```

```

        _vehicleRb.MoveRotation(_vehicleRb.rotation * deltaRot);
        _vehicleRb.MovePosition(_vehicleRb.position + deltaPos);
    }
    else
    {
        // Fallback ohne Rigidbody
        _vehicleInstance.transform.rotation = _vehicleInstance.transform.rotation * c
        _vehicleInstance.transform.position += deltaPos;
    }
}

private float GetForwardMaxSpeed(int gear)
{
    if (gear < 1) return 0f;

    if (forwardGearMaxSpeed != null && gear < forwardGearMaxSpeed.Length)
        return forwardGearMaxSpeed[gear];

    if (forwardGearMaxSpeed != null && forwardGearMaxSpeed.Length > 0)
        return forwardGearMaxSpeed[forwardGearMaxSpeed.Length - 1];

    return 0f;
}

private void UpdateGearUI()
{
    if (gearText == null) return;
    gearText.text = (_gear == 0) ? "Gear: R" : $"Gear: {_gear}";
}
}

```

Was du danach noch machen musst

- Am Prefab wirklich WheelColliders **deaktivieren** + Rigidbody isKinematic = true, sonst "funkt" die Physik weiterhin rein. [\[769\]](#)
- Falls du noch irgendwo WheelColliderDrive im Prefab/Scene aktiv hast: aus.

Wenn du mir sagst „Kollisionen ja/nein“ für diesen kinematischen Modus, kann ich dir als nächsten Schritt eine einfache Collision-Variante geben (z.B. CapsuleCast nach vorne, damit du nicht durch Wände fährst), ohne WheelCollider-Physik wieder anzufassen.



ok, zeit für eine neue interaction&locomotion method, das tweaken aller methoden kommt später.

In der neuen methode soll ein cockpit simuliert werden. in diesem cockpit sollen zwei hebel, ein joystick, zwei knöpfe und zwei stellräder sein.

Die hebel sollen wie ein kettenlaufwerk funktionieren (beide vor → vor, linker vor → rechts drehen, rechter vor → links drehen und entsprechend rückwärts) der joystick soll ein crosshair in

4 richtungen steuern. die knöpfe sollen select(interaction) oder destroy(coins) sein und die stellräder in der interaction rotation ermöglichen. Ich könnte ein panzer prefab verwenden, weiss aber nicht in wie weit ich da den trum separat drehen kann.

Ja, das passt sehr gut als **eigene Technique** in eurem System: ein „CockpitMode“, der (a) ein Cockpit-Prefab spawn, (b) den Player in den Sitz „dockt“, (c) Inputs ausschließlich aus Interactions (Hebel/Joystick/Knöpfe/Stellräder) liest und damit den Tank bewegt bzw. UI/Interact steuert. Das lässt sich mit Meta XR Interaction SDK sauber aufbauen (Grabbable + OneGrab... Transformer, PokeInteractable für Buttons). [\[803\]](#) [\[804\]](#)

Architekturvorschlag (damit es nicht chaotisch wird)

- CockpitLocomotionTechnique (analog zu deinem VehicleMode): spawn Cockpit/Tank, parentet RigRoot an SeatAnchor, toggelt HUD/Crosshair.
- CockpitControls (sitzt auf dem Cockpit-Prefab): hält Referenzen auf alle Controls und liefert **normalisierte Werte**:
 - leftTrack $\in [-1, 1]$ (Hebel links: rückwärts..vorwärts)
 - rightTrack $\in [-1, 1]$
 - joyX, joyY $\in [-1, 1]$ fürs Crosshair
 - selectPressed, destroyPressed (Buttons)
 - rotA, rotB $\in [-1, 1]$ (Stellräder für Rotation)
- TankMover (sitzt auf Tank-Root): nimmt leftTrack/rightTrack und bewegt kinematisch (erstmal wie dein MVP; Physics kann später kommen).

Tank-Drive (zwei Hebel = Kettenlaufwerk)

Du bekommst das gewünschte Verhalten mit **Differential Drive**:

- Vorwärtsgeschwindigkeit $v = (L + R)/2$
- Drehgeschwindigkeit $\omega = (R - L)/2$

Damit gilt automatisch:

- Beide vor $\rightarrow$ vor
- Links vor, rechts 0 $\rightarrow$ Rechtskurve
- Rechts vor, links 0 $\rightarrow$ Linkskurve
- Einer vor, einer zurück $\rightarrow$ Drehung auf der Stelle

Das ist genau das, was du beschrieben hast. [\[805\]](#)

Interactions: so baust du die Controls (Meta Interaction SDK)

Hebel (2x)

- Nimm ein Lever-Objekt mit `Grabbable + OneGrabRotateTransformer` (Achse z.B. local X) und clamp auf z.B. $[-35^\circ, +35^\circ]$. [\[806\]](#) [\[803\]](#)
- Aus der lokalen Rotation liest du einen normierten Wert $[-1, 1]$ (wird `leftTrack/rightTrack`).

Joystick (1x)

- Technisch wie ein „2D constrained“ Grab: entweder 2 Achsen über (a) 2 DoF-Mechanik (zwei Rotationsachsen), oder (b) ein kleines Grab-Target, das du in einer Box begrenzt und daraus X/Y ableitest (Arcade, sehr robust).
- Damit steuerst du ein Crosshair im HUD (4 Richtungen ist dann einfach: du kannst optional auf die dominante Achse „snappen“).

Buttons (2x)

- Nutze `PokeInteractable` (oder das fertige Prefab) und hänge an das Press-Event deine Methoden („Select“ / „Destroy coin“). [\[807\]](#) [\[804\]](#)

Stellräder (2x)

- Ebenfalls `OneGrabRotateTransformer` (um lokale Z oder Y Achse), clamp z.B. $[-90^\circ, +90^\circ]$ oder endlos mit Wrap (für später). [\[808\]](#) [\[806\]](#)
- Die zwei Werte kannst du auf „Rotation in Interaction“ mappen: z.B. `rotA = yaw`, `rotB = pitch` fürs gezielte Ausrichten/Rotieren eines selected Objects.

Panzer-Prefab & Turm drehen

Ja, du kannst ein Panzer-Prefab verwenden. Der Turm muss dafür **separates Child-Objekt** sein (z.B. `Hull + Turret + Barrel`).

- Wenn dein Prefab das nicht hat, kannst du es oft trotzdem lösen, indem du im Prefab die Turm-Mesh-Transform als eigenen Node findest und gezielt rotierst.
- Falls es wirklich „ein einziges Mesh“ ist: dann ist Turm separat drehen kaum möglich ohne Re-Rigging/Splitting (dann wäre ein anderes Prefab besser).

Konkreter nächster Schritt (damit wir starten können)

1. Spawne ein „Cockpit“ als leeres Prototype-Prefab: `SeatAnchor`, `Lever_L`, `Lever_R`, `Joystick`, `Btn_Select`, `Btn_Destroy`, `Dial_A`, `Dial_B`.
2. Wir implementieren zuerst nur:
 - Hebel → Tank bewegen (kinematisch),
 - Joystick → Crosshair bewegen,
 - Buttons → `Debug.Log` (später echte Funktion),
 - Dials → `Debug.Log` Rotation value.

Schick mir bitte einen Screenshot aus der Hierarchy deines Panzer-Prefabs (aufgeklappt), damit ich dir sagen kann, ob der Turm bereits separat ist und wie wir die Referenzen am saubersten benennen (z.B. `TurretPivot`).



Ich habe nachgesehen, das tank prefab ist in der grafik.

Beginne mich schritt für schritt nach dem bewährten system durch deinen eben vorgestellten prozess zur vollständigen implementierung zu führen.

Sehr gut: dein Tank-Prefab ist **hierarchisch getrennt**, d.h. du kannst Turm/Barrel separat drehen (z.B. `Tank_03_UpperBody` für Turret-Yaw und `Tank_03_MainGunPart/Main_Gun` für Barrel-Pitch). Wir gehen jetzt Schritt für Schritt wie beim `VehicleMode` vor und bauen daraus eine dritte Technique „`CockpitTank`“. [\[843\]](#) [\[844\]](#)

Schritt 1: Prefab vorbereiten (Tank + Anchors)

1. Dupliziere dein Tank-Prefab (damit du das Original nicht "kaputtkonfigurierst"):

`Tank_03_Prefab` → Duplicate → `Tank_03_CockpitPrefab`. [\[843\]](#)

2. Öffne `Tank_03_CockpitPrefab` im Prefab Mode.

3. Lege diese leeren Child-Transforms am Root `Tank_03` an:

- `SeatAnchor` (Sitzposition des Spielers)
- `CockpitRoot` (wo deine Controls dranhängen)
- `TurretPivot` (optional; oder wir referenzieren direkt `Tank_03_UpperBody`)

4. Setze `SeatAnchor` auf die Position „im Cockpit“, Blickrichtung nach vorne. [\[843\]](#)

Schritt 2: Cockpit UI-Objekte anlegen (ohne Interaction SDK, nur Werte)

Wir machen zuerst "Dummy Controls" als `GameObjects`, die später zu `Interactables` werden.

Unter `CockpitRoot` erstelle:

- `Lever_L` (linker Hebel)
- `Lever_R` (rechter Hebel)
- `Joystick`
- `Btn_Select`
- `Btn_Destroy`
- `Dial_RotA`
- `Dial_RotB`

Fürs Prototyping kannst du das einfach als `Cubes/Cylinders` modellieren (schnell), oder direkt deine späteren `Meshes` nutzen.

Schritt 3: TankMover (Differential Drive) implementieren

Erstelle Script:

Assets/Scripts/Techniques/Implementations/CockpitTank/TankDifferentialMover.cs

```
using UnityEngine;

public class TankDifferentialMover : MonoBehaviour
{
    [SerializeField] private Transform tankRoot; // meist this.transform
    [SerializeField] private float maxForwardSpeed = 4f; // m/s
    [SerializeField] private float maxReverseSpeed = 2f; // m/s
    [SerializeField] private float maxTurnRateDeg = 90f; // deg/s
    [SerializeField] private float accel = 6f; // m/s^2
    [SerializeField] private float turnAccel = 240f; // deg/s^2

    private float _v; // current forward speed
    private float _omega; // current yaw rate deg/s

    public void Drive(float leftTrack, float rightTrack)
    {
        leftTrack = Mathf.Clamp(leftTrack, -1f, 1f);
        rightTrack = Mathf.Clamp(rightTrack, -1f, 1f);

        float vCmd = (leftTrack + rightTrack) * 0.5f; // -1..1
        float wCmd = (rightTrack - leftTrack) * 0.5f; // -1..1

        float targetV = (vCmd >= 0f) ? vCmd * maxForwardSpeed : vCmd * maxReverseSpeed;
        float targetW = wCmd * maxTurnRateDeg;

        _v = Mathf.MoveTowards(_v, targetV, accel * Time.deltaTime);
        _omega = Mathf.MoveTowards(_omega, targetW, turnAccel * Time.deltaTime);

        tankRoot.position += tankRoot.forward * (_v * Time.deltaTime);
        tankRoot.Rotate(0f, _omega * Time.deltaTime, 0f, Space.World);
    }
}
```

Hänge das Script auf Tank_03 (Root) im Prefab und setze tankRoot = Tank_03.

Schritt 4: CockpitControls (liefert Werte) – erstmal per Tastatur/Controller debug

Erstelle Script: CockpitControls.cs im gleichen Ordner.

```
using UnityEngine;

public class CockpitControls : MonoBehaviour
{
    [Header("Outputs (-1..1)")]
    [Range(-1,1)] public float leftTrack;
    [Range(-1,1)] public float rightTrack;
    [Range(-1,1)] public float joyX;
    [Range(-1,1)] public float joyY;
    [Range(-1,1)] public float rotA;
```

```

[Range(-1,1)] public float rotB;
public bool selectPressed;
public bool destroyPressed;

// DEBUG input so everything works before Interaction wiring
private void Update()
{
    // Tank: WASD als Debug (später kommen Hebel)
    float fwd = Input.GetKey(KeyCode.W) ? 1f : Input.GetKey(KeyCode.S) ? -1f : 0f;
    float turn = Input.GetKey(KeyCode.D) ? 1f : Input.GetKey(KeyCode.A) ? -1f : 0f;

    leftTrack = Mathf.Clamp(fwd - turn, -1f, 1f);
    rightTrack = Mathf.Clamp(fwd + turn, -1f, 1f);

    // Joystick: Pfeiltasten
    joyX = (Input.GetKey(KeyCode.RightArrow) ? 1f : 0f) + (Input.GetKey(KeyCode.LeftArrow) ? -1f : 0f);
    joyY = (Input.GetKey(KeyCode.UpArrow) ? 1f : 0f) + (Input.GetKey(KeyCode.DownArrow) ? -1f : 0f);

    // Buttons
    selectPressed = Input.GetKeyDown(KeyCode.E);
    destroyPressed = Input.GetKeyDown(KeyCode.Q);

    // Dials
    rotA = Input.GetKey(KeyCode.Z) ? -1f : Input.GetKey(KeyCode.X) ? 1f : 0f;
    rotB = Input.GetKey(KeyCode.C) ? -1f : Input.GetKey(KeyCode.V) ? 1f : 0f;
}
}

```

Hänge das Script auf CockpitRoot im Prefab.

Schritt 5: CockpitTankLocomotionTechnique (spawnen, einsteigen, verbinden)

Erstelle Script: CockpitTankLocomotionTechnique.cs

```

using UnityEngine;

public class CockpitTankLocomotionTechnique : MonoBehaviour, ILocomotionTechnique
{
    [SerializeField] private GameObject tankCockpitPrefab;
    [SerializeField] private Transform rigRoot;
    [SerializeField] private Transform head;
    [SerializeField] private Vector3 spawnOffset = new Vector3(0, 0, 2f);
    [SerializeField] private bool autoEnterOnEnable = true;

    private GameObject _instance;
    private Transform _seat;
    private bool _seated;

    private Transform _rigParentBefore;
    private Vector3 _rigPosBefore;
    private Quaternion _rigRotBefore;

    private TankDifferentialMover _mover;
    private CockpitControls _controls;
}

```



```

public bool IsActive { get; private set; }
public void Initialize(TechniqueContext context) { }

public void SetActive(bool active)
{
    IsActive = active;
    enabled = active;

    if (active)
    {
        Spawn();
        if (autoEnterOnEnable) Enter();
    }
    else
    {
        Exit();
        Despawn();
    }
}

private void Spawn()
{
    if (_instance != null || tankCockpitPrefab == null || head == null) return;

    Vector3 fwd = Vector3.ProjectOnPlane(head.forward, Vector3.up).normalized;
    if (fwd.sqrMagnitude < 0.001f) fwd = head.forward;

    Vector3 pos = head.position + fwd * spawnOffset.z + Vector3.up * spawnOffset.y +
    _instance = Instantiate(tankCockpitPrefab, pos, Quaternion.LookRotation(fwd, Vector3.up));

    _seat = _instance.transform.Find("SeatAnchor");
    if (_seat == null) _seat = _instance.transform;

    _mover = _instance.GetComponentInChildren<TankDifferentialMover>();
    _controls = _instance.GetComponentInChildren<CockpitControls>();
}

private void Despawn()
{
    if (_instance == null) return;
    Destroy(_instance);
    _instance = null;
    _seat = null;
    _mover = null;
    _controls = null;
    _seated = false;
}

public void Enter()
{
    if (_instance == null || rigRoot == null || _seat == null || _seated) return;

    _rigParentBefore = rigRoot.parent;
    _rigPosBefore = rigRoot.position;
    _rigRotBefore = rigRoot.rotation;
}

```

```

        rigRoot.SetParent(_seat, worldPositionStays: true);
        rigRoot.localPosition = Vector3.zero;
        rigRoot.localRotation = Quaternion.identity;

        _seated = true;
    }

    public void Exit()
    {
        if (!_seated || rigRoot == null) return;

        rigRoot.SetParent(_rigParentBefore, worldPositionStays: true);
        rigRoot.position = _rigPosBefore;
        rigRoot.rotation = _rigRotBefore;

        _seated = false;
    }

    private void Update()
    {
        if (!IsActive || !_seated || _mover == null || _controls == null) return;

        _mover.Drive(_controls.leftTrack, _controls.rightTrack);

        // Hier kommen als nächstes: Crosshair, select/destroy, rotation dials
    }
}

```

Schritt 6: In dein bestehendes Menü einhängen

- `TechniqueManager` bekommt eine dritte Methode `SetLocomotionCockpitTankMode()` (analog zu `Legacy/Vehicle`).^[844]
- Ein neuer Button „CockpitTank“ ruft diese Methode.

Stopp-Punkt zum Prüfen (bitte jetzt machen)

Wenn du Schritt 1–6 umgesetzt hast, solltest du:

- Tank spawnen können,
- einsteigen (Rig wird ans `SeatAnchor` geparentet),
- im Editor per WASD Tank bewegen (Differential Drive fühlt sich „tankig“ an).^[845]

Schreib mir danach:

1. Spawn + Enter klappt?
2. Bewegt er sich wie erwartet mit WASD?

Dann machen wir als nächstes **Interaction SDK Wiring** für die zwei Hebel (statt WASD) und danach Joystick/Crosshair, Buttons (Select/Destroy), und zuletzt die Stellräder (Rotation in Interaction).



Gehe bitte nochmal detaillierter durch die schritte zur einbindung der neuen locomotion technique in das bestehende system ein. ich fürchte ich habe etwas vergessen.

Damit die neue Cockpit-Tank Technique wirklich „wie die anderen“ in dein bestehendes System passt, musst du **drei Stellen** sauber verdrahten: (1) Technique-Komponente in der Scene, (2) TechniqueManager erweitert, (3) Menü/Buttons rufen die neuen Manager-Methoden auf. Das Repo ist genau so gedacht: Technique implementieren, auswählen, testen. [\[865\]](#)

1) Neue Technique in die Scene bringen

1. Wähle in der Hierarchy dein OVRCameraRig (das Objekt, auf dem auch Vehicle/Legacy hängen).
2. **Add Component** → CockpitTankLocomotionTechnique.
3. Inspector-Felder setzen:
 - tankCockpitPrefab → dein Tank_03_CockpitPrefab (Prefab Asset, nicht Scene-Instance).
 - rigRoot → OVRCameraRig (Root bewegen/parenten).
 - head → OVRCameraRig/TrackingSpace/CenterEyeAnchor.
 - spawnOffset → z.B. (0, 0, 2) (nur Startwert).
4. Stelle sicher: Die Komponente ist im Inspector **enabled egal** – der Manager ruft `SetActive(...)`, aber Unitys `enabled` muss nicht zwingend manuell gesetzt werden (wir schalten im `SetActive` bereits).

Wichtig: Es darf nur **eine** aktive Locomotion gleichzeitig den Rig bewegen (deshalb Manager-Logik). [\[865\]](#)

2) TechniqueManager um 3. Modus erweitern

Du hast aktuell Legacy und Vehicle. Jetzt kommt TankCockpit dazu.

Variante A (wenn du die Enum-Version nutzt – empfohlen)

In `TechniqueManager.cs`:

1. Enum erweitern:

```
public enum LocomotionMode { Legacy, Vehicle, CockpitTank }
```

2. Feld hinzufügen:

```
[SerializeField] private CockpitTankLocomotionTechnique cockpitTankLocomotion;
```

3. ApplyLocomotionMode erweitern (disable alle, dann enable eins):

```
public void ApplyLocomotionMode(LocomotionMode mode)
{
    if (legacyLocomotion != null) legacyLocomotion.SetActive(false);
    if (vehicleLocomotion != null) vehicleLocomotion.SetActive(false);
    if (cockpitTankLocomotion != null) cockpitTankLocomotion.SetActive(false);

    switch (mode)
    {
        case LocomotionMode.Legacy:
            if (legacyLocomotion != null) legacyLocomotion.SetActive(true);
            break;
        case LocomotionMode.Vehicle:
            if (vehicleLocomotion != null) vehicleLocomotion.SetActive(true);
            break;
        case LocomotionMode.CockpitTank:
            if (cockpitTankLocomotion != null) cockpitTankLocomotion.SetActive(true);
            break;
    }
}
```

4. UI-Wrapper hinzufügen:

```
public void SetLocomotionCockpitTankMode() => ApplyLocomotionMode(LocomotionMode.CockpitTankMode);
```

Scene-Referenz setzen

In der Scene: Systems auswählen → TechniqueManager:

- Ziehe vom OVRCameraRig die Komponente CockpitTankLocomotionTechnique in das neue Feld cockpitTankLocomotion.

3) Menü: Button hinzufügen und korrekt verdrahten

Du hast aktuell 2 Buttons (Legacy/Vehicle) über TechniqueMenuController.SelectLegacy/SelectVehicle.

3.1 TechniqueMenuController erweitern

In TechniqueMenuController.cs ergänze:

```
public void SelectCockpitTank()
{
    if (manager != null) manager.SetLocomotionCockpitTankMode();
    if (menuRoot != null) menuRoot.SetActive(false);
}
```

3.2 Button anlegen

1. Canvas → MenuRoot → Duplicate eines bestehenden Buttons → Name z.B. Btn_CockpitTank, Text „CockpitTank“.
2. OnClick:
 - Zielobjekt: dein GameObject mit `TechniqueMenuController` (Canvas oder MenuAnchor).
 - Methode: `TechniqueMenuController.SelectCockpitTank()`.

4) Minimaler Funktionstest (damit du weißt, dass die Einbindung passt)

1. Play → Menü öffnen.
2. Klick „CockpitTank“:
 - Tank spawnt vor dir,
 - du wirst in `SeatAnchor` geparentet,
 - WASD-Debug (oder später Hebel) bewegt den Tank.
3. Wechsel zurück auf „Legacy“:
 - Tank despawnt/Exit wird ausgeführt,
 - normale Locomotion wieder aktiv.

Häufigster „Vergessen“-Punkt (Checkliste)

- Prefab hat wirklich ein Child namens `SeatAnchor` (genau so geschrieben).
- `tankCockpitPrefab` ist gesetzt (nicht `None`).
- `TechniqueManager` hat das neue Feld gesetzt (sonst passiert beim Umschalten nichts).
- Button ruft **MenuController**-Methode auf (nicht direkt Manager, falls du die anderen so aufgebaut hast).

Wenn du mir kurz sagst, welche `TechniqueManager`-Variante du aktuell benutzt (mit Enum `ApplyLocomotionMode` oder noch die ältere ohne Enum), passe ich dir den exakten Code 1:1 passend zu deinem aktuellen Stand an.

✱✱

was habe ich vergessen?

Die Fehlermeldung sagt genau, was fehlt: In `TechniqueManager` gibt es **noch keine Methode** `SetLocomotionCockpitTankMode`, auf die dein `TechniqueMenuController` (oder der Button) zugreift. [\[886\]](#)

Kurz: Du hast im `MenuController` schon `SelectCockpitTank()` eingebaut, aber im `TechniqueManager` die passende öffentliche Methode noch nicht (oder sie heißt anders).

Checkliste – so behebst du es

1) TechniqueManager: Methode wirklich anlegen

Öffne TechniqueManager.cs und ergänze (oder prüfe):

```
public class TechniqueManager : MonoBehaviour
{
    public enum LocomotionMode
    {
        Legacy,
        Vehicle,
        CockpitTank
    }

    [Header("References")]
    [SerializeField] private LocomotionLegacyWrapper legacyLocomotion;
    [SerializeField] private VehicleLocomotionMVP vehicleLocomotion;
    [SerializeField] private CockpitTankLocomotionTechnique cockpitTankLocomotion;

    [Header("Defaults")]
    [SerializeField] private LocomotionMode startMode = LocomotionMode.Legacy;

    private void Start()
    {
        ApplyLocomotionMode(startMode);
    }

    public void ApplyLocomotionMode(LocomotionMode mode)
    {
        if (legacyLocomotion != null) legacyLocomotion.SetActive(false);
        if (vehicleLocomotion != null) vehicleLocomotion.SetActive(false);
        if (cockpitTankLocomotion != null) cockpitTankLocomotion.SetActive(false);

        switch (mode)
        {
            case LocomotionMode.Legacy:
                if (legacyLocomotion != null) legacyLocomotion.SetActive(true);
                break;
            case LocomotionMode.Vehicle:
                if (vehicleLocomotion != null) vehicleLocomotion.SetActive(true);
                break;
            case LocomotionMode.CockpitTank:
                if (cockpitTankLocomotion != null) cockpitTankLocomotion.SetActive(true);
                break;
        }
    }

    public void SetLocomotionLegacyMode() => ApplyLocomotionMode(LocomotionMode.Legacy);
    public void SetLocomotionVehicleMode() => ApplyLocomotionMode(LocomotionMode.Vehicle);
    public void SetLocomotionCockpitTankMode() => ApplyLocomotionMode(LocomotionMode.CockpitTank);
}
```

Wichtig:

- Die Signatur muss exakt `public void SetLocomotionCockpitTankMode()` heißen (Name, Groß/Kleinschreibung identisch zur Fehlermeldung).^[886]

2) Scene-Referenz für cockpitTankLocomotion setzen

Im Inspector von Systems/TechniqueManager:

- Neues Feld Cockpit Tank Locomotion sollte sichtbar sein.
- Ziehe vom OVRCameraRig die CockpitTankLocomotionTechnique-Komponente hinein.

3) TechniqueMenuController: Methode ruft Manager auf

In TechniqueMenuController.cs:

```
public void SelectCockpitTank()
{
    if (manager != null) manager.SetLocomotionCockpitTankMode();
    if (menuRoot != null) menuRoot.SetActive(false);
}
```

Und dein Button „CockpitTank“ ruft **diese** Methode auf (nicht direkt den Manager).

Wenn nach dem Einfügen/Anpassen von `SetLocomotionCockpitTankMode()` die Fehlermeldung immer noch auftaucht, ist sehr wahrscheinlich:

- die Datei nicht gespeichert, oder
- der Name in MenuController/Fehlerzeile minimal anders (z.B. Tippfehler).

Dann poste bitte kurz den aktuellen Inhalt von `TechniqueManager.cs`, dann passe ich ihn dir 1:1 auf deinen Stand an.



Ok, das ist alles fertig. jetzt können wir mit dem porten der controls beginnen

Alles klar—wir „portieren“ jetzt die Controls weg von Keyboard/Debug hin zu echten **Cockpit-Interaktionen** (2 Hebel, Joystick, Buttons, Stellräder) und verdrahten sie so, dass CockpitControls weiterhin nur **normierte Werte** liefert und CockpitTankLocomotionTechnique damit arbeitet. Für Meta Interaction SDK ist der Standardweg: Grabbable + OneGrab... Transformer (Hebel/Knobs) und PokeInteractable + Event Wrapper (Buttons).^[908] ^[909]

Schritt 0: Zielzustand (damit du weißt, worauf wir hinarbeiten)

- CockpitControls liest Werte **aus Transformständen** der Interactables (nicht aus Input).
- Jedes Bedienelement bewegt sich durch Interaction SDK, unser Code liest nur den aktuellen Winkel/Offset und mappt ihn auf [-1..1]/Events.

Schritt 1: Lever_L / Lever_R als grab-rotierte Hebel

Für **jeden** Hebel (Lever_L, Lever_R) im Prefab:

1. Stelle sicher, dass der Hebel ein eigenes GameObject ist (Mesh + ggf. Child).
2. Füge hinzu:
 - Grabbable (oder das passende ISDK Grabbable aus den Meta Building Blocks)
 - One Grab Transformer + **OneGrabRotateTransformer** (oder äquivalentes Rotate-Constraint Setup)^[908]
3. Constraint:
 - Rotationsachse: die lokale Achse, um die der Hebel kippt (typisch X oder Z).
 - Min/Max Winkel: z.B. -35° .. +35° (damit er „Hebel“ bleibt).^[908]
4. Pivot:
 - Lege einen leeren Child LeverPivot an (am Gelenkpunkt), und setze ihn als Pivot im RotateTransformer, damit er um den richtigen Punkt rotiert.^[908]

Stopp-Test: Im Play Mode muss jeder Hebel sauber greifbar sein und nur in einem Winkelbereich kippen.

Schritt 2: CockpitControls liest Hebelwerte (Track links/rechts)

In CockpitControls entfernst du den Keyboard-Teil für left/rightTrack und ersetzt ihn durch Transform-Mapping.

1. Füge Referenzen hinzu:

```
[SerializeField] private Transform leverL;  
[SerializeField] private Transform leverR;  
[SerializeField] private Transform leverPivotL;  
[SerializeField] private Transform leverPivotR;  
[SerializeField] private float leverMinDeg = -35f;  
[SerializeField] private float leverMaxDeg = 35f;
```

2. In Update():

- lies den lokalen Winkel um die Hebelachse (z.B. X):

```
leftTrack = MapAngleToMinus1Plus1(leverPivotL.localEulerAngles.x, leverMinDeg, leverMaxDeg);  
rightTrack = MapAngleToMinus1Plus1(leverPivotR.localEulerAngles.x, leverMinDeg, leverMaxDeg);
```

3. Helper (unten im Script):

```
private float MapAngleToMinus1Plus1(float eulerDeg, float minDeg, float maxDeg)  
{  
    // Euler wrap (0..360) -> (-180..180)  
    float a = eulerDeg;  
    if (a > 180f) a -= 360f;
```



```
float t = Mathf.InverseLerp(minDeg, maxDeg, a); // 0..1
return Mathf.Lerp(-1f, 1f, t);
}
```

Wichtig: Du musst die richtige Achse wählen (x/y/z). Wenn der Hebel „falsch“ reagiert, ist es fast immer die Achse.

Schritt 3: Buttons (Select/Destroy) via Poke + UnityEventWrapper

Für Btn_Select und Btn_Destroy:

1. Mache sie zu Poke Buttons (Building Block oder Komponenten):

- `PokeInteractable`
- `Interactable Unity Event Wrapper` (damit du Events im Inspector oder per Script bekommst) [\[909\]](#)

2. Im Wrapper:

- Event „Selected“ → ruft Methoden in `CockpitControls` auf (oder setze bools).

Minimal (Script-Seite): in `CockpitControls`:

```
public void OnSelectButton() { selectPressed = true; }
public void OnDestroyButton() { destroyPressed = true; }
```

Und am Ende von `Update()` setzt du sie wieder zurück:

```
selectPressed = false;
destroyPressed = false;
```

Damit sind es „one-frame“ Press Events.

Schritt 4: Stellräder (Dial_RotA / Dial_RotB) als Knobs

Für jedes Dial:

1. `Grabbable + OneGrabRotateTransformer` (Rotation um z.B. lokale Z Achse) [\[908\]](#)

2. Clamp:

- entweder -90..+90 (einfach) oder -180..+180.

3. In `CockpitControls`:

- ähnlich wie Hebel, nur anderer Winkelbereich und evtl. andere Achse → map auf `rotA/rotB` in [-1..1].

Schritt 5: Joystick → Crosshair in 4 Richtungen

Ich empfehle für den Anfang eine **2D-Mapping** ohne echte Federmechanik:

- Joystick als Grabbable, aber du liest nicht „Rotation“, sondern die Position eines kleinen JoystickTip (Child) relativ zu einem JoystickCenter (begrenzter Radius).
- Dann:
 - `joyX = clamp(localOffset.x / radius, -1..1)`
 - `joyY = clamp(localOffset.z / radius, -1..1)`
- Für „4 Richtungen“: optional snappen:
 - wenn $|x| > |y| \rightarrow (\text{sign}(x), 0)$, sonst $(0, \text{sign}(y))$.

(Die echte Joystick-Mechanik mit 2 Achsen kannst du später nachziehen.)

Schritt 6: Verdrahtung in CockpitTankLocomotionTechnique

Da bleibt es einfach:

- `mover.Drive(controls.leftTrack, controls.rightTrack);`
- `controls.selectPressed` triggert Interaction/Selection
- `controls.destroyPressed` triggert Coin-destroy (später)
- `rotA/rotB` nutzt du für Rotation eines selected Objects
- `joyX/joyY` steuert Crosshair

Damit wir sauber starten: Welche Interaction-SDK Komponenten hast du bei den Hebeln schon drauf?

Bitte schick einen Screenshot vom Inspector eines Hebels (z.B. `Lever_L`) mit den Interaction-Komponenten offen. Dann sage ich dir exakt, welche Felder (Axis, Pivot, Min/Max Angle) du setzen musst und auf welche Transform du im CockpitControls lesen solltest.

✱

können wir die controls erstmal auf die controller mappen und danach prefabs raussuchen die die control logik complementieren?

controller forward backward → lever fwd backward, left trigger → select, right trigger, destroy, left joystick → selection rotations (x and y axis), right joystick → joystick.

Außerdem könnten wir das crosshair vllt mit einem "laserentfernungsmesser" ersetzen, der einen roten beam entlang des laufes anzeigt?

Ja—das ist sogar ein sehr guter Weg: erst **alles auf Controller-Inputs mappen** (damit Logik/Technique stabil wird), danach suchst du passende Cockpit-Prefabs und ersetzt nur die Input-Quelle. Für OVRInput kannst du die Thumbsticks als `Axis2D.PrimaryThumbstick` (links) und `Axis2D.SecondaryThumbstick` (rechts) lesen sowie Triggers als `Axis1D.PrimaryIndexTrigger/SecondaryIndexTrigger`.[\[945\]](#) [\[946\]](#)

Schritt 1: CockpitControls auf OVRInput umstellen

Öffne `CockpitControls.cs` und ersetze den bisherigen `Update()`-Block durch Controller-Mapping:

```
using UnityEngine;

public class CockpitControls : MonoBehaviour
{
    [Header("Outputs (-1..1)")]
    [Range(-1, 1)] public float leftTrack;
    [Range(-1, 1)] public float rightTrack;
    [Range(-1, 1)] public float joyX;
    [Range(-1, 1)] public float joyY;
    [Range(-1, 1)] public float rotA;
    [Range(-1, 1)] public float rotB;
    public bool selectPressed;
    public bool destroyPressed;

    [Header("Tuning")]
    [SerializeField] private float leverDeadzone = 0.12f;
    [SerializeField] private bool invertLever = false;

    private void Update()
    {
        // --- Lever forward/backward via LEFT stick Y (one lever drives both for now) ---
        // later: we can split into true dual-lever mapping if you want
        Vector2 leftStick = OVRInput.Get(OVRInput.Axis2D.PrimaryThumbstick); // [-1..1]
        float lever = ApplyDeadzone(leftStick.y, leverDeadzone);
        if (invertLever) lever = -lever;

        leftTrack = lever;
        rightTrack = lever;

        // --- Select/Destroy via triggers ---
        selectPressed = OVRInput.GetDown(OVRInput.Button.PrimaryIndexTrigger); // left
        destroyPressed = OVRInput.GetDown(OVRInput.Button.SecondaryIndexTrigger); // right

        // --- Selection rotation via LEFT stick X/Y ---
        rotA = leftStick.x; // yaw
        rotB = leftStick.y; // pitch (oder umgekehrt, wie du willst)

        // --- Joystick for crosshair via RIGHT stick ---
        Vector2 rightStick = OVRInput.Get(OVRInput.Axis2D.SecondaryThumbstick); // [-1..1]
        joyX = rightStick.x;
        joyY = rightStick.y;
    }

    private float ApplyDeadzone(float v, float dz)
    {
        if (v < -dz) v = -dz;
        if (v > dz) v = dz;
        return v;
    }
}
```

```

    {
        if (Mathf.Abs(v) < dz) return 0f;
        // optional: rescale outside deadzone
        float sign = Mathf.Sign(v);
        float t = (Mathf.Abs(v) - dz) / (1f - dz);
        return sign * Mathf.Clamp01(t);
    }
}

```

Hinweis: Du wolltest „Controller forward/backward → lever fwd/back“. Ich habe das als **linker Stick Y** umgesetzt, weil du ansonsten keine echte Controller-Achse „forward/back“ hast. Wenn du stattdessen „beide Controller hoch/runter wie Hebel“ meinst, sag's kurz, dann mappen wir es wie beim Vehicle (Höhendifferenz oder je Hand absolute Y-Bewegung).

Schritt 2: TankDifferentialMover bleibt gleich

CockpitTankLocomotionTechnique.Update() bleibt:

```

_mover.Drive(_controls.leftTrack, _controls.rightTrack);

```

Wenn du später echte Dual-Lever willst (links/rechts unabhängig), ändern wir CockpitControls so, dass leftTrack und rightTrack aus zwei separaten Quellen kommen.

Schritt 3: Crosshair durch Laser-Entfernungsmesser ersetzen

Ja, und das ist sogar „cockpitiger“: ein roter Beam entlang des Laufs.

Setup: LineRenderer am Barrel

1. Im Tank-Prefab: wähle den Lauf-Transform (z.B. Tank_03_MainGunPart/Main_Gun aus deiner Hierarchie).
2. Add Component → LineRenderer.
3. Material: ein unlit rotes Material (oder Standard, erstmal ok).
4. Position Count = 2, Start Width/End Width klein (z.B. 0.01).

Script: LaserRangefinder

Lege LaserRangefinder.cs an und hänge es an den Lauf:

```

using UnityEngine;

public class LaserRangefinder : MonoBehaviour
{
    [SerializeField] private LineRenderer line;
    [SerializeField] private float maxDistance = 50f;
    [SerializeField] private LayerMask hitMask = ~0;

    private void Reset()
    {

```

```

        line = GetComponent<LineRenderer>();
    }

    private void Update()
    {
        if (line == null) return;

        Vector3 origin = transform.position;
        Vector3 dir = transform.forward;

        Vector3 end = origin + dir * maxDistance;
        if (Physics.Raycast(origin, dir, out RaycastHit hit, maxDistance, hitMask))
            end = hit.point;

        line.positionCount = 2;
        line.SetPosition(0, origin);
        line.SetPosition(1, end);
    }
}

```

Das nutzt einen Raycast für die Entfernung und zeichnet die Linie entsprechend. (LineRenderer ist genau dafür gedacht.)^[947]

Was du als Nächstes entscheiden musst (kurz)

1. Soll der Tank mit Controller-Mapping schon „richtig tankig“ sein (linker Stick Y = beide Hebel ist eher „Traktor“), oder willst du direkt **echte Dual-Lever**?
2. Soll „Select/Destroy“ beim Trigger **Press** passieren (GetDown, wie oben), oder solange gedrückt (Get)?

✱✱

Ich hätte gerne die relative position der joysticks zur quest als lever und den joystick als rotator für die späterre interaction. anstelle der höhe der joysticks wie beim vehiucle halt die entfernung zur quest (näher → nach hinten, weiter weg → nach vorne)

Ja, das geht gut—und du brauchst dafür nicht `OVRInput.GetLocalControllerPosition`, weil du in deiner Scene bereits saubere Anchors (`LeftControllerAnchor/RightControllerAnchor + CenterEyeAnchor`) hast. Du misst einfach die Controller-Position **relativ zur Head/Quest** im lokalen Head-Raum via `InverseTransformPoint` (World → lokal) und nimmst dann die **Z-Komponente** als „näher/weiter“. `InverseTransformPoint` ist genau für so „Position relativ zu einem Transform“ gedacht.^{[980] [981]}

Zielmapping (wie du es willst)

- Lever (Track links/rechts): **Controller-Abstand zur Quest** (näher = rückwärts, weiter weg = vorwärts).
- Joystick (für spätere Interaction-Rotation): **Controller-Rotation** (z.B. Pitch/Yaw) oder alternativ der rechte Stick; du wolltest "joystick als rotator", also nehmen wir Controller-Rotation.
- Trigger: Select/Destroy bleibt wie gehabt.

Schritt 1: CockpitControls erweitern (Head-relative Distanz)

In CockpitControls.cs:

1. Füge Felder hinzu:

```
[SerializeField] private Transform head;           // CenterEyeAnchor
[SerializeField] private Transform leftController;
[SerializeField] private Transform rightController;

[Header("Lever via distance (meters)")]
[SerializeField] private float neutralZ = 0.35f;    // "neutral" Entfernung vor dem Head
[SerializeField] private float rangeZ = 0.15f;      // +-15cm = voller Ausschlag
[SerializeField] private float leverDeadzoneZ = 0.02f; // 2cm deadzone
[SerializeField] private bool invertLeverZ = false;
```

2. Ersetze die Hebelberechnung in Update() durch:

```
// Output defaults
selectPressed = OVRInput.GetDown(OVRInput.Button.PrimaryIndexTrigger);
destroyPressed = OVRInput.GetDown(OVRInput.Button.SecondaryIndexTrigger);

// lever values from controller distance to head (local Z)
leftTrack = ComputeLeverFromController(leftController);
rightTrack = ComputeLeverFromController(rightController);

// joystick as rotator (later interaction rotation) from right controller rotation
Vector3 rLocalEuler = head.InverseTransformRotation(rightController.rotation).eulerAngles;
rotA = NormalizeAngleToMinus1Plus1(rLocalEuler.y, 45f); // yaw range +-45°
rotB = NormalizeAngleToMinus1Plus1(rLocalEuler.x, 45f); // pitch range +-45°
```

Dazu brauchst du 2 Helper:

```
private float ComputeLeverFromController(Transform controller)
{
    if (head == null || controller == null) return 0f;

    Vector3 local = head.InverseTransformPoint(controller.position); // [web:166][web:169]
    float z = local.z; // vor/zurück relativ zum Head

    // Map: näher -> rückwärts, weiter -> vorwärts
    float raw = (z - neutralZ) / Mathf.Max(0.0001f, rangeZ); // -inf..inf
```

```

raw = Mathf.Clamp(raw, -1f, 1f);

// deadzone
if (Mathf.Abs(raw) < (leverDeadzoneZ / Mathf.Max(0.0001f, rangeZ)))
    raw = 0f;

if (invertLeverZ) raw = -raw;
return raw;
}

private float NormalizeAngleToMinus1Plus1(float eulerDeg, float maxAbsDeg)
{
    // Euler wrap: 0..360 -> -180..180
    float a = eulerDeg;
    if (a > 180f) a -= 360f;
    return Mathf.Clamp(a / Mathf.Max(1f, maxAbsDeg), -1f, 1f);
}

```

Wichtig: `head.InverseTransformRotation` gibt es nicht als Unity-API. Du machst Rotation relativ zum Head so:

```

Quaternion localRot = Quaternion.Inverse(head.rotation) * rightController.rotation;
Vector3 rLocalEuler = localRot.eulerAngles;

```

(Und dann wie oben normalisieren.)

Schritt 2: Inspector verdrahten

Auf dem GameObject mit `CockpitControls`:

- `head` = `OVRCameraRig/TrackingSpace/CenterEyeAnchor`
- `leftController` = `.../LeftControllerAnchor`
- `rightController` = `.../RightControllerAnchor`

Schritt 3: Kalibrierung (NeutralZ)

Damit "neutral" wirklich angenehm ist:

1. Setze `neutralZ` so, dass deine Hände in entspannter Position **0** ergeben.
2. `rangeZ`: wie weit du die Hände bewegen willst, bis Vollgas/Vollreverse (z.B. 0.12–0.20 m).

Laser-Entfernungsmesser statt Crosshair

Ja, das passt perfekt zum Tank. Wir verwenden den Beam entlang des Laufes (LineRenderer + Raycast) wie zuvor beschrieben; dadurch brauchst du das Crosshair kaum noch. (Optional kannst du am Hitpoint ein kleines rotes Dot anzeigen.)

Eine Rückfrage, damit das Mapping „intuitiv“ wird: Willst du wirklich **beide** Hebel aus der Entfernung der jeweiligen Hand messen (linke Hand steuert linke Kette, rechte Hand rechte Kette), oder sollen beide Hände zusammen nur einen gemeinsamen "Gashebel" bilden?

using UnityEngine;

```

public class CockpitControls : MonoBehaviour
{
    [Header("Outputs (-1..1)")]
    [Range(-1,1)] public float leftTrack;
    [Range(-1,1)] public float rightTrack;
    [Range(-1,1)] public float joyX;
    [Range(-1,1)] public float joyY;
    [Range(-1,1)] public float rotA;
    [Range(-1,1)] public float rotB;
    public bool selectPressed;
    public bool destroyPressed;
    [Header("Tuning")]
    [SerializeField] private float leverDeadzone = 0.12f;
    [SerializeField] private bool invertLever = false;

    private void Update()
    {
        // --- Lever forward/backward via LEFT stick Y (one lever drives both for now) ---
        // later: we can split into true dual-lever mapping if you want
        Vector2 leftStick = OVRInput.Get(OVRInput.Axis2D.PrimaryThumbstick);    // [-1..1]
[web:20][web:23]
        float lever = ApplyDeadzone(leftStick.y, leverDeadzone);
        if (invertLever) lever = -lever;

        leftTrack = lever;
        rightTrack = lever;

        // --- Select/Destroy via triggers ---
        selectPressed = OVRInput.GetDown(OVRInput.Button.PrimaryIndexTrigger);    // left trigger
        press [web:23]
        destroyPressed = OVRInput.GetDown(OVRInput.Button.SecondaryIndexTrigger); // right
        trigger press [web:23]

        // --- Selection rotation via LEFT stick X/Y ---
        rotA = leftStick.x;    // yaw
        rotB = leftStick.y;    // pitch (oder umgekehrt, wie du willst)

        // --- Joystick for crosshair via RIGHT stick ---
        Vector2 rightStick = OVRInput.Get(OVRInput.Axis2D.SecondaryThumbstick); // [-1..1]
[web:20][web:23]
        joyX = rightStick.x;
        joyY = rightStick.y;
    }
}

```



```

private float ApplyDeadzone(float v, float dz)
{
    if (Mathf.Abs(v) < dz) return 0f;
    // optional: rescale outside deadzone
    float sign = Mathf.Sign(v);
    float t = (Mathf.Abs(v) - dz) / (1f - dz);
    return sign * Mathf.Clamp01(t);
}
}

```

Kannst du die datei bitte umschreiben? und mach den rechten joystick bitte für das drehen des turmes und die höhenrichtung der hauptwaffe zuständig.

Ja—ich schreibe dir die Datei so um, dass:

- **leftTrack/rightTrack** aus der *Entfernung (Head-relatives local Z)* der beiden Controller zur Quest kommen (näher = rückwärts, weiter = vorwärts). Dafür nutzen wir `head.InverseTransformPoint(controller.position)` – genau dafür ist es da. ^[1017]
- **Select/Destroy** bleiben auf Trigger-Press. ^[1018]
- **rechter Joystick** steuert **Turm drehen** (yaw) und **Hauptwaffe hoch/runter** (pitch), also `joyX/joyY`. Thumbsticks bekommst du über `Axis2D.SecondaryThumbstick`. ^[1019] ^[1018]
- `rotA/rotB` lassen wir erstmal **frei** (0), weil du gesagt hast: der Joystick soll später der Rotator für Interaction sein (das können wir danach umlegen).

Ersetze deine `CockpitControls.cs` komplett durch:

```

using UnityEngine;

public class CockpitControls : MonoBehaviour
{
    [Header("Refs")]
    [SerializeField] private Transform head;           // CenterEyeAnchor
    [SerializeField] private Transform leftController; // LeftControllerAnchor
    [SerializeField] private Transform rightController; // RightControllerAnchor

    [Header("Outputs (-1..1)")]
    [Range(-1, 1)] public float leftTrack;
    [Range(-1, 1)] public float rightTrack;

    // We use joyX/joyY for turret yaw + gun pitch
    [Range(-1, 1)] public float joyX;
    [Range(-1, 1)] public float joyY;

    // Reserved for later "interaction rotation"
    [Range(-1, 1)] public float rotA;
    [Range(-1, 1)] public float rotB;

    public bool selectPressed;
    public bool destroyPressed;

    [Header("Lever by distance-to-head (meters)")]

```

```

[SerializeField] private float neutralZ = 0.35f;        // neutral: controller this far
[SerializeField] private float rangeZ = 0.15f;        // +/- range around neutral =>
[SerializeField] private float deadzoneMeters = 0.02f; // 2cm deadzone around neutral
[SerializeField] private bool invertLever = false;

[Header("Turret stick tuning")]
[SerializeField] private float stickDeadzone = 0.15f;
[SerializeField] private bool invertTurretYaw = false;
[SerializeField] private bool invertGunPitch = true;

private void Update()
{
    // --- Track levers: distance (near -> back, far -> forward) ---
    leftTrack = LeverFromControllerZ(leftController);
    rightTrack = LeverFromControllerZ(rightController);

    // --- Select/Destroy via trigger press ---
    selectPressed = OVRInput.GetDown(OVRInput.Button.PrimaryIndexTrigger); // left
    destroyPressed = OVRInput.GetDown(OVRInput.Button.SecondaryIndexTrigger); // right

    // --- Right stick: turret yaw + gun pitch ---
    Vector2 stick = OVRInput.Get(OVRInput.Axis2D.SecondaryThumbstick); // right
    stick = ApplyDeadzone2D(stick, stickDeadzone);

    joyX = invertTurretYaw ? -stick.x : stick.x;        // turret yaw
    joyY = invertGunPitch ? -stick.y : stick.y;        // gun pitch

    // Reserved for later
    rotA = 0f;
    rotB = 0f;
}

private float LeverFromControllerZ(Transform controller)
{
    if (head == null || controller == null) return 0f;

    // Head-local coordinates: z is forward distance from headset [web:166]
    Vector3 local = head.InverseTransformPoint(controller.position);

    // Map local.z around neutralZ to [-1..1]
    float dz = local.z - neutralZ;

    // deadzone in meters
    if (Mathf.Abs(dz) < deadzoneMeters) return 0f;

    float v = dz / Mathf.Max(0.0001f, rangeZ);
    v = Mathf.Clamp(v, -1f, 1f);

    // closer -> negative (reverse), farther -> positive (forward) by construction
    if (invertLever) v = -v;
    return v;
}

private Vector2 ApplyDeadzone2D(Vector2 v, float dz)
{
    float m = v.magnitude;

```

```

        if (m < dz) return Vector2.zero;

        float t = (m - dz) / Mathf.Max(0.0001f, (1f - dz));
        return v.normalized * Mathf.Clamp01(t);
    }
}

```

Was du im Inspector setzen musst

Auf deinem `CockpitControls` GameObject:

- `head` → `OVRCameraRig/TrackingSpace/CenterEyeAnchor`
- `leftController` → `.../LeftControllerAnchor`
- `rightController` → `.../RightControllerAnchor`

Dann im Play Mode:

- Hände näher an die Quest ziehen → beide Tracks negativ (rückwärts)
- Hände weiter nach vorne → positiv (vorwärts)

Nächster Schritt (Turret wirklich drehen)

Damit `joyX/joyY` den Tank wirklich bewegt, ergänzen wir als nächstes im Tank-Prefab ein Script `TankTurretController`, das `joyX` auf `Tank_03_UpperBody` (Yaw) und `joyY` auf `Tank_03_MainGunPart/Main_Gun` (Pitch) anwendet. [\[1020\]](#)

Sag mir kurz, welche Transforms du dafür verwenden willst:

- Turret yaw: `Tank_03_UpperBody` oder `Tank_03_UpperBody1?` [\[1020\]](#)
- Gun pitch: `Main_Gun` oder `Tank_03_MainGunPart?` [\[1020\]](#)

✱✱

Tank upper body und main gun part

Passt — dann bauen wir jetzt einen kleinen Turret/Gun-Controller, der `CockpitControls.joyX/joyY` ausliest und auf **`Tank_03_UpperBody`** (Yaw) und **`Tank_03_MainGunPart`** (Pitch) anwendet. (Du hast damit zwei Achsen auf zwei getrennten Transforms, das ist genau der saubere Ansatz.) [\[1050\]](#)

Schritt 1: Script anlegen

Erstelle `TankTurretController.cs` unter
Assets/Scripts/Techniques/Implementations/CockpitTank/:

```

using UnityEngine;

public class TankTurretController : MonoBehaviour
{
    [Header("Refs")]

```

```

[SerializeField] private CockpitControls controls;
[SerializeField] private Transform turretYaw;    // Tank_03_UpperBody
[SerializeField] private Transform gunPitch;    // Tank_03_MainGunPart

[Header("Rates")]
[SerializeField] private float yawSpeedDeg = 70f;
[SerializeField] private float pitchSpeedDeg = 40f;

[Header("Pitch limits")]
[SerializeField] private float minPitchDeg = -10f;
[SerializeField] private float maxPitchDeg = 25f;

private float _pitch; // tracked pitch angle in degrees

private void Awake()
{
    // Initialize pitch from current local angle (wrap to -180..180)
    if (gunPitch != null)
        _pitch = Wrap180(gunPitch.localEulerAngles.x);
}

private void Update()
{
    if (controls == null) return;

    // Yaw: rotate turret around local Y
    if (turretYaw != null)
    {
        float yawInput = controls.joyX; // -1..1
        turretYaw.Rotate(0f, yawInput * yawSpeedDeg * Time.deltaTime, 0f, Space.Self)
    }

    // Pitch: clamp around local X
    if (gunPitch != null)
    {
        float pitchInput = controls.joyY; // -1..1
        _pitch += pitchInput * pitchSpeedDeg * Time.deltaTime;
        _pitch = Mathf.Clamp(_pitch, minPitchDeg, maxPitchDeg);

        Vector3 e = gunPitch.localEulerAngles;
        gunPitch.localEulerAngles = new Vector3(Unwrap0_360(_pitch), e.y, e.z);
    }
}

private float Wrap180(float deg)
{
    deg %= 360f;
    if (deg > 180f) deg -= 360f;
    return deg;
}

private float Unwrap0_360(float deg)
{
    // Convert -180..180 to 0..360 style euler
    if (deg < 0f) deg += 360f;
    return deg;
}

```

```
}  
}
```

Schritt 2: Im Prefab verdrahten

Im Tank_03_CockpitPrefab (Prefab Mode):

1. Root oder CockpitRoot auswählen → **Add Component** → TankTurretController.
2. Felder setzen:
 - controls → dein CockpitControls (Component am CockpitRoot).
 - turretYaw → Transform Tank_03_UpperBody. [\[1050\]](#)
 - gunPitch → Transform Tank_03_MainGunPart. [\[1050\]](#)

Schritt 3: Schnelltest

- Play → CockpitTank Mode wählen → rechter Stick links/rechts: Turm dreht.
- Rechter Stick hoch/runter: Hauptwaffe pitcht innerhalb der Grenzen.

Wenn Pitch/Yaw „falsch herum“ ist, sag kurz Bescheid — dann invertieren wir entweder direkt im CockpitControls (invertGunPitch / invertTurretYaw) oder multiplizieren im TurretController mit -1.

Als nächstes können wir den „Laserentfernungsmesser“ an Tank_03_MainGunPart oder an ein Child Muzzle hängen, damit der rote Beam exakt aus der Mündung kommt.

✱

ok, turret controls klappen super, allerdings ist der LEM nicht sichtbar und die wanne bewegt sich noch nicht mit controllerbewegung

Zwei getrennte Themen:

1. **LEM (Laser) nicht sichtbar** → fast immer LineRenderer-Setup (Material/Width/useWorldSpace/Layer) oder er wird auf (0,0,0) gezeichnet. [\[1086\]](#) [\[1087\]](#)
2. **Wanne bewegt sich nicht** → dein TankDifferentialMover bekommt vermutlich noch leftTrack/rightTrack = 0, weil CockpitControls nicht korrekt auf head/leftController/rightController verdrahtet ist (oder du hast noch die alte Thumbstick-Version aktiv). Die Anchors liefern laufend Controller-Posen, wenn sie korrekt referenziert sind. [\[1088\]](#)

A) LEM sichtbar machen (robuster Fix)

1. Hänge LaserRangefinder **direkt** an Tank_03_MainGunPart oder (besser) an ein Child Muzzle am Laufende. [\[1089\]](#)
2. Im LineRenderer:

- `Position Count = 2`
- `Use World Space = true` (damit du Weltkoordinaten setzen kannst). [\[1087\]](#)
- `Start Width/End Width` testweise **0.01–0.02**
- `Material` setzen (ohne `Material` bleibt er oft unsichtbar): z.B. `Sprites/Default` und Farbe rot. [\[1086\]](#)

Falls du dein Script nutzt, setze in `Start()` einmal:

```
line.useWorldSpace = true;
line.material = new Material(Shader.Find("Sprites/Default"));
line.material.color = Color.red;
line.startWidth = 0.015f;
line.endWidth = 0.015f;
```

Das entspricht einem bekannten Quick-Debug für „LineRenderer nicht sichtbar“. [\[1087\]](#) [\[1086\]](#)

Zusatz-Check: Stelle sicher, dass du nicht versehentlich den LineRenderer unter einem UI-Canvas renderst (Screen-Space UI kann „drüber“ liegen). [\[1086\]](#)

B) Wanne bewegt sich nicht (TankDrive)

Damit sich die Wanne bewegt, müssen 3 Dinge stimmen:

1) CockpitControls muss die neue „distance-to-head“ Version sein

Du wolltest „näher/weiter“ pro Controller → `leftTrack/rightTrack`. Wenn du noch die alte Version drin hast (Stick-Y steuert beide), kommt eventuell immer 0 raus.

2) Inspector-Referenzen gesetzt

Auf `CockpitControls`:

- `head = CenterEyeAnchor`
- `leftController = LeftControllerAnchor`
- `rightController = RightControllerAnchor`

Wenn eins `None` ist, liefert die Funktion 0 → Tank steht. [\[1088\]](#)

3) CockpitTankLocomotionTechnique ruft mover wirklich auf

In `CockpitTankLocomotionTechnique.Update()` muss drin sein:

```
_mover.Drive(_controls.leftTrack, _controls.rightTrack);
```

Schnellster Debug (bitte einmal machen)

1. Füge in `CockpitTankLocomotionTechnique.Update()` temporär ein:

```
Debug.Log($"L={_controls.leftTrack:F2} R={_controls.rightTrack:F2}");
```

2. Starte Play, bewege Controller näher/weiter.
 - Wenn Log **immer 0**: Referenzen in `CockpitControls` fehlen oder `neutralZ/rangeZ` sind so gesetzt, dass du nie aus der Deadzone kommst.
 - Wenn Log Werte zeigt, aber Tank steht: `TankDifferentialMover` hängt nicht am richtigen Root oder wird nicht gefunden (`GetComponentInChildren` liefert null).

Zwei konkrete Rückfragen (damit ich's gezielt fixen kann)

1. Ist dein aktuelles `CockpitControls.cs` bereits die Version mit `head.InverseTransformPoint(...)` (distance-to-head), oder noch die Thumbstick-Version?
2. Kannst du einen Screenshot vom Inspector von `LaserRangefinder` + `LineRenderer` (am `MainGunPart`) posten?



ok, den rangefinder machen wir später fertig, ich würde gerne die steuerunglogik nochmal überarbeiten. aktuell dreht der panzer in die richtung des am meißten vorgelegten hebels, es müsste aber der kettenlogik folgend die gegenrichtung sein. können wir das noch umändern?

Ja—das ist nur ein **Vorzeichen-/Konventionsproblem** in deiner Differential-Drive-Formel (oder ein Side-Invert), und das ist bei Tank-Steuerungen sehr typisch. Bei Differential Drive gilt: die Drehrate ist proportional zu $\omega \propto v_R - v_L$; je nachdem, wie du „positive yaw“ definierst und welche Seite du als „rechts“/„links“ interpretierst, muss man genau dieses Vorzeichen einmal drehen. [\[1120\]](#) [\[1121\]](#)

Fix 1 (einzeilig, am besten): Drehrichtung invertieren

In deinem `TankDifferentialMover.Drive(leftTrack, rightTrack)` hast du vermutlich sowas:

```
float wCmd = (rightTrack - leftTrack) * 0.5f;
```

Wenn er aktuell „in die falsche Richtung“ dreht, ändere es zu:

```
float wCmd = (leftTrack - rightTrack) * 0.5f;
```

Das spiegelt die Drehlogik und entspricht deinem Wunsch „wenn links mehr vorgelegt ist, soll er in die Gegenrichtung drehen“. ^[1121]

Fix 2 (alternativ): Eine Seite invertieren (realistische Track-Konvention)

Manchmal ist nicht die Formel falsch, sondern eine Seite hat "forward" invertiert (mechanisch/Koordinaten). Dann lässt du die Formel wie sie ist und invertierst genau einen Track:

```
rightTrack = -rightTrack;
```

Das ist auch in vielen Differential-Drive-Konventionen üblich, weil eine Seite „gegenläufig“ ist. ^[1122]

Welche Variante du nehmen solltest

- Wenn **alle Kombinationen** "falsch herum" sind (z.B. links vor → dreht links statt rechts und umgekehrt), nimm **Fix 1** (wCmd tauschen). ^[1121]
- Wenn nur **eine Seite** sich „verkehrt“ anfühlt (z.B. rechter Hebel vorwärts macht rechtes Track rückwärts), nimm **Fix 2** (eine Seite invertieren). ^[1122]

Mini-Test (30 Sekunden)

Nach dem Fix:

- Beide Hebel gleich weit vor → geradeaus.
- Linker Hebel vor, rechter neutral → Tank dreht **nach rechts**.
- Rechter Hebel vor, linker neutral → Tank dreht **nach links**.

Wenn du mir kurz sagst, welches dieser drei Tests gerade falsch läuft (nur 1 oder alle), sage ich dir eindeutig, ob Fix 1 oder Fix 2 die richtige Wahl ist.



1. ProjectName.txt
2. README.md
3. VR-locomotion-parkour-main.slnx
4. EditorUserSettings.asset
5. default-6000.dwt
6. XRPackageSettings.asset
7. VFXManager.asset
8. XRSettings.asset
9. VersionControlSettings.asset
10. README.md

11. VR-locomotion-parkour-main.slnx
12. ProjectName.txt
13. EditorUserSettings.asset
14. default-6000.dwlt
15. XRPackageSettings.asset
16. VFXManager.asset
17. XRSettings.asset
18. VersionControlSettings.asset
19. grafik.jpg
20. VersionControlSettings.asset
21. VR-locomotion-parkour-main.slnx
22. ProjectName.txt
23. EditorUserSettings.asset
24. default-6000.dwlt
25. README.md
26. XRPackageSettings.asset
27. VFXManager.asset
28. XRSettings.asset
29. grafik.jpg
30. README.md
31. grafik.jpg
32. XRSettings.asset
33. VersionControlSettings.asset
34. VR-locomotion-parkour-main.slnx
35. ProjectName.txt
36. EditorUserSettings.asset
37. default-6000.dwlt
38. XRPackageSettings.asset
39. VFXManager.asset
40. README.md
41. grafik.jpg
42. VersionControlSettings.asset
43. grafik.jpg
44. VR-locomotion-parkour-main.slnx
45. ProjectName.txt
46. EditorUserSettings.asset
47. default-6000.dwlt
48. XRPackageSettings.asset

49. VFXManager.asset
50. XRSettings.asset
51. README.md
52. grafik.jpg
53. grafik.jpg
54. XRSettings.asset
55. VersionControlSettings.asset
56. VR-locomotion-parkour-main.slnx
57. ProjectName.txt
58. EditorUserSettings.asset
59. default-6000.dwt
60. XRPackageSettings.asset
61. VFXManager.asset
62. README.md
63. grafik.jpg
64. grafik.jpg
65. XRSettings.asset
66. VersionControlSettings.asset
67. grafik.jpg
68. VR-locomotion-parkour-main.slnx
69. ProjectName.txt
70. EditorUserSettings.asset
71. default-6000.dwt
72. XRPackageSettings.asset
73. VFXManager.asset
74. grafik.jpg
75. README.md
76. grafik.jpg
77. XRSettings.asset
78. VersionControlSettings.asset
79. grafik.jpg
80. VR-locomotion-parkour-main.slnx
81. ProjectName.txt
82. EditorUserSettings.asset
83. default-6000.dwt
84. XRPackageSettings.asset
85. VFXManager.asset
86. README.md

87. grafik.jpg
88. VersionControlSettings.asset
89. grafik.jpg
90. grafik.jpg
91. VR-locomotion-parkour-main.slnx
92. ProjectName.txt
93. EditorUserSettings.asset
94. default-6000.dwlt
95. XRPackageSettings.asset
96. VFXManager.asset
97. XRSettings.asset
98. grafik.jpg
99. VersionControlSettings.asset
100. grafik.jpg
101. grafik.jpg
102. grafik.jpg
103. VR-locomotion-parkour-main.slnx
104. ProjectName.txt
105. EditorUserSettings.asset
106. default-6000.dwlt
107. README.md
108. XRPackageSettings.asset
109. VFXManager.asset
110. XRSettings.asset
111. grafik.jpg
112. VersionControlSettings.asset
113. grafik.jpg
114. grafik.jpg
115. grafik.jpg
116. VR-locomotion-parkour-main.slnx
117. ProjectName.txt
118. EditorUserSettings.asset
119. default-6000.dwlt
120. README.md
121. XRPackageSettings.asset
122. VFXManager.asset
123. XRSettings.asset
124. README.md

125. grafik.jpg
126. grafik.jpg
127. grafik.jpg
128. grafik.jpg
129. VR-locomotion-parkour-main.slnx
130. ProjectName.txt
131. EditorUserSettings.asset
132. default-6000.dwlt
133. XRPackageSettings.asset
134. VFXManager.asset
135. XRSettings.asset
136. VersionControlSettings.asset
137. README.md
138. grafik.jpg
139. grafik.jpg
140. XRSettings.asset
141. VersionControlSettings.asset
142. grafik.jpg
143. grafik.jpg
144. VR-locomotion-parkour-main.slnx
145. ProjectName.txt
146. EditorUserSettings.asset
147. default-6000.dwlt
148. XRPackageSettings.asset
149. VFXManager.asset
150. grafik.jpg
151. grafik.jpg
152. XRSettings.asset
153. VersionControlSettings.asset
154. grafik.jpg
155. grafik.jpg
156. grafik.jpg
157. VR-locomotion-parkour-main.slnx
158. ProjectName.txt
159. EditorUserSettings.asset
160. default-6000.dwlt
161. README.md
162. XRPackageSettings.asset

163. VFXManager.asset
164. grafik.jpg
165. VersionControlSettings.asset
166. grafik.jpg
167. grafik.jpg
168. grafik.jpg
169. grafik.jpg
170. grafik.jpg
171. VR-locomotion-parkour-main.slnx
172. ProjectName.txt
173. EditorUserSettings.asset
174. default-6000.dwlt
175. README.md
176. XRPackageSettings.asset
177. VFXManager.asset
178. XRSettings.asset
179. grafik.jpg
180. VersionControlSettings.asset
181. grafik.jpg
182. grafik.jpg
183. grafik.jpg
184. grafik.jpg
185. grafik.jpg
186. grafik.jpg
187. VR-locomotion-parkour-main.slnx
188. ProjectName.txt
189. EditorUserSettings.asset
190. default-6000.dwlt
191. README.md
192. XRPackageSettings.asset
193. VFXManager.asset
194. XRSettings.asset
195. grafik.jpg
196. VersionControlSettings.asset
197. grafik.jpg
198. grafik.jpg
199. grafik.jpg
200. grafik.jpg

201. grafik.jpg
202. grafik.jpg
203. VR-locomotion-parkour-main.slnx
204. ProjectName.txt
205. EditorUserSettings.asset
206. default-6000.dwlt
207. README.md
208. XRPackageSettings.asset
209. VFXManager.asset
210. XRSettings.asset
211. grafik.jpg
212. VersionControlSettings.asset
213. grafik.jpg
214. grafik.jpg
215. grafik.jpg
216. grafik.jpg
217. grafik.jpg
218. grafik.jpg
219. VR-locomotion-parkour-main.slnx
220. ProjectName.txt
221. EditorUserSettings.asset
222. default-6000.dwlt
223. README.md
224. XRPackageSettings.asset
225. VFXManager.asset
226. XRSettings.asset
227. README.md
228. grafik.jpg
229. grafik.jpg
230. grafik.jpg
231. grafik.jpg
232. grafik.jpg
233. grafik.jpg
234. grafik.jpg
235. VR-locomotion-parkour-main.slnx
236. ProjectName.txt
237. EditorUserSettings.asset
238. default-6000.dwlt

239. XRPackageSettings.asset
240. VFXManager.asset
241. XRSettings.asset
242. VersionControlSettings.asset
243. README.md
244. grafik.jpg
245. grafik.jpg
246. grafik.jpg
247. grafik.jpg
248. grafik.jpg
249. grafik.jpg
250. grafik.jpg
251. VR-locomotion-parkour-main.slnx
252. ProjectName.txt
253. EditorUserSettings.asset
254. default-6000.dwt
255. XRPackageSettings.asset
256. VFXManager.asset
257. XRSettings.asset
258. VersionControlSettings.asset
259. README.md
260. grafik.jpg
261. grafik.jpg
262. grafik.jpg
263. VFXManager.asset
264. XRSettings.asset
265. VersionControlSettings.asset
266. grafik.jpg
267. grafik.jpg
268. grafik.jpg
269. grafik.jpg
270. VR-locomotion-parkour-main.slnx
271. ProjectName.txt
272. EditorUserSettings.asset
273. default-6000.dwt
274. XRPackageSettings.asset
275. README.md
276. grafik.jpg

277. grafik.jpg
278. grafik.jpg
279. grafik.jpg
280. grafik.jpg
281. grafik.jpg
282. grafik.jpg
283. VR-locomotion-parkour-main.slnx
284. ProjectName.txt
285. EditorUserSettings.asset
286. default-6000.dwt
287. XRPackageSettings.asset
288. VFXManager.asset
289. XRSettings.asset
290. VersionControlSettings.asset
291. grafik.jpg
292. README.md
293. VersionControlSettings.asset
294. grafik.jpg
295. grafik.jpg
296. grafik.jpg
297. grafik.jpg
298. grafik.jpg
299. grafik.jpg
300. grafik.jpg
301. VR-locomotion-parkour-main.slnx
302. ProjectName.txt
303. EditorUserSettings.asset
304. default-6000.dwt
305. XRPackageSettings.asset
306. VFXManager.asset
307. XRSettings.asset
308. README.md
309. grafik.jpg
310. grafik.jpg
311. XRSettings.asset
312. VersionControlSettings.asset
313. grafik.jpg
314. grafik.jpg

315. grafik.jpg
316. grafik.jpg
317. grafik.jpg
318. grafik.jpg
319. VR-locomotion-parkour-main.slnx
320. ProjectName.txt
321. EditorUserSettings.asset
322. default-6000.dwlt
323. XRPackageSettings.asset
324. VFXManager.asset
325. README.md
326. grafik.jpg
327. grafik.jpg
328. grafik.jpg
329. grafik.jpg
330. grafik.jpg
331. grafik.jpg
332. grafik.jpg
333. grafik.jpg
334. VR-locomotion-parkour-main.slnx
335. ProjectName.txt
336. EditorUserSettings.asset
337. default-6000.dwlt
338. XRPackageSettings.asset
339. VFXManager.asset
340. XRSettings.asset
341. VersionControlSettings.asset
342. grafik.jpg
343. VersionControlSettings.asset
344. grafik.jpg
345. grafik.jpg
346. grafik.jpg
347. grafik.jpg
348. grafik.jpg
349. grafik.jpg
350. grafik.jpg
351. grafik.jpg
352. VR-locomotion-parkour-main.slnx

353. ProjectName.txt
354. EditorUserSettings.asset
355. default-6000.dwt
356. README.md
357. XRPackageSettings.asset
358. VFXManager.asset
359. XRSettings.asset
360. README.md
361. grafik.jpg
362. grafik.jpg
363. XRSettings.asset
364. VersionControlSettings.asset
365. grafik.jpg
366. grafik.jpg
367. grafik.jpg
368. grafik.jpg
369. grafik.jpg
370. grafik.jpg
371. grafik.jpg
372. VR-locomotion-parkour-main.slnx
373. ProjectName.txt
374. EditorUserSettings.asset
375. default-6000.dwt
376. XRPackageSettings.asset
377. VFXManager.asset
378. grafik.jpg
379. VersionControlSettings.asset
380. grafik.jpg
381. grafik.jpg
382. grafik.jpg
383. grafik.jpg
384. grafik.jpg
385. grafik.jpg
386. grafik.jpg
387. grafik.jpg
388. grafik.jpg
389. VR-locomotion-parkour-main.slnx
390. ProjectName.txt

391. EditorUserSettings.asset
392. default-6000.dwt
393. README.md
394. XRPackageSettings.asset
395. VFXManager.asset
396. XRSettings.asset
397. README.md
398. grafik.jpg
399. VersionControlSettings.asset
400. grafik.jpg
401. grafik.jpg
402. grafik.jpg
403. grafik.jpg
404. grafik.jpg
405. grafik.jpg
406. grafik.jpg
407. grafik.jpg
408. grafik.jpg
409. VR-locomotion-parkour-main.slnx
410. ProjectName.txt
411. EditorUserSettings.asset
412. default-6000.dwt
413. XRPackageSettings.asset
414. VFXManager.asset
415. XRSettings.asset
416. <https://github.com/immersive-command-system/ImmersiveDroneInterface/blob/master/Assets/Oculus/VR/Scripts/OVRInput.cs>
417. <https://github.com/tverona1/QuestAppLauncher/blob/master/Assets/OVRInputSelection/InputSystem/OVRInputHelpers.cs>
418. grafik.jpg
419. <https://github.com/NguyenAndrew/Grabbing-and-Throwing-objects-with-Oculus-Touch-and-CV1/blob/master/Assets/OVR/Scripts/OVRInput.cs>
420. XRPackageSettings.asset
421. VFXManager.asset
422. XRSettings.asset
423. VersionControlSettings.asset
424. grafik.jpg
425. grafik.jpg
426. grafik.jpg

427. grafik.jpg

428. grafik.jpg

429. grafik.jpg

430. grafik.jpg

431. grafik.jpg

432. grafik.jpg

433. <https://docs.unity3d.com/550/Documentation/Manual/OculusControllers.html>

434. <https://communityforums.atmeta.com/discussions/dev-general/issue-with-ovrinput-get/812184>

435. https://www.reddit.com/r/oculus/comments/5j34sv/unity_touchsecondarythumbrest_keeps_on_switching/

436. https://www.reddit.com/r/learnVRdev/comments/kaitky/i_added_a_rigidbody_box_collider_and_ovr/

437. <https://joyplot.com/documents/oculus-integration-sample/>

438. <https://stackoverflow.com/questions/63211352/unity-oculus-quest-how-to-read-thumbstick-input>

439. <https://communityforums.atmeta.com/discussions/dev-quest/converting-touch-controller-position-to-handtracking-pinch-position/810172>

440. <https://cs444-practice.epfl.ch/tp/tp3/>

441. <https://communityforums.atmeta.com/discussions/dev-unity/why-is-ovr-get-not-returning-correct-values-for-axis2d-arguments-on-quest/738102>

442. <https://www.marui-plugin.com/documentation-mivry-questhands/>

443. <https://timcoster.com/2024/01/02/unity-gamepad-controller-input-button-mapping-examples/>

444. <https://gitlab.inf.uni-konstanz.de/felix.kalchschmid/ia-land-usage/-/blob/fixd-setup/Assets/Oculus/VR/Scripts/OVRCameraRig.cs>

445. VR-locomotion-parkour-main.slnx

446. ProjectName.txt

447. EditorUserSettings.asset

448. default-6000.dwt

449. README.md

450. <https://docs.unity3d.com/550/Documentation/Manual/OculusControllers.html>

451. <https://github.com/immersive-command-system/ImmersiveDroneInterface/blob/master/Assets/Oculus/VR/Scripts/OVRInput.cs>

452. XRSettings.asset

453. VersionControlSettings.asset

454. grafik.jpg

455. grafik.jpg

456. grafik.jpg

457. grafik.jpg

458. grafik.jpg

459. grafik.jpg

460. grafik.jpg

461. grafik.jpg

462. grafik.jpg

463. grafik.jpg

464. <https://communityforums.atmeta.com/discussions/dev-general/issue-with-ovrinput-get/812184>

465. https://www.reddit.com/r/oculus/comments/5j34sv/unity_touchsecondarythumbrest_keeps_on_switching/

466. https://www.reddit.com/r/learnVRdev/comments/kaitky/i_added_a_rigidbody_box_collider_and_ovr/

467. <https://joyplot.com/documents/oculus-integration-sample/>

468. <https://cs444-practice.epfl.ch/tp/tp3/>

469. VR-locomotion-parkour-main.slnx

470. ProjectName.txt

471. EditorUserSettings.asset

472. default-6000.dwt

473. README.md

474. XRPackageSettings.asset

475. VFXManager.asset

476. <https://github.com/NguyenAndrew/Grabbing-and-Throwing-objects-with-Oculus-Touch-and-CV1/blob/master/Assets/OVR/Scripts/OVRInput.cs>

477. VersionControlSettings.asset

478. grafik.jpg

479. grafik.jpg

480. grafik.jpg

481. grafik.jpg

482. grafik.jpg

483. grafik.jpg

484. grafik.jpg

485. grafik.jpg

486. grafik.jpg

487. VR-locomotion-parkour-main.slnx

488. grafik.jpg

489. <https://docs.unity3d.com/550/Documentation/Manual/OculusControllers.html>

490. <https://communityforums.atmeta.com/discussions/dev-general/issue-with-ovrinput-get/812184>

491. https://www.reddit.com/r/oculus/comments/5j34sv/unity_touchsecondarythumbrest_keeps_on_switching/

492. https://developer.unigine.com/en/docs/latest/vr_development/vr_input

493. <https://docs.unity3d.com/2019.2/Documentation/Manual/OculusControllers.html>

494. <https://cs444-practice.epfl.ch/tp/tp3/>

495. <https://timcoster.com/2024/01/02/unity-gamepad-controller-input-button-mapping-examples/>

496. ProjectName.txt

497. EditorUserSettings.asset

498. default-6000.dwlt
499. README.md
500. XRPackageSettings.asset
501. VFXManager.asset
502. XRSettings.asset
503. README.md
504. grafik.jpg
505. grafik.jpg
506. grafik.jpg
507. grafik.jpg
508. grafik.jpg
509. grafik.jpg
510. grafik.jpg
511. grafik.jpg
512. grafik.jpg
513. grafik.jpg
514. VR-locomotion-parkour-main.slnx
515. ProjectName.txt
516. EditorUserSettings.asset
517. default-6000.dwlt
518. XRPackageSettings.asset
519. VFXManager.asset
520. XRSettings.asset
521. VersionControlSettings.asset
522. grafik.jpg
523. VersionControlSettings.asset
524. grafik.jpg
525. grafik.jpg
526. grafik.jpg
527. grafik.jpg
528. grafik.jpg
529. grafik.jpg
530. grafik.jpg
531. grafik.jpg
532. grafik.jpg
533. VR-locomotion-parkour-main.slnx
534. grafik.jpg
535. ProjectName.txt

536. EditorUserSettings.asset
537. default-6000.dwt
538. README.md
539. XRPackageSettings.asset
540. VFXManager.asset
541. XRSettings.asset
542. <https://www.vrwiki.cs.brown.edu/vr-development-software/unity/quest-controller-input-mapping>
543. VersionControlSettings.asset
544. grafik.jpg
545. grafik.jpg
546. grafik.jpg
547. grafik.jpg
548. grafik.jpg
549. grafik.jpg
550. grafik.jpg
551. grafik.jpg
552. grafik.jpg
553. VR-locomotion-parkour-main.slnx
554. grafik.jpg
555. grafik.jpg
556. <https://www.youtube.com/watch?v=oujH2Dz33ZY>
557. <https://www.youtube.com/watch?v=yDpINn9HIS4>
558. <https://docs.unity3d.com/550/Documentation/Manual/OculusControllers.html>
559. <https://timcoster.com/2024/01/02/unity-gamepad-controller-input-button-mapping-examples/>
560. https://www.reddit.com/r/oculus/comments/5j34sv/unity_touchsecondarythumbrest_keeps_on_switching/
561. <https://blog.nobreakpoints.com/oculus-quest-unity-controller-buttons-verwenden/>
562. <https://stackoverflow.com/questions/63211352/unity-oculus-quest-how-to-read-thumbstick-input>
563. https://docs.unity3d.com/6000.3/Documentation/Manual/xr_input.html
564. ProjectName.txt
565. EditorUserSettings.asset
566. default-6000.dwt
567. README.md
568. XRPackageSettings.asset
569. VFXManager.asset
570. XRSettings.asset
571. https://www.reddit.com/r/Unity3D/comments/1qih5zl/tutorial_arcade_raycast_car_physics_in_4_min/
572. <https://www.youtube.com/watch?v=sWshRRDxdSU>
573. <https://docs.unity3d.com/6000.3/Documentation/Manual/wheel-colliders-introduction.html>

574. VFXManager.asset
575. XRSettings.asset
576. VersionControlSettings.asset
577. grafik.jpg
578. grafik.jpg
579. grafik.jpg
580. grafik.jpg
581. grafik.jpg
582. grafik.jpg
583. grafik.jpg
584. grafik.jpg
585. grafik.jpg
586. grafik.jpg
587. grafik.jpg
588. <https://www.youtube.com/watch?v=7PYREq2OmHA>
589. https://www.youtube.com/watch?v=239_6ra2V3w
590. https://www.reddit.com/r/unity_tutorials/comments/rki5f2/using_raycast/
591. https://www.youtube.com/watch?v=YXxOOo5_z6c
592. https://www.reddit.com/r/Unity3D/comments/rzqd40/i_made_an_object_movementanimation_system_to_my/
593. https://www.reddit.com/r/Unity2D/comments/5gbo77/raycast_collisions_vs_rigidbody/
594. https://www.reddit.com/r/gamedev/comments/1mvtjhv/how_to_add_force_at_the_wheels_of_a_raycast/
595. <https://www.facebook.com/groups/132728896890594/posts/2571748329655293/>
596. <https://github.com/hayden-donnely/vehicle-physics>
597. https://www.youtube.com/watch?v=Yyx_rkBNear
598. <https://www.youtube.com/watch?v=9Cdky9Wmus8>
599. <https://www.youtube.com/watch?v=SIJPqI5cnks>
600. VR-locomotion-parkour-main.slnx
601. ProjectName.txt
602. EditorUserSettings.asset
603. default-6000.dwt
604. README.md
605. XRPackageSettings.asset
606. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/WheelCollider.html>
607. <https://docs.unity3d.com/2020.1/Documentation/Manual/class-WheelCollider.html>
608. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rigidbody-centerOfMass.html>
609. VFXManager.asset
610. XRSettings.asset
611. VersionControlSettings.asset

612. grafik.jpg
613. grafik.jpg
614. grafik.jpg
615. grafik.jpg
616. grafik.jpg
617. grafik.jpg
618. grafik.jpg
619. grafik.jpg
620. grafik.jpg
621. grafik.jpg
622. grafik.jpg
623. <https://docs.unity3d.com/2020.1/Documentation/ScriptReference/WheelCollider.html>
624. <https://stackoverflow.com/questions/66492545/unity-car-acceleration-problem-after-brake>
625. <https://www.gamedeveloper.com/business/unity-wheel-collider-for-motor-vehicle-tutorial-2018>
626. <https://hutonggames.com/playmakerforum/index.php?topic=19321.0>
627. <https://www.youtube.com/watch?v=n2VOu5d2wVM>
628. <https://www.youtube.com/watch?v=tQmxW277dXk>
629. https://www.youtube.com/watch?v=05DFrT_fm1E
630. VR-locomotion-parkour-main.slnx
631. ProjectName.txt
632. EditorUserSettings.asset
633. default-6000.dwt
634. [README.md](#)
635. XRPackageSettings.asset
636. <https://docs.unity3d.com/560/Documentation/Manual/class-WheelCollider.html>
637. <https://docs.unity3d.com/ScriptReference/WheelCollider-suspensionSpring.html>
638. <https://docs.unity3d.com/6000.3/Documentation/Manual/wheel-colliders-introduction.html>
639. VFXManager.asset
640. XRSettings.asset
641. VersionControlSettings.asset
642. grafik.jpg
643. grafik.jpg
644. grafik.jpg
645. grafik.jpg
646. grafik.jpg
647. grafik.jpg
648. grafik.jpg
649. grafik.jpg

650. grafik.jpg

651. grafik.jpg

652. grafik.jpg

653. https://www.reddit.com/r/Unity3D/comments/1hlc8qx/wheel_collider_making_my_car_jump_in_unity/

654. https://www.reddit.com/r/unity/comments/1hl8wls/problem_car_jumping_on_collision_entities_unity/

655. https://www.reddit.com/r/Unity3D/comments/4jc0e2/unity_5_wheel_collider_how_to_prevent_sudden/

656. https://www.reddit.com/r/Unity3D/comments/39kuug/i_cant_turn_the_wheel_collider_even_after/

657. https://www.reddit.com/r/Unity3D/comments/4k8ogc/help_why_do_the_wheelcollider_jitter_so_hard/

658. <https://www.facebook.com/groups/1000038303359383/posts/8495351500494655/>

659. <https://support.vehiclephysics.com/64/suspension-spring-and-damper-values-change-in-runtime>

660. <https://www.youtube.com/watch?v=3D0dYwe3U6c>

661. https://www.reddit.com/r/Unity3D/comments/76vkhz/cant_make_wheelcollider_work_no_matter_what/

662. https://www.youtube.com/watch?v=c-yhZwXSx_c

663. VR-locomotion-parkour-main.slnx

664. ProjectName.txt

665. EditorUserSettings.asset

666. default-6000.dwlt

667. [README.md](#)

668. XRPackageSettings.asset

669. <https://docs.unity3d.com/560/Documentation/Manual/class-WheelCollider.html>

670. <https://docs.unity3d.com/ScriptReference/WheelCollider-suspensionSpring.html>

671. <https://docs.unity3d.com/6000.3/Documentation/Manual/WheelColliderTutorial.html>

672. https://www.reddit.com/r/unity/comments/1hl8wls/problem_car_jumping_on_collision_entities_unity/

673. https://www.reddit.com/r/Unity3D/comments/4k8ogc/help_why_do_the_wheelcollider_jitter_so_hard/

674. https://www.reddit.com/r/Unity3D/comments/9qnj68/wheelcolliders_drive_me_crazy/

675. default-6000.dwlt

676. [README.md](#)

677. XRPackageSettings.asset

678. VFXManager.asset

679. XRSettings.asset

680. VersionControlSettings.asset

681. grafik.jpg

682. grafik.jpg

683. grafik.jpg

684. grafik.jpg

685. grafik.jpg

686. grafik.jpg

687. grafik.jpg

688. grafik.jpg

689. grafik.jpg

690. grafik.jpg

691. grafik.jpg

692. <https://stackoverflow.com/questions/43218183/rigidbody-without-extra-physics-behavior-in-unity>

693. <https://www.facebook.com/groups/511573835572177/posts/4670295739699945/>

694. <https://darajat.ly/wp-content/uploads/2024/06/CGameProgramming.pdf>

695. https://www.youtube.com/watch?v=j6_SMdWeGFI

696. <https://docs.unity3d.com/6000.3/Documentation/Manual/rigidbody-interpolation.html>

697. <https://github.com/shadowmage45/KSPWheel/blob/master/UnityProject/Assets/Scripts/KSPWheelCollider.cs>

698. <https://www.facebook.com/groups/unity3d/posts/1667424036659451/>

699. VR-locomotion-parkour-main.slnx

700. ProjectName.txt

701. EditorUserSettings.asset

702. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/WheelCollider.ConfigureVehicleSubsteps.html>

703. <https://docs.unity3d.com/6000.3/Documentation/Manual/wheel-colliders-suspension.html>

704. <https://docs.unity3d.com/ScriptReference/WheelCollider-suspensionSpring.html>

705. <https://docs.unity3d.com/560/Documentation/Manual/class-WheelCollider.html>

706. <https://docs.unity3d.com/6000.1/Documentation/ScriptReference/Physics-defaultSolverIterations.html>

707. README.md

708. XRPackageSettings.asset

709. VFXManager.asset

710. XRSettings.asset

711. VersionControlSettings.asset

712. grafik.jpg

713. grafik.jpg

714. grafik.jpg

715. grafik.jpg

716. grafik.jpg

717. grafik.jpg

718. grafik.jpg

719. grafik.jpg

720. grafik.jpg

721. grafik.jpg

722. grafik.jpg

723. https://www.reddit.com/r/Unity3D/comments/76vkhz/cant_make_wheelcollider_work_no_matter_what/

724. https://www.youtube.com/watch?v=c-yhZwXSx_c

725. <https://www.youtube.com/watch?v=YaZSuOYgsLs>

726. <https://www.youtube.com/watch?v=j1IPUXhEB08>

727. https://www.reddit.com/r/unity/comments/1hl8wls/problem_car_jumping_on_collision_entities_unity/

728. <https://docs.unity.cn/2021.1/Documentation/Manual/class-WheelCollider.html>

729. <https://docs.unity3d.com/420/Documentation/Components/class-PhysicsManager.html>

730. https://www.reddit.com/r/Unity3D/comments/1hlc8qx/wheel_collider_making_my_car_jump_in_unity/

731. <https://docs.unity3d.com/es/2018.3/Manual/class-PhysicsManager.html>

732. VR-locomotion-parkour-main.slnx

733. ProjectName.txt

734. EditorUserSettings.asset

735. default-6000.dwt

736. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rigidbody-isKinematic.html>

737. <https://docs.unity3d.com/2022.2/Documentation/Manual/class-WheelCollider.html>

738. https://www.reddit.com/r/Unity3D/comments/utmzfx/best_way_to_move_kinematic_rigidbody/

739. VFXManager.asset

740. XRSettings.asset

741. VersionControlSettings.asset

742. grafik.jpg

743. grafik.jpg

744. grafik.jpg

745. grafik.jpg

746. grafik.jpg

747. grafik.jpg

748. grafik.jpg

749. grafik.jpg

750. grafik.jpg

751. grafik.jpg

752. grafik.jpg

753. <https://docs.unity3d.com/2017.2/Documentation/ScriptReference/Rigidbody-isKinematic.html>

754. https://www.reddit.com/r/Unity3D/comments/vsxf75/i_am_trying_to_disable_this_objects_rigidbody_if/

755. https://www.youtube.com/watch?v=_7gl-_64xgM

756. <https://docs.unity.cn/2017.2/Documentation/Manual/class-WheelCollider.html>

757. <https://stackoverflow.com/questions/52356896/disable-wheelcolliders-slip-non-realistic>

758. <https://www.youtube.com/watch?v=UA2F8s3fCLs>

759. <https://www.youtube.com/watch?v=rY689NT1MD8>

760. <https://www.youtube.com/watch?v=qKP2RzI5wiA>

761. <https://issuetracker.unity3d.com/issues/moving-a-rigidbody-with-transform-dot-position-causes-its-kinematic-child-object-to-be-misplaced-after-the-movement>

762. VR-locomotion-parkour-main.slnx

763. ProjectName.txt

764. EditorUserSettings.asset

765. default-6000.dwt

766. README.md

767. XRPackageSettings.asset

768. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rigidbody.MovePosition.html>

769. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Rigidbody-isKinematic.html>

770. XRSettings.asset

771. VersionControlSettings.asset

772. grafik.jpg

773. grafik.jpg

774. grafik.jpg

775. grafik.jpg

776. grafik.jpg

777. grafik.jpg

778. grafik.jpg

779. grafik.jpg

780. grafik.jpg

781. grafik.jpg

782. grafik.jpg

783. <https://docs.unity3d.com/ScriptReference/Rigidbody.MovePosition.html>

784. <https://docs.unity3d.com/6000.2/Documentation/ScriptReference/Rigidbody.MovePosition.html>

785. <https://www.youtube.com/watch?v=KjaRQr74jV0>

786. <https://docs.huihoo.com/unity/5.5/Documentation/ScriptReference/Rigidbody.MovePosition.html>

787. <https://stackoverflow.com/questions/68801079/unity-rigidbody-moveposition-is-not-working-if-transform-position-is-just-cha>

788. <https://docs.unity3d.com/ru/2019.3/ScriptReference/Rigidbody.MovePosition.html>

789. <https://gamedevbeginner.com/how-to-use-move-position-in-unity/>

790. VR-locomotion-parkour-main.slnx

791. <https://stackoverflow.com/questions/51454024/when-to-use-transform-position-and-rigidbody-position-in-unity3d>

792. <https://stackoverflow.com/questions/48900499/rigidbody-moveposition-issue>

793. https://www.reddit.com/r/Unity3D/comments/vbtpgl/rigidbody_moveposition_is_not_respecting/

794. <https://www.youtube.com/watch?v=ixM2W2tPn6c>

795. https://www.reddit.com/r/Unity3D/comments/3ou3n6/moveposition_not_as_precise_as_translate_with/

796. https://www.reddit.com/r/Unity3D/comments/1rekls/how_important_is_it_to_not_move_a_rigidbody_via/

797. ProjectName.txt

798. EditorUserSettings.asset

799. default-6000.dwt

800. [README.md](#)

801. XRPackageSettings.asset

802. VFXManager.asset

803. <https://www.youtube.com/watch?v=YdhtPAiebHo>

804. <https://mixedrealitynow.com/getting-started-with-meta-xr-interaction-sdk-quest-3-how-to-crucial-interactions>

805. <https://nextftc.dev/nextftc/hardware/drivetrain-commands/differential>

806. <https://www.youtube.com/watch?v=heqAlbxtonE>

807. https://www.youtube.com/watch?v=OaPQPY_wuzk

808. https://www.reddit.com/r/OculusQuest/comments/v201lb/i_made_satisfying_knobs_for_the_interaction_sdk/

809. default-6000.dwlt

810. [README.md](#)

811. XRPackageSettings.asset

812. VFXManager.asset

813. XRSettings.asset

814. VersionControlSettings.asset

815. grafik.jpg

816. grafik.jpg

817. grafik.jpg

818. grafik.jpg

819. grafik.jpg

820. grafik.jpg

821. grafik.jpg

822. grafik.jpg

823. grafik.jpg

824. grafik.jpg

825. grafik.jpg

826. <https://www.youtube.com/watch?v=4kGD8q5kEx8>

827. <https://developers.facebook.com/blog/post/2022/11/22/building-intuitive-interactions-vr/>

828. https://www.reddit.com/r/unity/comments/1g90v5p/make_a_joystick_in_vr/

829. <https://www.youtube.com/watch?v=xk-LLY7scvM>

830. <https://www.youtube.com/watch?v=K5Jtfsar-IE>

831. https://www.reddit.com/r/explainlikeimfive/comments/p0z84y/eli5_how_do_tanks_control_both_sets_of_tracks/

832. <https://www.youtube.com/watch?v=Rbdyr-qQqV0>

833. <https://communityforums.atmeta.com/discussions/dev-unity/unity-6-1--meta-quest--building-blocks--simple-interactable/1341398>

834. <https://communityforums.atmeta.com/discussions/dev-unity/any-resources-for-rotating-knob-interaction/1228926>

835. https://www.reddit.com/r/oculusdev/comments/1f0sc9i/one_grab_translate_transformer_also_constraint/

836. <https://www.youtube.com/watch?v=pnoOm6vrTcw>

837. https://communityforums.atmeta.com/discussions/Questions_Discussions/reset-a-one-grab-rotate-transformer/1269247

838. <https://www.youtube.com/watch?v=aI7CMBPsPHI>

839. http://www.th-nuremberg.de/mshopf/uploads/Main/Zargatalebi_2025.pdf

840. VR-locomotion-parkour-main.slnx

841. ProjectName.txt

842. EditorUserSettings.asset

843. grafik.jpg

844. [README.md](#)

845. <https://nextftc.dev/nextftc/hardware/drivetrain-commands/differential>

846. XRSettings.asset

847. VersionControlSettings.asset

848. grafik.jpg

849. grafik.jpg

850. grafik.jpg

851. grafik.jpg

852. grafik.jpg

853. grafik.jpg

854. grafik.jpg

855. grafik.jpg

856. grafik.jpg

857. grafik.jpg

858. grafik.jpg

859. VR-locomotion-parkour-main.slnx

860. ProjectName.txt

861. EditorUserSettings.asset

862. default-6000.dwt

863. XRPackageSettings.asset

864. VFXManager.asset

865. [README.md](#)

866. grafik.jpg

867. grafik.jpg

868. grafik.jpg

869. grafik.jpg

870. grafik.jpg

871. grafik.jpg

872. grafik.jpg
873. grafik.jpg
874. grafik.jpg
875. grafik.jpg
876. VR-locomotion-parkour-main.slnx
877. grafik.jpg
878. grafik.jpg
879. ProjectName.txt
880. EditorUserSettings.asset
881. default-6000.dwt
882. XRPackageSettings.asset
883. VFXManager.asset
884. XRSettings.asset
885. VersionControlSettings.asset
886. grafik.jpg
887. VersionControlSettings.asset
888. grafik.jpg
889. grafik.jpg
890. grafik.jpg
891. grafik.jpg
892. grafik.jpg
893. grafik.jpg
894. grafik.jpg
895. grafik.jpg
896. grafik.jpg
897. VR-locomotion-parkour-main.slnx
898. grafik.jpg
899. grafik.jpg
900. grafik.jpg
901. ProjectName.txt
902. EditorUserSettings.asset
903. default-6000.dwt
904. README.md
905. XRPackageSettings.asset
906. VFXManager.asset
907. XRSettings.asset
908. <https://www.youtube.com/watch?v=YdhtPAiebHo>
909. <https://communityforums.atmeta.com/discussions/dev-quest/interaction-button/1132675>

910. XRSettings.asset

911. VersionControlSettings.asset

912. grafik.jpg

913. grafik.jpg

914. grafik.jpg

915. grafik.jpg

916. grafik.jpg

917. grafik.jpg

918. grafik.jpg

919. grafik.jpg

920. grafik.jpg

921. grafik.jpg

922. grafik.jpg

923. grafik.jpg

924. grafik.jpg

925. https://communityforums.atmeta.com/discussions/Questions_Discussions/reset-a-one-grab-rotate-transformer/1269247

926. https://www.reddit.com/r/oculusdev/comments/1f0sc9i/one_grab_translate_transformer_also_constraining/

927. <https://github.com/oculus-samples/Unity-CrypticCabinet/blob/main/Assets/CrypticCabinet/Scripts/Interactions/OneGrabToggleRotateTransformer.cs>

928. <https://www.youtube.com/watch?v=heqAlbxtonE>

929. https://www.reddit.com/r/vrdev/comments/18ljsux/oculus_integration_sdkmeta_xr_all_in_one_sdk_poke/

930. VR-locomotion-parkour-main.slnx

931. <https://communityforums.atmeta.com/discussions/dev-unreal/v77-grab-hand-pose-translationrotation/1343107>

932. <https://developers.facebook.com/blog/post/2022/11/22/building-intuitive-interactions-vr/>

933. <https://communityforums.atmeta.com/discussions/dev-unity/poke-interaction---trigger-event-in-unity/1244682>

934. <https://www.youtube.com/watch?v=P0rbFHWpKnA>

935. https://www.youtube.com/watch?v=OaPQPY_wuzk

936. <https://stackoverflow.com/questions/57636366/how-do-i-lock-the-rotation-of-a-grabbed-object-with-unity-oculus-vr>

937. https://www.youtube.com/watch?v=Q_mpCIHtG9I

938. <https://www.youtube.com/watch?v=pnoOm6vrTcw>

939. ProjectName.txt

940. EditorUserSettings.asset

941. default-6000.dwl

942. README.md

943. XRPackageSettings.asset

944. VFXManager.asset

945. <https://docs.unity3d.com/550/Documentation/Manual/OculusControllers.html>

946. <https://github.com/immersive-command-system/ImmersiveDroneInterface/blob/master/Assets/Oculus/VR/Scripts/OVRInput.cs>

947. <https://docs.unity3d.com/560/Documentation/Manual/class-LineRenderer.html>

948. VFXManager.asset

949. XRSettings.asset

950. VersionControlSettings.asset

951. grafik.jpg

952. grafik.jpg

953. grafik.jpg

954. grafik.jpg

955. grafik.jpg

956. grafik.jpg

957. grafik.jpg

958. grafik.jpg

959. grafik.jpg

960. grafik.jpg

961. grafik.jpg

962. grafik.jpg

963. grafik.jpg

964. https://www.reddit.com/r/oculus/comments/5j34sv/unity_touchsecondarythumbrest_keeps_on_switching/

965. <https://timcoster.com/2024/01/02/unity-gamepad-controller-input-button-mapping-examples/>

966. <https://docs.unity3d.com/2017.1/Documentation/Manual/OpenVRControllers.html>

967. https://www.reddit.com/r/Unity3D/comments/63xee1/need_help_making_a_moving_laser_beam_using_a_line/

968. <https://stackoverflow.com/questions/74415157/how-do-i-map-continuous-movement-to-oculus-quest-2-controllers-thumbstick>

969. <https://stackoverflow.com/questions/61976721/an-object-was-hit-by-a-beam-unity>

970. <https://communityforums.atmeta.com/discussions/dev-unity/ovrinput-thumbstickupdownleftright-threshold-configurable/745092>

971. <https://www.youtube.com/watch?v=yDpINn9HIS4>

972. <https://forum.deovr.com/d/2653-quest-3-steamvr-thumbstick-abxy-not-registering-still>

973. <https://forums.flightsimulator.com/t/cannot-assign-vr-controller-thumbstick-up-down-like-in-msfs2020/676913>

974. VR-locomotion-parkour-main.slnx

975. ProjectName.txt

976. EditorUserSettings.asset

977. default-6000.dwl

978. [README.md](#)

979. XRPackageSettings.asset

980. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Transform.InverseTransformPoint.html>

981. <https://stackoverflow.com/questions/65220848/gameobject-get-position-relative-to-grandparent>

982. XRSettings.asset

983. VersionControlSettings.asset

984. grafik.jpg

985. grafik.jpg

986. grafik.jpg

987. grafik.jpg

988. grafik.jpg

989. grafik.jpg

990. grafik.jpg

991. grafik.jpg

992. grafik.jpg

993. grafik.jpg

994. grafik.jpg

995. grafik.jpg

996. grafik.jpg

997. <https://www.youtube.com/watch?v=nDvHgSTXAUU>

998. https://www.reddit.com/r/oculusdev/comments/16cguue/is_there_a_way_to_distinguish_controller_being_in/

999. <https://github.com/NguyenAndrew/Grabbing-and-Throwing-objects-with-Oculus-Touch-and-CV1/blob/master/Assets/OVR/Scripts/OVRInput.cs>

1000. <https://skarredghost.com/2022/01/05/how-to-oculus-spatial-anchors-unity-2/>

1001. <https://communityforums.atmeta.com/t5/Unity-Development/How-to-get-position-vector3-variable-of-quest-headset/td-p/755976>

1002. VR-locomotion-parkour-main.slnx

1003. <https://cgvr.git.ulo.no.net/henrikel/DeltaVR3DModelGeneration/src/commit/17598f81afc6ab5db2bf005878aa36bee5d908e1/Assets/Oculus/VR/Scripts/OVRCameraRig.cs>

1004. <https://github.com/IndieGameMaker/UnityVRAR/blob/master/ShadowGunVR/Assets/Oculus/VR/Scripts/OVRCameraRig.cs>

1005. <https://communityforums.atmeta.com/discussions/dev-quest/position-of-ovrcamerarig/1164020>

1006. <https://www.youtube.com/watch?v=xk-LLY7scvM>

1007. <https://gitlab.inf.uni-konstanz.de/felix.kalchschmid/ia-land-usage/-/blob/fixed-setup/Assets/Oculus/VR/Scripts/OVRCameraRig.cs>

1008. https://www.reddit.com/r/Unity3D/comments/d6hz0y/is_there_a_way_to_mimic/

1009. <https://wlsdn629.tistory.com/entry/유니티-OVR-Controller의-속도-및-위치-값을-받아오고-싶을-때>

1010. <https://github.com/RootSoft/Oculus-Quest-Setup>

1011. ProjectName.txt

1012. EditorUserSettings.asset

1013. default-6000.dwl

1014. README.md

1015. XRPackageSettings.asset

1016. VFXManager.asset

1017. <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Transform.InverseTransformPoint.html>

1018. <https://github.com/immersive-command-system/ImmersiveDroneInterface/blob/master/Assets/Oculus/VR/Scripts/OVRInput.cs>

1019. <https://docs.unity3d.com/2017.1/Documentation/Manual/OpenVRControllers.html>

1020. grafik.jpg

1021. XRPackageSettings.asset

1022. VFXManager.asset

1023. XRSettings.asset

1024. VersionControlSettings.asset

1025. grafik.jpg

1026. grafik.jpg

1027. grafik.jpg

1028. grafik.jpg

1029. grafik.jpg

1030. grafik.jpg

1031. grafik.jpg

1032. grafik.jpg

1033. grafik.jpg

1034. grafik.jpg

1035. grafik.jpg

1036. grafik.jpg

1037. <https://docs.unity3d.com/ja/2017.1/Manual/OpenVRControllers.html>

1038. <https://qiita.com/nkjzm/items/96cd9cddc645c45dd5e5>

1039. <https://joyplot.com/documents/oculus-integration-sample/>

1040. <https://github.com/NguyenAndrew/Grabbing-and-Throwing-objects-with-Oculus-Touch-and-CV1/blob/master/Assets/OVR/Scripts/OVRInput.cs>

1041. <https://www.twblogs.net/a/5b8318a82b717766a1eb090b>

1042. <https://communityforums.atmeta.com/discussions/dev-unity/ovrinput-getovrinput-axis2d-secondarythumbstick-always-returns-00/896632>

1043. <https://infos.tistory.com/4486>

1044. https://www.reddit.com/r/godot/comments/sicfhd/oculus_controller_buttons/

1045. VR-locomotion-parkour-main.slnx

1046. ProjectName.txt
1047. EditorUserSettings.asset
1048. default-6000.dwlt
1049. README.md
1050. grafik.jpg
1051. VersionControlSettings.asset
1052. grafik.jpg
1053. grafik.jpg
1054. grafik.jpg
1055. grafik.jpg
1056. grafik.jpg
1057. grafik.jpg
1058. grafik.jpg
1059. grafik.jpg
1060. grafik.jpg
1061. VR-locomotion-parkour-main.slnx
1062. grafik.jpg
1063. grafik.jpg
1064. grafik.jpg
1065. <https://stackoverflow.com/questions/59392800/clamping-rotation-quaternion>
1066. https://www.reddit.com/r/Unity3D/comments/732uqs/limiting_the_rotation_of_pitch_and_yaw_without/
1067. <https://community.gamedev.tv/t/x-y-rotation-dont-go-smooth/207337>
1068. <https://www.facebook.com/groups/IndieGameDevs/posts/10154986360606573/>
1069. <https://thunderstore.io/c/repo/p/DirtyGames/REPOGambling/source/>
1070. https://thunderstore.io/c/h3vr/p/JerryAr/BN2_NV_Binoculars/source/
1071. https://docs.unity3d.com/520/Documentation/ScriptReference/30_search.html?q=
1072. ProjectName.txt
1073. https://thunderstore.io/c/repo/p/SNEUS_STUDIO/RETEMINATOR/source/
1074. https://docs.huihoo.com/unity/5.5/Documentation/ScriptReference/40_history.html
1075. https://docs.unity3d.com/2017.1/Documentation/ScriptReference/30_search.html?q=
1076. <https://gist.github.com/dd93bfc7d9b93166ccde3b5be131c294>
1077. https://thunderstore.io/c/repo/p/Dafini/Dafis_Mega_Monster_Pack_Emporium_REPO_Edition/source/
1078. <http://stephenchildree.com/files/UnityEngineKeywords.txt>
1079. https://docs.unity3d.com/520/Documentation/ScriptReference/40_history.html
1080. EditorUserSettings.asset
1081. default-6000.dwlt
1082. README.md
1083. XRPackageSettings.asset

1084. VFXManager.asset
1085. XRSettings.asset
1086. <https://stackoverflow.com/questions/40553713/unity3d-linereenderer-not-visible>
1087. <https://docs.unity3d.com/ScriptReference/LineRenderer-useWorldSpace.html>
1088. <https://www.tengio.com/blog/oculus-touch-controls/>
1089. grafik.jpg
1090. XRPackageSettings.asset
1091. VFXManager.asset
1092. XRSettings.asset
1093. VersionControlSettings.asset
1094. grafik.jpg
1095. grafik.jpg
1096. grafik.jpg
1097. grafik.jpg
1098. grafik.jpg
1099. grafik.jpg
1100. grafik.jpg
1101. grafik.jpg
1102. grafik.jpg
1103. grafik.jpg
1104. grafik.jpg
1105. grafik.jpg
1106. https://www.reddit.com/r/Unity3D/comments/1et9bry/why_is_the_line_renderer_not_showing_a_smooth/
1107. https://www.reddit.com/r/Unity3D/comments/19aox47/line_renderer_material_only_visible_when_unlit/
1108. <https://www.youtube.com/watch?v=lq9DvQyEync>
1109. <https://communityforums.atmeta.com/discussions/dev-unity/fixing-position-of-positional-tracker-in-virtual-space-unity/307520>
1110. <https://docs.unity3d.com/6000.0/Documentation/Manual/urp/particles-unlit-shader.html>
1111. <https://argos.vu/a-detailed-look-at-oculus-utilities-for-unity/>
1112. <https://gitlab.inf.uni-konstanz.de/felix.kalchschmid/ia-land-usage/-/blob/fixd-setup/Assets/Oculus/VR/Scripts/OVRCameraRig.cs>
1113. <https://github.com/RootSoft/Oculus-Quest-Setup>
1114. https://www.reddit.com/r/vrdev/comments/gkgr86/unity_only_able_to_move_my_ovrplayercontroller/
1115. VR-locomotion-parkour-main.slnx
1116. ProjectName.txt
1117. EditorUserSettings.asset
1118. default-6000.dwl
1119. README.md
1120. <https://www.cs.columbia.edu/~allen/F17/NOTES/icckinematics.pdf>

1121. https://en.wikipedia.org/wiki/Differential_wheeled_robot

1122. <https://docs.wpilib.org/en/stable/docs/software/hardware-apis/motors/wpi-drive-classes.html>

1123. VFXManager.asset

1124. XRSettings.asset

1125. VersionControlSettings.asset

1126. grafik.jpg

1127. grafik.jpg

1128. grafik.jpg

1129. grafik.jpg

1130. grafik.jpg

1131. grafik.jpg

1132. grafik.jpg

1133. grafik.jpg

1134. grafik.jpg

1135. grafik.jpg

1136. grafik.jpg

1137. grafik.jpg

1138. grafik.jpg

1139. <https://www.studysmarter.co.uk/explanations/engineering/robotics-engineering/differential-drive/>

1140. <https://stackoverflow.com/questions/3621244/accounting-for-misalignment-of-wheels-in-diff-drive-odometry>

1141. https://www.roboticsbook.org/S52_diffdrive_actions.html

1142. https://www.reddit.com/r/legotechnic/comments/10vvb6q/how_could_i_build_tank_steering_like_this_with/

1143. <http://ais.informatik.uni-freiburg.de/teaching/ss17/robotics/exercises/solutions/03/sheet03sol.pdf>

1144. <https://www.scribd.com/doc/291891164/Tank-Steering-Systems>

1145. https://control.ros.org/rolling/doc/ros2_controllers/doc/mobile_robot_kinematics.html

1146. <https://www.mathworks.com/help/robotics/ref/differentialdrivekinematicmodel.html>

1147. https://existentialrobotics.org/RobotProvingGrounds/algorithms/control/content/diffy_drive/

1148. https://ethz.ch/content/dam/ethz/special-interest/mavt/dynamic-systems-n-control/idsc-dam/Lectures/amod/Lecture_13/20191104 - ETH - 01 - Modeling.pdf

1149. <https://www.instructables.com/Joystick-to-Differential-Drive-Python/>

1150. <https://groups.google.com/g/lvl1/c/RU4peF8JWZk>

1151. VR-locomotion-parkour-main.slnx

1152. ProjectName.txt

1153. EditorUserSettings.asset

1154. default-6000.dwt

1155. README.md

1156. XRPackageSettings.asset

